LBEADS-NET: Algorithm Unrolling for Learned Baseline Estimation and Denoising in Chromatographic Signals

THESIS

Submitted in Partial Fulfillment of

the Requirements for

the Degree of

MASTER OF SCIENCE (Computer Engineering)

at the

NEW YORK UNIVERSITY
TANDON SCHOOL OF ENGINEERING

by

Saad Zubairi

May 2026

LBEADS-NET: Algorithm Unrolling for Learned Baseline Estimation and Denoising in Chromatographic Signals

THESIS

Submitted in Partial Fulfillment of

the Requirements for

the Degree of

MASTER OF SCIENCE (Computer Engineering)

at the

NEW YORK UNIVERSITY
TANDON SCHOOL OF ENGINEERING

by

Saad Zubairi

May 2026

Approved:

_____

Department Chair Signature

_____

Date

University ID: N14291040
Net ID: shz2020

Approved by the Guidance Committee:

Major: Computer Engineering

Ivan Selesnick
Professor of
Electrical and Computer Engineering

Date

Vita

Saad Zubairi received his Bachelor of Science degree in Computer Science from the Institute of Business Administration (IBA), Karachi, in 2023. He enrolled in the Master of Science program in Computer Engineering at the New York University Tandon School of Engineering in 2024, where he conducted research on algorithm unrolling for chromatographic signal processing under the supervision of Professor Ivan Selesnick.

Acknowledgements

I would like to express my sincere gratitude to my advisor, Professor Ivan Selesnick, for his guidance, expertise, and support throughout this research. His foundational work on BEADS inspired and shaped the direction of this thesis, and his mentorship was invaluable in navigating the challenges of algorithm unrolling.

I am deeply thankful to my parents for their unwavering love, support, and encouragement throughout my academic journey. None of this would have been possible without them.

I owe a special thanks to my friend Ali Hamza, who was with me every step of the way through this degree. His companionship and support made the experience all the more meaningful.

Finally, I thank Izza Babar for her encouragement and belief in me during the final stretch of this work.

Saad Zubairi

May 2026

YOUR DEDICATION (1-2 LINES)

ABSTRACT

LBEADS-NET: Algorithm Unrolling for Learned Baseline Estimation and Denoising in Chromatographic Signals

by

Saad Zubairi

Advisor: Assistant Prof. Ivan Selesnick, Ph.D.

Submitted in Partial Fulfillment of the Requirements for the Degree of Master of Science (Computer Engineering)

May 2026

Baseline drift in chromatographic signals corrupts peak area integration, the standard method for analyte quantification, yet existing correction methods require per-signal parameter tuning (classical optimization) or sacrifice interpretability (deep learning). This thesis proposes LBEADS-NET, a neural network that bridges this gap through *algorithm unrolling*: each iteration of the classical BEADS (Baseline Estimation And Denoising using Sparsity) algorithm is mapped to a differentiable network layer with learnable parameters, creating a 20-layer architecture with 100 learned scalar parameters that inherits the optimization structure of BEADS while gaining adaptivity through end-to-end training. The ISTA-style proximal gradient formulation, adopted after a conjugate gradient variant exhibited

gradient vanishing at effective depth 96, enables stable training across all layers. A multi-stage curriculum manages an 11-term composite loss function designed to suppress *baseline leakage*, a phenomenon in which the ℓ_1 sparsity penalty causes peak energy to be absorbed into the baseline estimate in dense-peak regions. Progressive experiments show that all LBEADS-NET variants outperform classical BEADS with fixed parameters on peak reconstruction error ($\sim$2$\times$ improvement) without per-signal tuning. However, the Baseline Leakage Index plateaus at approximately 0.59 across all loss configurations, revealing a fundamental limit: loss-based mitigations are regularizers on a sparsity-biased architecture, and no amount of loss engineering overcomes the leakage when the sparsity assumption is violated. This finding, corroborated by concurrent work on unrolled sparse-recovery networks, generalizes to any algorithm-unrolled model inheriting ℓ_1 priors. The thesis further contributes analyses of the autograd boundary (non-learnable filter cutoff frequency f_c), training/inference length mismatch, and the over-regularization effect observed when 12 competing loss terms exceed the training budget. Learned parameters exhibit emergent layer specialization, increasing sparsity weight and non-monotonic asymmetry ratio across depth, demonstrating that algorithm unrolling discovers iteration schedules inaccessible to the fixed-parameter classical algorithm.

Contents

Vita iii

Acknowledgements iv

Abstract vi

List of Figures xi

List of Tables xiv

1 Introduction **1**

1.1 The Problem: Baseline Drift in Chromatography 1

1.2 Limitations of Existing Approaches 3

1.3 Contributions: LBEADS-NET 6

1.4 Thesis Outline 8

2 Background and Related Work **10**

2.1 Chromatographic Signal Processing 11

2.2 Classical Baseline Correction Methods 14

2.3 Algorithm Unrolling 26

2.4 Neural Approaches to Baseline Correction 33

3 Proposed Method: LBEADS-NET **38**

3.1 Problem Formulation 39

3.2 From BEADS to BEADSLayer , the ISTA Reformulation 40
3.3 Why ISTA Over Conjugate Gradient 47
3.4 Full K-Layer Pipeline 50
3.5 Non-Learnable Cutoff Frequency and the Autograd Boundary . . . 54
3.6 Multi-Stage Curriculum Training 56
3.7 Composite Loss Function 59
3.8 Hybrid Inference Pipeline 69

4 Experimental Setup **72**

4.1 Synthetic Data Generation 72
4.2 Training Protocol 76
4.3 Evaluation Metrics 80
4.4 Progressive Experiment Design 84
4.5 Baseline Comparison Methods 89

5 Results and Analysis **91**

5.1 Progressive Development Results 92
5.2 Ablation Studies 100
5.3 Qualitative Examples 102
5.4 Training Dynamics 102
5.5 Learned Parameter Analysis 102
5.6 Real Chromatogram Results 105
5.7 Summary of Findings 106

6 Discussion and Limitations **110**

6.1 Baseline Leakage , Diagnosis and Mitigation 111
6.2 Training/Inference Length Mismatch 119

6.3 The Conjugate Gradient Variant's Failure 121
6.4 Non-Learnable Cutoff Frequency and the Autograd Boundary . . . 124
6.5 Softplus Approximation , Train/Inference Gap 127
6.6 The Over-Regularization Problem 128

7 Conclusions and Future Work **131**

A APPENDIX TITLE **132**

List of Figures

List of Tables

2.1 BEADS parameters and their roles. 21

3.1 Learnable parameters per ISTA layer. All parameters are stored in log-space and recovered via exponentiation. Default initial values correspond to the training configuration used in the final model ($K = 20$ layers, 100 total learnable scalars). 41

3.2 Multi-stage curriculum training configuration. Each stage defines a subset of active loss terms and a corresponding epoch budget. Stages are executed sequentially; parameters are carried forward between stages. 58

3.3 Summary of the twelve-term composite loss function. Weight ranges reflect values used across development iterations. The "Stage" column indicates the earliest curriculum stage in which the term is active (A = peak reconstruction, B = leakage mitigation, C = full refinement). Terms marked ", " under "Failure Mode" serve as general priors rather than targeting a specific failure. 69

4.1 Synthetic data generation parameters. All ranges denote uniform or log-uniform distributions as described in the text. 77

4.2 Training hyperparameters. All progressive experiments share this configuration; only the loss function weights differ across experiments. 79

4.3 Loss configurations for the progressive experiment series. Each row shows the active loss terms and their weights. Dashes indicate inactive terms ($\alpha = 0$). Experiment 1.0 uses classical BEADS with no learning. 88

5.1 Progressive experiment results on the 40-signal test set. All values are mean ± standard deviation. Bold entries indicate the best value in each column (excluding Classical BEADS, which uses a fundamentally different evaluation setting). Peak MSE and Baseline MSE are $\times 10^{-3}$ for readability. Neg. Fraction is reported as a proportion of total signal length. 93

5.2 Baseline leakage metrics across the progressive experiment series. BLI measures the directional (upward) leakage of peak energy into the baseline; PBER measures the ratio of baseline error in peak regions to baseline error in background regions. Values closer to zero (BLI) and 1.0 (PBER) indicate less leakage. 97

5.3 Loss term ablation results. Each row removes one group of loss terms from the full configuration (Ablation F). Metrics are reported as mean over the test set. Arrows indicate the direction of improvement; bold indicates best in column. 101

5.4 Network depth ablation. All experiments use the full loss configuration with 60 epochs (10 warmup + 50 full). Total learnable parameters scale linearly as $5K$. 101

5.5 Final learned parameters for selected layers of the 6-layer curriculum-trained model (100 epochs, 3-stage curriculum, signal length $N = 1024$). Parameters that reached clamp boundaries are marked with $*$. 103

Chapter 1

Introduction

1.1 The Problem: Baseline Drift in Chromatography

Chromatography is among the most widely used analytical techniques in chemistry, pharmacology, environmental science, and biochemistry, enabling the separation and quantification of chemical compounds within complex mixtures. A chromatographic signal, commonly referred to as a chromatogram, records detector response as a function of time or elution volume, producing a series of peaks whose areas are proportional to the concentrations of the corresponding analytes [1].

In practice, the raw detector output is corrupted by instrumental and environmental artifacts. The most consequential of these is *baseline drift*: a slowly varying, low-frequency distortion that shifts the signal away from its true zero level. Baseline drift originates from multiple physical sources, including column bleed (thermal degradation of the stationary phase that produces volatile fragments), detector aging and electronic drift, temperature gradients within the chromatographic

column, and compositional changes in the mobile phase during gradient elution [2]. The drift is generally smooth and low-frequency relative to the chromatographic peaks, but its precise shape varies unpredictably across instruments, analytes, and operating conditions.

The observed chromatographic signal can be modeled as a three-component additive decomposition:

$$\mathbf{y} = \mathbf{x} + \mathbf{f} + \mathbf{w}, \tag{1.1}$$

where $\mathbf{y} \in \mathbb{R}^N$ is the observed (measured) signal of length N, $\mathbf{x} \in \mathbb{R}^N$ denotes the *peak component* (the analyte signal of interest), $\mathbf{f} \in \mathbb{R}^N$ represents the *baseline* (the low-frequency drift), and $\mathbf{w} \in \mathbb{R}^N$ captures additive measurement noise. The central goal of baseline correction is to estimate $\mathbf{f}$ from the observed mixture $\mathbf{y}$, thereby recovering the peak signal $\mathbf{x}$ for downstream quantitative analysis.

Accurate baseline estimation is critical because *peak area integration*, the standard method for determining analyte concentration from chromatograms, is directly affected by the estimated baseline level. An overestimated baseline reduces the computed peak area, leading to systematic underestimation of analyte concentration; conversely, an underestimated baseline inflates peak areas and can produce false positives or concentration overestimates. In regulated domains such as pharmaceutical quality control, food safety testing, and clinical diagnostics, even small systematic errors in baseline estimation can compromise the reliability of analytical results [2, 3].

1.2 Limitations of Existing Approaches

A variety of methods have been proposed for baseline estimation in chromatographic and spectroscopic signals, ranging from polynomial fitting and morphological operations to optimization-based formulations and, more recently, deep learning. Despite decades of development, no existing method simultaneously achieves robust generalization across diverse signal conditions and algorithmic interpretability.

1.2.1 Classical Optimization-Based Methods

Among classical approaches, BEADS (Baseline Estimation And Denoising using Sparsity), proposed by Ning et al. [2], is one of the most principled. BEADS formulates baseline correction as a regularized optimization problem that jointly estimates the peak signal and the baseline by exploiting two structural priors: the *sparsity* of the peak signal in the derivative domain (chromatographic peaks occupy a small fraction of the signal) and the *smoothness* of the baseline (drift varies slowly relative to peaks). The formulation integrates a zero-phase Butterworth high-pass filter to enforce frequency-domain separation, first- and second-order total variation penalties to promote sparse derivatives, and an asymmetric penalty controlled by a parameter r that distinguishes positive peaks from negative artifacts.

However, BEADS requires careful specification of six parameters: the sparsity regularization weight λ_0, the first- and second-order smoothness weights λ_1 and λ_2, the asymmetry ratio r, the Butterworth filter cutoff frequency f_c, and the filter order d. In practice, no single parameter configuration generalizes well across instruments, analytes, column conditions, or noise levels. Analysts must tune these parameters for each experimental setup, a tedious and expert-dependent process

that undermines the method's practical utility for high-throughput applications.

Other widely used classical methods share this fundamental limitation. Asymmetric Least Squares (AsLS) [3] and its adaptive extensions, asymmetrically reweighted Penalized Least Squares (arPLS) [4] and adaptive iteratively reweighted Penalized Least Squares (airPLS) [5], require specification of smoothing parameters and asymmetry weights. The Statistics-sensitive Non-linear Iterative Peak-clipping (SNIP) algorithm [6] depends on window size selection. In all cases, optimal parameter values vary across signals, and no automated selection procedure reliably generalizes.

1.2.2 Deep Learning Approaches

Recent work has applied deep learning to baseline correction, demonstrating the capacity of neural networks to learn the signal-to-baseline mapping directly from training data. Kensert et al. [7] trained a 1D convolutional autoencoder on 190,000 synthetic chromatograms, achieving effective baseline removal without per-signal parameter tuning. Chen et al. [8] combined ResNet and U-Net architectures for Raman spectral baseline estimation. More recently, Transformer-based architectures [9] and self-supervised learning frameworks [10] have been proposed for spectroscopic baseline correction, further demonstrating the potential of data-driven approaches.

While these methods eliminate the need for per-signal parameter tuning, they introduce a fundamental trade-off: the learned mapping from input signal to estimated baseline is encoded in millions of opaque network parameters, offering no insight into the decomposition process. The resulting models are "black boxes", one cannot inspect what regularization strategy the network has learned, verify that it

respects physical constraints such as baseline smoothness or peak non-negativity, or diagnose why it fails on a particular signal. This lack of interpretability is a significant limitation in analytical chemistry, where understanding and validating the preprocessing pipeline is often as important as the final result [11, 12].

1.2.3 The Gap

The existing landscape thus presents a dichotomy. Classical methods such as BEADS provide interpretable, physically motivated decompositions but require per-signal parameter tuning and cannot generalize across conditions. Deep learning methods achieve generalization through data-driven training but sacrifice the interpretability and structural transparency that domain scientists require. *No existing method bridges this gap*, combining learned adaptivity (eliminating manual tuning) with algorithmic interpretability (retaining the structure and transparency of classical optimization).

Moreover, a fundamental challenge persists even for methods that might bridge this gap: sparsity-based formulations such as BEADS assume that the peak component $\mathbf{x}$ is sparse, that chromatographic peaks occupy only a small fraction of the signal length. When peaks are densely packed, as is common in metabolomics and complex mixture analysis, this assumption is violated. The ℓ_1 penalty over-shrinks dense peaks, and the surplus energy is absorbed into the baseline estimate, a phenomenon we term *baseline leakage*. This is not an implementation artifact but an inherent limitation of sparsity-based signal decomposition, independently confirmed by concurrent work on unrolled sparse-recovery networks [11] and physics-informed deep learning approaches [12]. Any learned baseline correction system built on sparse decomposition principles must therefore confront this challenge directly.

Algorithm unrolling [13, 14] offers a principled framework for closing the interpretability–generalization gap. By mapping each iteration of an iterative optimization algorithm to a neural network layer with learnable parameters, algorithm unrolling creates architectures that inherit the interpretability and inductive biases of the original algorithm while gaining the adaptivity of end-to-end training. This approach has been successfully applied in compressed sensing [15], medical image reconstruction [16], and other inverse problems, but has seen only limited exploration for chromatographic baseline correction [11].

1.3 Contributions: LBEADS-NET

This thesis proposes **LBEADS-NET** (**L**earned **B**aseline **E**stimation **A**nd **D**enoising using **S**parsity **Net**work), a neural network architecture that bridges the interpretability of classical optimization-based baseline correction with the adaptivity of data-driven learning. The central idea is *algorithm unrolling*: each iteration of the classical BEADS algorithm is mapped to a differentiable neural network layer with learnable parameters, creating a K-layer deep network whose architecture directly encodes the optimization structure of BEADS while allowing end-to-end training via backpropagation.

The principal contributions of this thesis are:

1. **Algorithm unrolling of BEADS.** We unroll the BEADS iterative optimization algorithm into K=20 differentiable ISTA-style layers, each with independently learnable regularization parameters λ_0, λ_1, λ_2, asymmetry ratio r, and step size η, 100 learned scalar parameters in total. A log-space parameterization ($\lambda = \exp(\log \lambda)$) guarantees positivity of all regularization

weights. The per-layer independence enables *emergent parameter specialization*, early layers learn coarse peak-baseline separation while later layers refine the decomposition, without explicit design at the architectural level.

2. **Multi-stage curriculum training with anti-leakage composite loss.** We introduce a three-stage curriculum training strategy to manage an 11-term composite loss function. The loss is designed with two objectives: training accurate signal decomposition and actively suppressing baseline leakage. Key anti-leakage terms include an asymmetric baseline penalty that penalizes over-estimation 9× more than under-estimation, an element-wise orthogonality penalty that enforces mutual exclusivity of peak and baseline components, and an envelope constraint that prevents the baseline from exceeding local signal minima. The curriculum progresses from pure reconstruction (Stage A) through structured separation with anti-leakage losses (Stage B) to full-loss refinement (Stage C), stabilizing optimization of an otherwise intractable multi-objective landscape.

3. **Systematic diagnosis and mitigation of baseline leakage.** We identify three root causes of baseline leakage in BEADS, sparsity assumption violation in dense-peak regions, low-pass filter bandwidth mismatch, and spatially uniform regularization, and document the systematic mitigation effort across six development iterations. The resulting analysis reveals a fundamental insight: all loss-based mitigations are regularizers on top of a sparsity-biased formulation, and no amount of loss engineering fully overcomes the leakage when the sparsity assumption is violated. This finding generalizes to any unrolled network inheriting ℓ_1-based sparsity priors, as independently

corroborated by concurrent work [11, 12].

4. **Failure mode analysis.** We provide detailed analyses of four additional failure modes: (i) gradient vanishing in the conjugate gradient solver variant, motivating the ISTA-style reformulation; (ii) non-learnability of the Butterworth filter cutoff frequency f_c due to the autograd boundary between SciPy and PyTorch; (iii) training/inference signal length mismatch; and (iv) softplus approximation artifacts at inference. These findings yield design principles for the algorithm unrolling community, in particular, that the inner solver's gradient properties determine whether an iterative algorithm can be successfully unrolled.

5. **Hybrid inference pipeline.** We develop a hybrid inference pipeline that uses the trained LBEADS-NET as a learned initialization for classical BEADS refinement, combined with quality-scored stage selection. This design leverages the network's learned adaptivity for fast approximate decomposition while retaining the convergence guarantees of classical iterative optimization for final refinement.

1.4 Thesis Outline

The remainder of this thesis is organized as follows. A central thread running through the method design, evaluation, and discussion is the baseline leakage problem, its root causes, the engineering effort to suppress it, and the fundamental limits of that effort. Chapter 2 provides the necessary background, covering chromatographic signal processing, the classical BEADS algorithm in full mathematical detail including its failure modes in dense-peak regions, the algorithm unrolling

framework and its origins in LISTA [13], and existing neural approaches to baseline correction. Chapter 3 presents the proposed LBEADS-NET architecture, including the mapping from BEADS iterations to differentiable BEADSLayers, the ISTA-style fast variant that resolved gradient vanishing in the conjugate gradient formulation, the multi-stage curriculum training strategy, the eleven-term composite loss function and its anti-leakage design, and the hybrid inference pipeline. Chapter 4 describes the experimental setup: synthetic data generation, the training protocol, evaluation metrics, including leakage-specific measures such as the Baseline Leakage Index and stratified sparse/dense-region evaluation, and baseline comparison methods. Chapter 5 presents quantitative and qualitative results on both synthetic and real chromatographic data, including analysis of learned parameter evolution across layers, baseline leakage metrics, and hybrid pipeline performance. Chapter 6 discusses the principal limitations, with baseline leakage as the centerpiece: the mitigation journey across six development iterations, the fundamental insight that sparsity-biased formulations have inherent limits in dense regimes, and additional failure modes including the conjugate gradient variant's gradient collapse, the non-learnable cutoff frequency, and train/inference gaps. Chapter 7 summarizes the contributions, argues for the generalizability of the approach, and outlines concrete directions for future work motivated by the leakage analysis.

Chapter 2

Background and Related Work

This chapter provides the technical foundation for the LBEADS-NET architecture developed in Chapter 3. Section 2.1 reviews the chromatographic signal model and the physical origins of baseline drift. Section 2.2 presents the classical BEADS algorithm in full mathematical detail, identifies its failure modes in dense-peak regions, and surveys other penalized least squares methods. Section 2.3 introduces the algorithm unrolling framework and its key architectures. Finally, Section 2.4 reviews neural approaches to baseline correction and positions LBEADS-NET relative to existing methods.

2.1 Chromatographic Signal Processing

2.1.1 Signal Model

As introduced in Section 1.1, the observed chromatographic signal is modeled as the three-component additive decomposition

$$\mathbf{y} = \mathbf{x} + \mathbf{f} + \mathbf{w}, \tag{1.1}$$

where $\mathbf{y} \in \mathbb{R}^N$ is the measured detector response, $\mathbf{x} \in \mathbb{R}^N$ is the peak component containing analyte information, $\mathbf{f} \in \mathbb{R}^N$ is the baseline drift, and $\mathbf{w} \in \mathbb{R}^N$ is additive measurement noise [2]. The baseline correction problem is to estimate $\mathbf{f}$ from $\mathbf{y}$ so that the corrected signal $\hat{\mathbf{x}} = \mathbf{y} - \hat{\mathbf{f}}$ faithfully represents the true peak component.

Two structural priors underpin optimization-based approaches to this problem. First, the peak component $\mathbf{x}$ is assumed to be *sparse* in some domain: chromatographic peaks occupy a small fraction of the signal length, and their derivatives are concentrated around peak onsets and offsets. Second, the baseline $\mathbf{f}$ is assumed to be *smooth*: it varies slowly relative to the peaks, occupying a low-frequency subspace of the signal. These complementary assumptions enable frequency-domain and sparsity-based separation of $\mathbf{x}$ and $\mathbf{f}$ [2].

2.1.2 Physical Causes of Baseline Drift

Baseline drift arises from multiple instrumental and environmental sources. *Column bleed* results from thermal degradation of the stationary phase, producing volatile fragments that generate a slowly rising detector response. *Detector aging and electronic drift* introduce low-frequency shifts in the detector's zero level over

the course of an analytical run. *Temperature gradients* within the chromatographic column alter analyte retention and detector response characteristics. During gradient elution, compositional changes in the *mobile phase* produce systematic shifts in detector background, particularly in UV-Vis and refractive index detection [1, 2].

The drift is generally smooth and low-frequency relative to the chromatographic peaks, but its precise shape is unpredictable: it varies across instruments, columns, analyte systems, and operating conditions. This variability is the fundamental reason that no single fixed set of baseline correction parameters generalizes across experimental setups.

2.1.3 Importance of Accurate Baseline Estimation

Accurate baseline estimation is critical for quantitative chromatographic analysis. The standard method for determining analyte concentration is *peak area integration*: the area between the corrected signal and the zero level is proportional to the mass of analyte present. Any error in the estimated baseline directly propagates to the integrated area. An overestimated baseline reduces computed peak areas, leading to systematic underestimation of analyte concentrations. Conversely, an underestimated baseline inflates peak areas and can produce false positives or concentration overestimates [2, 3].

Beyond integration accuracy, baseline errors affect the *limit of detection* (LOD), the smallest analyte concentration that can be reliably distinguished from noise. A poorly estimated baseline increases the apparent noise floor, raising the LOD and reducing the sensitivity of the analytical method. In regulated domains such as pharmaceutical quality control, food safety testing, and clinical diagnostics, even small systematic errors in baseline estimation can compromise the reliability of

analytical results.

2.1.4 Peak Shape Models

The shape of chromatographic peaks is important for understanding the signal structure that baseline correction methods must accommodate. Under idealized conditions, chromatographic theory predicts Gaussian peak profiles. In practice, however, extra-column effects, including detector dead volume, connecting tubing, and mixing chambers, introduce exponential tailing.

Grushka [17] defined the *exponentially modified Gaussian* (EMG) peak as the convolution of a Gaussian function with an exponential decay, providing a theoretically justified model for chromatographic peak shapes that accounts for extra-column band broadening. The EMG peak shape is parameterized by the Gaussian standard deviation σ, the retention time t_R, and the exponential time constant τ, where the ratio τ/σ controls the degree of peak asymmetry [17]. The skewness ranges from 0 (pure Gaussian, $\tau/\sigma = 0$) to 2.0 (pure exponential decay, $\tau/\sigma \to \infty$).

Naish and Hartwell [18] validated the EMG function as a good model for isocratic HPLC peaks by least-squares fitting the full EMG expression to over 100 experimental peaks spanning asymmetry ratios from 1.03 to 1.25. They showed that least-squares fitting of EMG functions to experimental LC peaks produced close agreement across the full peak profile, whereas fitted Gaussian functions produced large errors particularly in the tail region, even for peaks with low skew [18]. The EMG model is widely used as a realistic synthetic peak shape for generating training data in chromatographic signal processing, owing to its theoretical justification and ability to reproduce the asymmetric tailing commonly

observed in real chromatograms [17].

2.2 Classical Baseline Correction Methods

2.2.1 BEADS: Baseline Estimation And Denoising using Sparsity

BEADS, proposed by Ning et al. [2], formulates baseline correction as a convex optimization problem that jointly estimates the peak signal $\mathbf{x}$ and the baseline $\mathbf{f}$ by exploiting the sparsity of $\mathbf{x}$ and the smoothness of $\mathbf{f}$. This section presents the BEADS formulation in full mathematical detail, as it forms the algorithmic foundation that LBEADS-NET unrolls in Chapter 3.

2.2.1.1 The BEADS Cost Function

BEADS models the baseline as a low-pass signal rather than a parametric function such as a polynomial, providing a more generic representation that avoids the limitations of polynomial fitting over long ranges [2]. Let H denote a zero-phase high-pass filter operator. Given the signal model $\mathbf{y} = \mathbf{x} + \mathbf{f} + \mathbf{w}$, the high-pass filtered residual $H(\mathbf{y} - \mathbf{x})$ removes the low-frequency baseline and retains only the noise component. The BEADS cost function is

$$J(\mathbf{x}) = \frac{1}{2} \|H(\mathbf{y} - \mathbf{x})\|_2^2 + \lambda_0 \sum_{n=0}^{N-1} \theta(x_n) + \lambda_1 \sum_n \phi(\Delta x_n) + \lambda_2 \sum_n \phi(\Delta^2 x_n), \quad (2.1)$$

where Δ and Δ^2 denote the first and second difference operators, respectively. The four terms serve distinct purposes:

1. **Data fidelity**: $\frac{1}{2} \|H(\mathbf{y} - \mathbf{x})\|_2^2$ measures how well the high-pass filtered

residual matches zero-mean noise. By filtering through H, the smooth baseline component is annihilated, and the fidelity term depends only on the peak estimate and the noise.

2. **Asymmetric sparsity penalty**: $\lambda_0 \sum_n \theta(x_n)$ promotes sparsity in $\mathbf{x}$ while enforcing approximate non-negativity. The asymmetric penalty function θ penalizes negative values of x_n more heavily than positive values via a parameter $r > 1$:

$$\theta(x) = \begin{cases} x, & x > \epsilon_0, \\ -r\,x, & x < -\epsilon_0, \\ \frac{1+r}{4\epsilon_0}x^2 + \frac{1-r}{2}x + \frac{\epsilon_0(1+r)}{4}, & |x| \le \epsilon_0, \end{cases} \tag{2.2}$$

where ϵ_0 is a small constant that smooths the transition at zero. For positive peaks, θ behaves as an ℓ_1 penalty; for negative values, the penalty is amplified by the factor r, discouraging artifacts below zero.

3. **First-order smoothness**: $\lambda_1 \sum_n \phi(\Delta x_n)$ penalizes the first differences of $\mathbf{x}$, promoting piecewise-constant behavior.

4. **Second-order smoothness**: $\lambda_2 \sum_n \phi(\Delta^2 x_n)$ penalizes the second differences of $\mathbf{x}$, promoting piecewise-linear behavior.

The penalty function ϕ is a smoothed approximation of the absolute value to ensure differentiability. BEADS uses the differentiable penalty

$$\phi(z) = |z| - \epsilon_1 \log(|z| + \epsilon_1), \tag{2.3}$$

where ϵ_1 is a small positive constant. This avoids the numerical issues that arise from the non-differentiable ℓ_1 norm at zero [2].

2.2.1.2 High-Pass Filter Construction

The high-pass filter H is constructed as a zero-phase Butterworth filter in banded matrix form. The construction proceeds as follows. Let d denote the filter degree (the filter order is $2d$), and let f_c denote the normalized cutoff frequency ($0 < f_c < 0.5$). Define the difference kernel

$$b_1[n] = [1,\ -1] \underbrace{*[-1,\ 2,\ -1] * \cdots * [-1,\ 2,\ -1]}_{d-1 \text{ times}}, \tag{2.4}$$

where $*$ denotes convolution. The numerator coefficients are then

$$\mathbf{b} = b_1 * [-1,\ 1], \tag{2.5}$$

yielding a vector of length $2d + 1$. The denominator coefficients incorporate the Butterworth pole placement. Define the frequency-dependent parameter

$$t = \left(\frac{1 - \cos(\omega_c)}{1 + \cos(\omega_c)} \right)^d, \quad \omega_c = 2\pi f_c. \tag{2.6}$$

The low-pass component is obtained by repeated convolution:

$$\mathbf{a}_{\text{lp}} = \underbrace{[1,\ 2,\ 1] * [1,\ 2,\ 1] * \cdots * [1,\ 2,\ 1]}_{d \text{ times}}, \tag{2.7}$$

and the combined filter coefficients are

$$\mathbf{a} = \mathbf{b} + t \cdot \mathbf{a}_{\text{lp}}. \tag{2.8}$$

The banded matrices A and B are $N \times N$ Toeplitz matrices with the coefficients $\mathbf{a}$ and $\mathbf{b}$ placed along diagonals $-d, \ldots, d$:

$$A = \text{BandedToeplitz}(\mathbf{a}), \quad B = \text{BandedToeplitz}(\mathbf{b}). \tag{2.9}$$

The high-pass filter operator is then

$$H = BA^{-1}, \tag{2.10}$$

so that for any vector $\mathbf{v}$, the high-pass filtered output is $H\mathbf{v} = B(A^{-1}\mathbf{v})$ [2]. The banded structure of A ensures that the linear system $A\mathbf{z} = \mathbf{v}$ can be solved in $O(N)$ time. BEADS leverages this banded structure to achieve computational efficiency and low memory usage, enabling application to long data series [2].

2.2.1.3 Majorization-Minimization Algorithm

The cost function in Equation 2.1 is minimized using a majorization-minimization (MM) iterative scheme that is guaranteed to converge regardless of initialization [2]. The key idea is to replace the non-quadratic penalties ϕ and θ with quadratic upper bounds at each iteration, reducing the update to a banded linear system solve.

Define the first- and second-order difference matrices $D_1 \in \mathbb{R}^{(N-1)\times N}$ and

$D_2 \in \mathbb{R}^{(N-2)\times N}$:

$$(D_1)_{i,j} = \begin{cases} -1, & j = i, \\ 1, & j = i+1, \\ 0, & \text{otherwise}, \end{cases} \qquad (D_2)_{i,j} = \begin{cases} 1, & j = i, \\ -2, & j = i+1, \\ 1, & j = i+2, \\ 0, & \text{otherwise}. \end{cases} \tag{2.11}$$

Let $D = [D_1^\top, D_2^\top]^\top$ denote the stacked difference matrix of size $(2N-3) \times N$. Define the weight vector $\mathbf{w} = [\lambda_1 \mathbf{1}_{N-1}^\top,\ \lambda_2 \mathbf{1}_{N-2}^\top]^\top$.

At iteration k, the algorithm computes diagonal reweighting matrices from the current estimate $\mathbf{x}^{(k)}$:

Smoothness reweighting.

$$\Lambda^{(k)} = \operatorname{diag}\big(\mathbf{w} \odot \psi(D\mathbf{x}^{(k)})\big), \quad \text{where } \psi(z) = \frac{1}{|z| + \epsilon_1}. \tag{2.12}$$

Sparsity reweighting.

$$\gamma_n^{(k)} = \begin{cases} \dfrac{(1+r)}{4\,|x_n^{(k)}|}, & |x_n^{(k)}| > \epsilon_0, \\ \dfrac{(1+r)}{4\,\epsilon_0}, & |x_n^{(k)}| \le \epsilon_0, \end{cases} \qquad \Gamma^{(k)} = \operatorname{diag}(\gamma^{(k)}). \tag{2.13}$$

The majorized problem yields the linear system

$$\big(B^\top B + A^\top M^{(k)} A\big)\mathbf{z} = \mathbf{d}, \tag{2.14}$$

where

$$M^{(k)} = 2\lambda_0 \Gamma^{(k)} + D^\top \Lambda^{(k)} D, \tag{2.15}$$

and the right-hand side is

$$\mathbf{d} = B^\top B(A^{-1}\mathbf{y}) - \lambda_0 A^\top \mathbf{b}_r, \quad \mathbf{b}_r = \frac{1-r}{2}\mathbf{1}_N. \tag{2.16}$$

Note that $\mathbf{d}$ is constant across iterations and can be precomputed. The peak estimate is then updated as

$$\mathbf{x}^{(k+1)} = A\mathbf{z}^{(k)}, \tag{2.17}$$

where $\mathbf{z}^{(k)}$ is the solution of Equation 2.14. After convergence (typically approximately 20 iterations [2]), the baseline is recovered as

$$\hat{\mathbf{f}} = \mathbf{y} - \hat{\mathbf{x}} - H(\mathbf{y} - \hat{\mathbf{x}}). \tag{2.18}$$

Algorithm 1 summarizes the complete BEADS algorithm.

2.2.1.4 Parameters and Their Roles

BEADS requires careful specification of six parameters. Table 2.1 summarizes each parameter, its role, and typical considerations for tuning.

Ning et al. noted that the regularization parameters λ_i should be chosen inversely proportional to the ℓ_1 norm of the i-th derivative of the signal, and a proportionality constant should be tuned according to the noise variance [2]. In practice, no single parameter configuration generalizes well across instruments, analytes, column conditions, or noise levels. Analysts must tune these parameters for each experimental setup, a tedious and expert-dependent process that undermines the method's practical utility for high-throughput applications [2].

Algorithm 1 Classical BEADS Algorithm

Require: Observed signal $\mathbf{y} \in \mathbb{R}^N$; parameters d, f_c, r, λ_0, λ_1, λ_2; number of iterations K

Ensure: Peak estimate $\hat{\mathbf{x}}$, baseline estimate $\hat{\mathbf{f}}$

Construct banded matrices A, B from d and f_c (Equation 2.4–Equation 2.9)
Build difference matrices D_1, D_2; stack $D = [D_1; D_2]$
Compute weight vector $\mathbf{w} = [\lambda_1 \mathbf{1}_{N-1};\ \lambda_2 \mathbf{1}_{N-2}]$
Precompute $\mathbf{d} = B^\top B(A^{-1}\mathbf{y}) - \lambda_0 A^\top \left(\frac{1-r}{2}\mathbf{1}_N\right)$
Initialize $\mathbf{x}^{(0)} \leftarrow \mathbf{y}$
for $k = 0, 1, \ldots, K-1$ **do**
    $\Lambda^{(k)} \leftarrow \mathrm{diag}\big(\mathbf{w} \odot \psi(D\mathbf{x}^{(k)})\big)$ (smoothness weights)
    Compute $\Gamma^{(k)}$ from $\mathbf{x}^{(k)}$ (sparsity weights, Equation 2.13)
    $M^{(k)} \leftarrow 2\lambda_0 \Gamma^{(k)} + D^\top \Lambda^{(k)} D$
    Solve $(B^\top B + A^\top M^{(k)} A)\,\mathbf{z} = \mathbf{d}$ (banded linear system)
    $\mathbf{x}^{(k+1)} \leftarrow A\mathbf{z}$
end for
$\hat{\mathbf{x}} \leftarrow \mathbf{x}^{(K)}$
$\hat{\mathbf{f}} \leftarrow \mathbf{y} - \hat{\mathbf{x}} - H(\mathbf{y} - \hat{\mathbf{x}})$ (baseline via low-pass residual)
return $\hat{\mathbf{x}}$, $\hat{\mathbf{f}}$

2.2.1.5 Failure Modes in Dense-Peak Regions

While BEADS performs well when the sparsity assumption holds, *i.e.*, when peaks are well-separated and occupy a small fraction of the signal, it exhibits systematic failure in dense-peak regions. Three root causes drive these failures:

Failure Mode 1: Sparsity Assumption Violation. The ℓ_1 penalty $\lambda_0 \sum_n \theta(x_n)$ assumes that most entries of $\mathbf{x}$ are zero. In dense-peak regions, common in metabolomics, complex mixture analysis, and gradient elution chromatography, peaks overlap and occupy a large fraction of the signal. The penalty over-shrinks the aggregate peak energy, and the surplus is absorbed into the baseline estimate $\hat{\mathbf{f}}$, which rises to compensate. We term this phenomenon *baseline leakage*: the estimated baseline rises into the true peak region, systematically overestimating the baseline and underestimating peak areas.

Table 2.1: BEADS parameters and their roles.

Parameter	**Symbol**	**Role**
Sparsity weight	λ_0	Controls the strength of the asymmetric sparsity penalty on $\mathbf{x}$. Larger values enforce sparser peak estimates.
First-order weight	λ_1	Penalizes the first differences of $\mathbf{x}$, promoting piecewise-constant signals.
Second-order weight	λ_2	Penalizes the second differences of $\mathbf{x}$, promoting piecewise-linear signals.
Asymmetry ratio	r	Controls the ratio of negative-to-positive penalty in θ. Larger r more strongly suppresses negative signal values.
Filter cutoff	f_c	Normalized cutoff frequency of the Butterworth high-pass filter. Determines the boundary between "peak" and "baseline" frequency content.
Filter degree	d	Degree of the Butterworth filter (order = $2d$). Higher d produces a sharper frequency transition.

Failure Mode 2: Low-Pass Filter Bandwidth Mismatch. The Butterworth filter cutoff f_c defines a hard boundary between frequencies assigned to the baseline and those assigned to the peaks. When broad peaks have significant spectral energy near or below f_c, the high-pass filter removes part of the peak energy, attributing it to the baseline. Conversely, if f_c is set too low to preserve broad peaks, rapidly varying baseline features are not removed. No single f_c simultaneously accommodates narrow peaks, broad peaks, and the baseline in the same signal.

Failure Mode 3: Spatially Uniform Regularization. The weights λ_0, λ_1, and λ_2 are constant across all signal positions. A sparse region with well-separated peaks requires strong regularization to enforce sparsity, while a dense region requires weaker regularization to preserve the aggregate peak signal. Uniform weights cannot adapt to local signal density, leading to over-regularization in dense regions and

under-regularization in sparse regions within the same signal.

These three failure modes are not independent: they compound in dense-peak signals, making BEADS unreliable precisely in the analytically challenging scenarios where accurate baseline correction is most needed. This observation motivates the development of LBEADS-NET, which learns per-layer parameters through end-to-end training rather than relying on a single fixed parameter set.

2.2.2 Other Classical Methods

Several other methods for baseline estimation predate or complement BEADS. Understanding their strengths and limitations clarifies why BEADS is the most suitable candidate for algorithm unrolling.

2.2.2.1 Asymmetric Least Squares (AsLS)

Eilers [19] introduced the Whittaker smoother to the analytical chemistry community as a penalized least squares method that balances fidelity to data against roughness of the fitted curve, controlled by a single smoothing parameter λ. The Whittaker smoother solves the linear system $(I + \lambda D^\top D)\mathbf{z} = \mathbf{y}$, where D is a difference matrix of order d and λ controls the trade-off between data fidelity and smoothness [19]. Eilers showed that incorporating a diagonal weight matrix W into the penalized least squares framework yields the system $(W + \lambda D^\top D)\mathbf{z} = W\mathbf{y}$, enabling asymmetric fitting [19].

Eilers and Boelens [3] extended this framework with asymmetric weights for baseline estimation (AsLS), assigning different penalties to positive and negative residuals. Points above the current estimate are given lower weight (presumed to be peaks), while points below receive higher weight (presumed to be baseline). This

iterative reweighting procedure produces a smooth curve that preferentially fits below the signal peaks.

2.2.2.2 airPLS

Zhang et al. [5] proposed the airPLS (adaptive iteratively reweighted penalized least squares) algorithm, which adaptively reweights the penalized least squares baseline estimation by setting weights to zero where the signal exceeds the current baseline estimate and using exponential weights proportional to the residual magnitude where the signal is below the estimate [5]. The airPLS algorithm eliminates the need for user-specified asymmetry parameters, requiring only a single smoothness parameter λ [5]. Zhang et al. demonstrated that the airPLS algorithm converges in only a few iterations (typically 3 to 6) due to its exponential reweighting strategy [5].

2.2.2.3 arPLS

Baek et al. [4] proposed the arPLS (asymmetrically reweighted penalized least squares) method, which uses a generalized logistic function to assign weights that give similar importance to signals slightly above and below the estimated baseline, thereby avoiding the baseline underestimation problem of earlier methods [4]. The arPLS weighting scheme uses the mean and standard deviation of negative residuals to parameterize a generalized logistic function that smoothly transitions weights from one (baseline region) to zero (peak region) [4]. Baek et al. showed that arPLS is relatively robust to the choice of the smoothness parameter λ, maintaining low RMSE over a range spanning more than two orders of magnitude [4].

Baek et al. showed that AsLS and airPLS methods systematically underestimate the baseline in non-peak regions because they assign zero or near-zero weight

to all signals above the fitted baseline, causing downward bias and peak height overestimation [4]. The arPLS method addresses this by providing a smooth transition in the weighting function.

2.2.2.4 SNIP

Ryan et al. [20] proposed the Statistics-sensitive Non-linear Iterative Peak-clipping (SNIP) algorithm, which estimates the background of a spectrum by iteratively replacing each channel value with the lesser of itself or the mean of two symmetrically placed neighbors [20]. SNIP compresses the dynamic range using a double log transformation prior to peak clipping, which reduces the required number of iterations from approximately 700 to approximately 24 [20]. SNIP is an early and influential iterative baseline estimation method for spectral data, originally developed for PIXE X-ray spectra but subsequently adopted in other spectroscopy and signal processing domains [20]. Unlike the penalized least squares family, SNIP does not use an optimization framework and does not produce a smooth baseline in the penalized-regression sense.

2.2.2.5 Why BEADS is the Best Unrolling Candidate

Among the classical methods surveyed above, BEADS is uniquely suited for algorithm unrolling for several reasons:

1. **Explicit iterative structure.** BEADS is formulated as a majorization-minimization algorithm with clearly defined per-iteration update equations. Each iteration involves a linear system solve, a reweighting step, and an implicit proximal operation, operations that map naturally to neural network layers.

2. **Learnable scalar parameters.** The six BEADS parameters (λ_0, λ_1, λ_2, r, f_c, d) are scalar quantities with clear physical interpretations. Making them learnable per-layer requires only $O(K)$ parameters for K layers, preserving the interpretability that generic deep networks sacrifice.

3. **Joint estimation.** BEADS jointly estimates $\mathbf{x}$ and $\mathbf{f}$ through its optimization structure, unlike PLS methods that estimate only the baseline. This joint formulation carries through to the unrolled network.

4. **Structural priors.** The asymmetric penalty, the sparsity prior, and the Butterworth filter encode domain-specific knowledge about chromatographic signals. An unrolled network inherits these inductive biases, requiring less training data than a black-box architecture.

2.2.2.6 BEADS vs. arPLS Failure Mode Contrast

An important contrast between BEADS and arPLS illuminates the nature of the baseline leakage problem. The arPLS method makes no sparsity assumption on the peak signal: it assumes only that the baseline lies below the signal. In dense-peak regions, arPLS tends to produce an *elevated* baseline that follows the lower envelope of the dense peaks, a failure mode we term *baseline elevation*. The estimated baseline is too high because the method has no mechanism to distinguish between the true baseline and the bottom of densely packed peaks.

BEADS, by contrast, fails in the opposite direction via *baseline leakage*: the sparsity penalty forces the peak estimate to be sparser than the true signal, and the excess energy is absorbed into the baseline. The baseline rises not because it follows the peak envelope (as in arPLS), but because the optimization attributes

dense-peak energy to the baseline component when the ℓ_1 penalty over-shrinks.

This distinction is important for the thesis narrative: when BEADS is unrolled into LBEADS-NET, the sparsity-induced leakage mechanism is inherited by the network architecture. Mitigating this failure mode is therefore a central challenge in the design and training of LBEADS-NET, addressed in detail in Chapter 3.

2.3 Algorithm Unrolling

2.3.1 Core Idea

Algorithm unrolling (or unfolding) provides a concrete and systematic connection between iterative algorithms used in signal processing and deep neural network architectures [14]. The central idea is to map each iteration of an iterative optimization algorithm to one layer of a deep network. Concatenating K such layers forms a neural network where passing through the network is equivalent to executing the iterative algorithm K times [14]. Algorithm parameters such as regularization coefficients and step sizes transfer to learnable network parameters and can be optimized through end-to-end training with backpropagation [14].

Consider a generic iterative algorithm that updates a solution estimate $\mathbf{x}^{(k+1)} = \mathcal{T}(\mathbf{x}^{(k)}; \boldsymbol{\theta})$, where $\boldsymbol{\theta}$ denotes the algorithm's parameters. In classical use, $\boldsymbol{\theta}$ is fixed across all iterations and tuned manually. Algorithm unrolling introduces two modifications:

1. **Truncation**: The iteration is run for a fixed number K of steps, producing a K-layer feed-forward network rather than iterating to convergence.

2. **Parameter untying**: Each layer k receives its own parameters $\boldsymbol{\theta}^{(k)}$, allowing

the network to learn iteration-specific behavior. The unrolled update becomes

$$\mathbf{x}^{(k+1)} = \mathcal{T}(\mathbf{x}^{(k)}; \boldsymbol{\theta}^{(k)}), \quad k = 0, 1, \ldots, K-1. \tag{2.19}$$

The entire network is trained end-to-end by minimizing a task-specific loss $\mathcal{L}(\mathbf{x}^{(K)}, \mathbf{x}^*)$ with respect to $\{\boldsymbol{\theta}^{(k)}\}_{k=0}^{K-1}$, where $\mathbf{x}^*$ is the ground-truth signal. Layer-specific (untied) parameters allow a departure from the original iterative algorithm, enhancing representation capacity and boosting performance, though the networks may not completely inherit convergence guarantees of the original algorithm [14].

2.3.2 LISTA: The Founding Work

Gregor and LeCun [13] introduced LISTA (Learned ISTA), the foundational algorithm unrolling method that unfolds a fixed number of ISTA iterations into a feed-forward neural network with learned parameters, achieving roughly 20 times faster convergence than FISTA for approximate sparse coding [13]. LISTA replaces the matrices W_e and S in ISTA, which are normally computed from the dictionary and sparsity parameters, with learned matrices optimized end-to-end via backpropagation to minimize the squared error between predicted and optimal sparse codes [13].

The ISTA update for sparse coding takes the form

$$\mathbf{x}^{(k+1)} = h_\alpha\big(W_e\mathbf{y} + S\mathbf{x}^{(k)}\big), \tag{2.20}$$

where h_α is the soft-thresholding operator with threshold α, W_e is a filter matrix, and S is a mutual inhibition matrix. In LISTA, W_e, S, and α are all learned from

data. Gregor and LeCun established the paradigm of algorithm unrolling, where an iterative optimization algorithm is truncated to a fixed depth and its parameters are learned end-to-end, trading generality for speed on a specific data distribution [13].

The significance of LISTA extends beyond its specific application to sparse coding. It demonstrated that learning the iteration parameters dramatically improves per-iteration efficiency: LISTA with a single iteration achieved prediction error of 1.50 for 100-unit codes, compared to 21.0 for FISTA with one iteration [13]. This result showed that it takes 18 iterations of FISTA to reach the same error as 1 iteration of LISTA [13], establishing that learned parameters can compensate for truncation.

2.3.3 Why Algorithm Unrolling Works

Algorithm unrolling derives its effectiveness from three properties that distinguish it from both classical iterative methods and generic deep networks:

Interpretability. The trained unrolled network can be naturally interpreted as a parameter-optimized algorithm, effectively overcoming the lack of interpretability in most conventional neural networks [14]. Each layer corresponds to a well-understood algorithmic step, and the learned parameters retain their physical meaning.

Parameter efficiency. Unrolled networks are highly parameter efficient compared to generic deep networks because they encode domain knowledge through unrolling, requiring less training data and generalizing better especially under limited training data regimes [14]. Rather than learning arbitrary transformations, the network learns only the parameters of a known-good algorithm.

Inductive bias. Unrolled networks naturally inherit prior structures and domain knowledge rather than learning them from intensive training data, and consequently

tend to generalize better than generic networks [14]. The architectural constraints imposed by the algorithm structure act as a strong regularizer, preventing the network from overfitting to spurious patterns.

From a functional approximation perspective, unrolled networks occupy an intermediate position between generic neural networks and iterative algorithms, offering relatively low bias and variance simultaneously [14].

2.3.4 Key Architectures

Following LISTA, a rich family of unrolled architectures has been developed. We briefly survey the most relevant ones.

ALISTA (Liu et al., 2019 [21]). ALISTA computes the weight matrix in LISTA analytically via a data-free coherence minimization problem, leaving only layer-wise step sizes and thresholds to be learned from data [21]. This reduces the number of learnable parameters from $O(KNM + K)$ in LISTA to $O(K)$ [21]. ALISTA demonstrated that learning only scalar parameters per layer achieves performance comparable to full LISTA models that learn entire weight matrices [21]. This result is particularly relevant to LBEADS-NET, which similarly learns only scalar parameters per layer.

ISTA-Net and ISTA-Net+ (Zhang and Ghanem, 2018 [15]). ISTA-Net maps the iterative update steps of ISTA into a fixed number of network phases for image compressive sensing [15]. Zhang and Ghanem replaced the hand-crafted linear sparsifying transform with a learnable nonlinear transform composed of two convolutional operators separated by a ReLU, and introduced a corresponding learned inverse transform [15]. ISTA-Net+ operates in the residual domain by explicitly extracting high-frequency residual components, introducing skip connections analo-

gous to ResNet [15]. ISTA-Net can be viewed as a significant extension of LISTA from the sparse coding problem to general compressive sensing reconstruction [15].

ISTA-Net++ (You et al., 2021 [22]). ISTA-Net++ introduces a dynamic unfolding strategy that handles multiple compressive sensing ratios through a single trained model by incorporating a condition module that transmits ratio-dependent step size and noise level parameters to each unfolded stage [22]. ISTA-Net++ uses a dynamic proximal mapping module consisting of residual blocks, replacing the hand-crafted proximal operator with a learned nonlinear mapping [22].

Hybrid ISTA (Zheng et al., 2022 [23]). Hybrid ISTA incorporates free-form deep neural networks into ISTA-based unfolded algorithms while maintaining theoretical convergence guarantees [23]. It addresses a fundamental tension in algorithm unrolling: existing convergence-provable methods restrict network architectures via partial weight coupling, while flexible methods lack convergence guarantees [23]. Hybrid ISTA integrates classical ISTA with free-form DNNs using a balancing parameter, ensuring convergence equivalent to ISTA in the worst case [23].

ADMM-Net (Yang et al., 2016 [24]). ADMM-Net unrolls the Alternating Direction Method of Multipliers for compressive sensing MRI, mapping each ADMM iteration to a stage with reconstruction, shrinkage, and multiplier update operations [24]. ADMM-Net replaces the hand-crafted shrinkage function with a learnable piecewise linear function parameterized by control points [24]. ADMM-Net is an early and influential example of algorithm unrolling that naturally combines the merits of model-based approaches with deep learning [24].

Unrolled Optimization with Deep Priors (ODP) (Diamond et al., 2018 [25]). Diamond et al. proposed a general framework that factors each unrolled iteration into a data step, which encodes prior knowledge of the forward model, and a CNN

prior step, which learns statistical priors end-to-end [25]. Diamond et al. showed that incorporating the forward model in the data step is critical: for deblurring and compressed sensing MRI, ODP networks with data steps substantially outperformed pure residual networks without data steps [25].

DeMUN (Chen et al., 2025 [26]). The Deep Memory Unrolled Network leverages the history of all gradients from previous iterations and generalizes a wide range of existing unrolled algorithms as special cases [26]. Chen et al. found that training with an intermediate loss function that penalizes reconstruction error at every layer uniformly outperforms training with a last-layer-only loss [26]. They also found that the number of unrolled iterations has a significantly greater impact on performance than the depth of the neural network used within each iteration [26].

2.3.5 Algorithm Unrolling for Analytical Chemistry

Despite the breadth of algorithm unrolling in imaging and compressed sensing, its application to analytical chemistry signal processing remains limited. Gharbi et al. [11] designed and compared three deep unrolled architectures, unrolled primal-dual (U-PD), unrolled ISTA (U-ISTA), and unrolled half-quadratic (U-HQ), for sparse signal recovery in chromatography [11]. Gharbi et al. noted that 1D signal restoration, as it occurs in analytical chemistry, remains scarcely studied in the deep unrolling literature compared to image processing applications [11].

In their architectures, algorithm hyperparameters such as step sizes and regularization weights are untied across layers and learned end-to-end via backpropagation [11]. Gharbi et al. reported that U-HQ, using a smoothed hybrid penalty mixing Fair and Cauchy potentials, better recovered non-null signal baselines and peak intensities without bias effect compared to ℓ_1-based unrolled methods [11].

This finding is significant for LBEADS-NET: it confirms that the ℓ_1-induced bias observed in classical BEADS persists in unrolled sparse-recovery architectures, independently corroborating the baseline leakage phenomenon.

In a related study, Gharbi et al. [27] presented a comprehensive comparative study of the same three architectures applied to mass spectrometry data, finding that deep unrolling provides three key benefits over iterative optimization: interpretability, computation speed, and efficiency through supervised task-oriented tuning [27]. They reported that the best-performing iterative method did not necessarily yield the best unrolled architecture, confirming that the relationship between iterative and unrolled performance is not straightforward [27].

2.3.6 Applying Algorithm Unrolling to BEADS

The BEADS algorithm is a natural candidate for unrolling. Each MM iteration (Algorithm 1, lines 6–11) performs a sequence of deterministic operations, reweighting, matrix assembly, linear system solve, and signal update, that can be mapped to a differentiable neural network layer. The learnable parameters (λ_0, λ_1, λ_2, r) are scalars with physical interpretations, enabling an extremely parameter-efficient architecture: with K layers and 5 parameters per layer (adding a learnable step size), the total parameter count is $5K$, orders of magnitude fewer than generic deep networks.

However, two challenges arise. First, the MM update involves solving a banded linear system (Equation 2.14), which requires either a differentiable linear solver or an alternative formulation. Second, the reweighting steps (Equation 2.12–Equation 2.13) involve division by $|\mathbf{x}|$, which can produce large gradients near zero. Chapter 3 addresses both challenges: the ISTA-style reformulation replaces the

linear system solve with a gradient-descent-plus-proximal-operator step, and the log-space parameterization ($\lambda = \exp(\log \lambda)$) ensures positivity without gradient issues.

2.4 Neural Approaches to Baseline Correction

2.4.1 Deep Learning Methods

Recent work has applied deep learning to baseline correction, demonstrating that neural networks can learn the signal-to-baseline mapping directly from training data, eliminating per-signal parameter tuning.

Convolutional autoencoders (Kensert et al., 2021 [7]). Kensert et al. trained a 1D convolutional autoencoder on 190,000 synthetic chromatograms, achieving effective baseline removal without per-signal parameter tuning [7]. This was among the first demonstrations that deep learning can replace classical baseline correction in chromatographic analysis. The model learns an implicit mapping from the corrupted signal to the corrected signal, with the encoder-decoder architecture serving as a nonlinear denoising filter.

CAE+ (Han et al., 2024 [28]). Han et al. proposed a unified preprocessing pipeline for Raman spectroscopy consisting of a convolutional denoising autoencoder for noise reduction followed by a convolutional autoencoder with a comparison function for baseline correction [28]. The comparison function exploits the prior knowledge that baseline values are typically lower than signal values at each point [28]. Han et al. achieved peak preservation rates between 0.85 and 0.96 with their denoising model [28]. However, as with all supervised approaches, the models are trained on synthetic data, and their performance on real spectra depends on the fidelity of the

simulation.

ResUNet (Chen et al., 2022 [29]). Chen et al. proposed ResUNet, a deep-learning baseline-correction model combining ResNet and UNet architectures that directly maps a raw spectrum to its corrected form [29]. The model contains approximately 2.35 million trainable parameters and was trained on 210,000 synthetically generated spectra [29]. Chen et al. demonstrated that deep-learning-based baseline-correction methods are significantly better than penalized-least-squares methods such as arPLS in terms of RMSE, and that the deep-learning approach eliminates the need for per-spectrum parameter tuning [29].

RSPSSL (Hu et al., 2024 [10]). Hu et al. proposed RSPSSL, a self-supervised learning scheme for Raman spectral preprocessing that performs both denoising and baseline correction without requiring experimentally obtained ideal reference spectra [10]. Hu et al. noted that the performance of supervised deep learning approaches for spectral preprocessing is limited by the impossibility of experimentally obtaining ideal noise-free and baseline-free reference spectra [10]. Their scheme achieved an 88% reduction in RMSE compared to established preprocessing techniques on experimental Raman data [10], and demonstrated generalization across different devices, laboratories, and excitation wavelengths without retraining [10].

2.4.2 Physics-Informed Hybrid Approaches

An alternative to end-to-end deep learning is the *hybrid* approach, in which a machine learning model predicts the parameters of a classical algorithm rather than directly estimating the baseline. This preserves the interpretability and physical guarantees of the classical solver while automating parameter selection.

DIRAS+ (Aradhye et al., 2025 [30]). Aradhye et al. developed DIRAS+, a

hybrid framework that uses a CNN autoencoder plus XGBoost regressor to predict the optimal smoothing parameter λ for the DIRAS solver, achieving tuning-level baseline-correction accuracy without per-spectrum adjustment [30]. The DIRAS+ hybrid approach confines the machine-learning model's role to predicting a single scalar parameter, while the physics-based DIRAS solver handles the actual baseline estimation, ensuring physically consistent results [30]. Aradhye et al. argued that end-to-end deep learning models for baseline correction are limited by substantial data requirements, lack of interpretability, and poor generalization to spectral conditions not seen in training [30].

OP-airPLS (Cui et al., 2025 [31]). Cui et al. developed ML-airPLS, a PCA-Random Forest model that predicts optimal airPLS parameters directly from input spectra, achieving a roughly 2100× speedup over iterative grid search optimization [31]. However, Cui et al. found that direct application of their model to experimental spectra without denoising yielded consistently negative improvement values, demonstrating that models trained on noise-free synthetic data may fail to generalize to noisy real-world spectra [31]. This finding highlights a key limitation of the external parameter prediction approach: the prediction model and the classical solver are trained separately, and errors in parameter prediction propagate directly to baseline estimation quality.

2.4.3 Positioning LBEADS-NET

The methods reviewed in this section fall into three categories, each with distinct trade-offs:

1. **Black-box neural networks** (Kensert et al., Chen et al., Han et al., Hu et al.). These methods learn the signal-to-baseline mapping end-to-end,

achieving strong generalization when sufficient training data is available. However, the learned mapping is encoded in millions of opaque parameters, offering no insight into the decomposition process. One cannot inspect what regularization strategy the network has learned, verify that it respects physical constraints, or diagnose why it fails on a particular signal.

2. **Physics-informed hybrid approaches** (DIRAS+, OP-airPLS). These methods use machine learning to predict the parameters of a classical algorithm, preserving interpretability while automating parameter selection. However, the ML model and the classical solver are decoupled: the ML model is not trained through the solver, so it cannot learn to compensate for the solver's systematic errors. The ML model predicts parameters externally, without feedback from the solver's output quality.

3. **Algorithm unrolling** (LBEADS-NET). The algorithm structure *is* the network architecture. Each layer corresponds to one iteration of BEADS, and the learnable parameters (λ_0, λ_1, λ_2, r, step size) are trained end-to-end through the solver. This means the loss function's gradients flow through the entire optimization pipeline, enabling the network to learn parameters that compensate for the solver's systematic biases, including baseline leakage. The architecture inherits the inductive biases of BEADS (sparsity, smoothness, asymmetry, frequency-domain separation) while gaining the adaptivity of data-driven learning.

LBEADS-NET thus occupies a unique position in the landscape of baseline correction methods: it is neither a black-box neural network that learns an arbitrary mapping, nor an external parameter predictor that is decoupled from the solver.

It is a *structurally interpretable* network whose architecture directly encodes the BEADS optimization, with parameters learned to optimize baseline correction performance on the training distribution. This positioning, and the challenges it entails, particularly the inherited baseline leakage problem, is the subject of Chapter 3.

Chapter 3

Proposed Method: LBEADS-NET

This chapter presents the architecture and design of LBEADS-NET, a neural network that unrolls the classical BEADS algorithm into a differentiable, end-to-end trainable pipeline. Section 3.1 formalizes the learning objective. Section 3.2 derives the core ISTA-style layer that constitutes the fundamental computational unit. Section 3.3 motivates the choice of this proximal gradient formulation over an earlier conjugate gradient variant. Section 3.4 describes the full K-layer pipeline that chains ISTA layers into a complete network. Section 3.5 discusses the non-learnable cutoff frequency and the autograd boundary that prevents its optimization. Section 3.6 presents the multi-stage curriculum training strategy. Section 3.7 details the twelve-term composite loss function and its anti-leakage design. Section 3.8 describes the hybrid inference pipeline that combines the network's output with classical post-processing.

3.1 Problem Formulation

Recall from Section 1.1 the additive signal model (Equation 1.1):

$$\mathbf{y} = \mathbf{x} + \mathbf{f} + \mathbf{w}, \tag{3.1}$$

where $\mathbf{y} \in \mathbb{R}^N$ is the observed chromatographic signal, $\mathbf{x} \in \mathbb{R}^N$ is the peak component, $\mathbf{f} \in \mathbb{R}^N$ is the baseline drift, and $\mathbf{w} \in \mathbb{R}^N$ is measurement noise.

The classical BEADS algorithm estimates $\mathbf{x}$ and $\mathbf{f}$ by solving a regularized optimization problem whose solution requires manual specification of the regularization weights $\lambda_0, \lambda_1, \lambda_2$, the asymmetry ratio r, the filter cutoff frequency f_c, and the filter order d. As discussed in Section 1.2.1, no single parameter configuration generalizes across instruments, analytes, or signal conditions.

The goal of LBEADS-NET is to learn a mapping

$$(\hat{\mathbf{x}}, \hat{\mathbf{f}}) = \mathcal{N}(\mathbf{y}; \Theta), \tag{3.2}$$

where $\mathcal{N}$ denotes the network, Θ collects all trainable parameters, $\hat{\mathbf{x}}$ is the estimated peak signal, and $\hat{\mathbf{f}}$ is the estimated baseline. The mapping is constructed so that $\hat{\mathbf{x}} + \hat{\mathbf{f}} \approx \mathbf{y} - \mathbf{w}$, recovering the noise-free signal decomposition.

The central design principle is to *unroll, not replace*: each iteration of the classical BEADS algorithm is mapped to a differentiable neural network layer, preserving the optimization structure of BEADS while making its parameters learnable via backpropagation. This approach inherits the inductive biases of BEADS, sparsity of $\mathbf{x}$ in the derivative domain, smoothness of $\mathbf{f}$, asymmetric penalization of negative values, while gaining adaptivity through end-to-end training

on labeled data.

Concretely, the network $\mathcal{N}$ consists of $K = 20$ sequentially chained ISTA-style layers. Each layer $k \in \{1, \ldots, K\}$ takes the current peak estimate $\mathbf{x}^{k-1}$ as input and produces an updated estimate $\mathbf{x}^k$. The baseline is computed once at the end by low-pass filtering the residual $\mathbf{y} - \mathbf{x}^K$. Each layer possesses its own learnable parameters $\theta^k = \{\lambda_0^k, \lambda_1^k, \lambda_2^k, r^k, \eta^k\}$, where η^k denotes a layer-specific step size, enabling parameter specialization across the depth of the network. The total parameter count is $5K = 100$ learnable scalars, orders of magnitude fewer than a conventional neural network, reflecting the strong inductive bias inherited from BEADS.

The training objective must not only encourage accurate decomposition of $\mathbf{y}$ into $\hat{\mathbf{x}}$ and $\hat{\mathbf{f}}$, but must also actively counteract *baseline leakage*, the phenomenon in which energy from the peak component is erroneously absorbed into the baseline estimate. As discussed in Section 1.2.3, this failure mode arises inherently from sparsity-based formulations when the peak signal is insufficiently sparse. The design of the loss function to address this challenge is deferred to Section 3.7.

3.2 From BEADS to BEADSLayer , the ISTA Reformulation

This section describes the transformation of a single BEADS iteration into a differentiable neural network layer with learnable parameters. The resulting module constitutes the core computational unit of LBEADS-NET. Rather than solving the BEADS linear system exactly, each layer performs a single proximal gradient step, an ISTA-style update [32, 33], that decomposes the BEADS update into a

Table 3.1: Learnable parameters per ISTA layer. All parameters are stored in log-space and recovered via exponentiation. Default initial values correspond to the training configuration used in the final model ($K = 20$ layers, 100 total learnable scalars).

Parameter	**Log-space name**	**Role**	**Init. value**
λ_0	`log_lam0`	Asymmetric sparsity weight	0.01
λ_1	`log_lam1`	First-derivative penalty weight	0.5
λ_2	`log_lam2`	Second-derivative penalty weight	0.5
r	`log_r`	Asymmetry ratio	6.0
η	`log_step_size`	Proximal gradient step size	0.05

gradient descent step on the smooth objectives followed by a proximal operator for the non-smooth sparsity penalty.

3.2.1 Learnable Parameters

Each layer maintains five learnable scalar parameters, summarized in Table 3.1. To guarantee positivity, each parameter ϕ is stored in log-space as $\log \phi$ and recovered via exponentiation:

$$\phi = \exp(\log \phi). \tag{3.3}$$

This parameterization ensures that gradients flow smoothly through the exponential map, preventing negative regularization weights that would render the optimization ill-posed. No clamping is applied, the exponential function alone guarantees positivity, and unconstrained log-space parameters allow the optimizer full freedom to explore the parameter space.

The five parameters are stored as independent `nn.ParameterList` entries, one list per parameter type, each containing K scalar entries, rather than being en-

capsulated in per-layer `nn.Module` objects. This flat storage avoids redundant precomputation of filter matrices within each layer. The parameters are initialized to values informed by the classical BEADS defaults but adapted for the proximal gradient formulation: the step size $\eta = 0.05$ is small to ensure stable refinement from the high-pass initialization, and $\lambda_0 = 0.01$ is conservative to avoid over-thresholding small peaks in the first layers.

3.2.2 Butterworth Filter Construction

The BEADS algorithm employs a zero-phase Butterworth high-pass filter, realized through banded matrices A and B, to enforce frequency-domain separation between peaks (high-frequency) and baseline (low-frequency). In LBEADS-NET, the filter coefficients are *fixed*, the cutoff frequency f_c and filter order d are not learnable parameters.

The filter coefficients are computed by the `BAfilt` function as follows. For filter order d and normalized cutoff frequency $f_c \in (0, 0.5)$, define the frequency-warping parameter

$$t = \left(\frac{1 - \cos(2\pi f_c)}{1 + \cos(2\pi f_c)} \right)^d . \tag{3.4}$$

The high-pass numerator polynomial $\mathbf{b} \in \mathbb{R}^{2d+1}$ is constructed by convolving $(2d+1)$-tap FIR kernels derived from the first-order difference operator $[1, -1]$:

$$\mathbf{b} = \underbrace{[1, -1] * [-1, 2, -1] * \cdots}_{d-1 \text{ convolutions}} * [-1, 1], \tag{3.5}$$

and the denominator polynomial is

$$\mathbf{a} = \mathbf{b} + t \cdot \underbrace{[1, 2, 1] * [1, 2, 1] * \cdots}_{d \text{ convolutions}}. \tag{3.6}$$

The coefficient vectors $\mathbf{a}$ and $\mathbf{b}$, each of length $2d + 1$, define banded Toeplitz matrices $A, B \in \mathbb{R}^{N \times N}$ with bandwidth d. The matrix A has entries $A_{ij} = a_{|i-j|+d}$ for $|i - j| \leq d$ and zero otherwise, and similarly for B. The high-pass filter operator is then $H = BA^{-1}$, and the complementary low-pass operator is $L = I - H = I - BA^{-1}$.

Unlike the classical BEADS implementation, which exploits banded structure for $O(dN)$ matrix-vector products, the v5 architecture precomputes and stores A and B as *dense* $N \times N$ matrices (`A_dense` and `B_dense`), registered as non-learnable buffers. The low-pass operator $L = I - BA^{-1}$ is also precomputed as a dense matrix (`lowpass_dense`) via the `compute_lowpass_matrix` function, which computes A^{-1} by solving $AZ = I$ and then forms $L = I - BZ$. When iterated low-pass filtering is desired, the matrix power L^p is computed by repeated matrix multiplication via `compute_iterated_lowpass_matrix`. In the final trained model, $p = 1$ (a single-pass low-pass filter, matching classical BEADS).

The use of dense matrices trades memory ($O(N^2)$ per buffer) for simplicity and full differentiability: every filter application reduces to a matrix-vector product that PyTorch's autograd can differentiate without custom backward passes. For the final model with $N = 4096$, each dense buffer requires approximately 128 MB in float64, which is feasible on modern hardware. The decision to fix f_c and d rather than learning them is discussed further in Chapter 6.

3.2.3 Difference Operators

The smoothness penalties in the BEADS objective require first- and second-order finite difference operators $D_1 \in \mathbb{R}^{(N-1)\times N}$ and $D_2 \in \mathbb{R}^{(N-2)\times N}$. Rather than storing these as matrices, the implementation computes their actions via efficient vectorized slicing operations:

$$[D_1\mathbf{x}]_i = x_{i+1} - x_i, \quad i = 1, \ldots, N-1, \tag{3.7}$$

$$[D_2\mathbf{x}]_i = x_{i+2} - 2x_{i+1} + x_i, \quad i = 1, \ldots, N-2. \tag{3.8}$$

The transpose operations $D_1^\top$ and $D_2^\top$, required for computing gradients, are implemented as adjoint accumulation operations:

$$[D_1^\top \mathbf{v}]_i = \begin{cases} -v_1 & i = 1, \\ v_{i-1} - v_i & 2 \le i \le N-1, \\ v_{N-1} & i = N, \end{cases} \tag{3.9}$$

$$[D_2^\top \mathbf{v}]_i = \begin{cases} v_1 & i = 1, \\ -2v_1 + v_2 & i = 2, \\ v_{i-2} - 2v_{i-1} + v_i & 3 \le i \le N-2, \\ v_{N-3} - 2v_{N-2} & i = N-1, \\ v_{N-2} & i = N. \end{cases} \tag{3.10}$$

These four operations, `diff1`, `diff2`, `diff1_T`, and `diff2_T`, are implemented as methods of the network class and operate on batched tensors of shape (batch, N). Each runs in $O(N)$ time and requires no stored matrices.

3.2.4 The ISTA Layer Forward Pass

Each ISTA layer implements one proximal gradient step for the BEADS objective. Given the current peak estimate $\mathbf{x}^{k-1}$ and the observed signal $\mathbf{y}$, the layer computes the updated estimate $\mathbf{x}^k$ through two stages: a gradient descent step on the smooth terms, followed by a proximal step for the non-smooth asymmetric sparsity penalty.

Data fidelity gradient. The data fidelity term measures how well the current peak estimate explains the high-frequency content of the observed signal. The residual $\mathbf{y} - \mathbf{x}^{k-1}$ contains both baseline (low-frequency) and unexplained peak (high-frequency) content. The gradient is computed by applying the high-pass filter to extract only the high-frequency component of the residual:

$$\mathbf{g}_{\text{data}} = H(\mathbf{y} - \mathbf{x}^{k-1}), \tag{3.11}$$

where $H = I - L$ is the high-pass operator and L is the precomputed dense low-pass matrix. This formulation ensures that the low-frequency (baseline) content of the residual does not influence the peak update, only the high-frequency residual drives the peak estimate toward the data. In the implementation, $H(\mathbf{v})$ is computed as $\mathbf{v} - L\mathbf{v}$ via the `apply_highpass_filter` function, which calls `apply_lowpass_filter` internally.

Smoothness penalty gradients. The first- and second-order smoothness penalties encourage sparse derivatives in the peak signal. Their gradients with respect to $\mathbf{x}$ are:

$$\nabla_{\mathbf{x}}\mathcal{R}_1 = \lambda_1^k\, D_1^\top \left(\frac{D_1\mathbf{x}^{k-1}}{|D_1\mathbf{x}^{k-1}| + \epsilon} \right), \qquad \nabla_{\mathbf{x}}\mathcal{R}_2 = \lambda_2^k\, D_2^\top \left(\frac{D_2\mathbf{x}^{k-1}}{|D_2\mathbf{x}^{k-1}| + \epsilon} \right), \tag{3.12}$$

where $\epsilon = 10^{-6}$ is a smoothing constant that prevents division by zero. These are the subgradients of the smoothed ℓ_1 norms $\sum_i |[D_j \mathbf{x}]_i|$, with the ratio $u/(|u| + \epsilon)$ serving as a differentiable approximation to $\text{sign}(u)$. The operations D_1, D_2 and their transposes $D_1^\top, D_2^\top$ are computed via the vectorized slicing operations described in Section 3.2.3.

Gradient descent step. The data fidelity gradient and the smoothness penalty gradients are combined into a single update, scaled by the learnable step size η^k:

$$\mathbf{x}_{\text{update}} = \mathbf{x}^{k-1} + \eta^k \, \mathbf{g}_{\text{data}} - \eta^k \left(\nabla_{\mathbf{x}} \mathcal{R}_1 + \nabla_{\mathbf{x}} \mathcal{R}_2 \right). \tag{3.13}$$

The first term moves the estimate toward the high-frequency content of the observed signal, while the second term penalizes non-sparse derivatives. The step size η^k controls the aggressiveness of the update: small values produce cautious refinements, while large values produce aggressive corrections.

Proximal step (asymmetric soft thresholding). The sparsity-promoting asymmetric penalty is handled via a proximal operator, the asymmetric soft-thresholding function:

$$[\mathbf{x}^k]_i = \mathcal{S}_{\text{asym}}([\mathbf{x}_{\text{update}}]_i;\ \lambda_0^k \eta^k,\ r^k) = \begin{cases} [\mathbf{x}_{\text{update}}]_i - \lambda_0^k \eta^k & \text{if } [\mathbf{x}_{\text{update}}]_i > \lambda_0^k \eta^k, \\ [\mathbf{x}_{\text{update}}]_i + \lambda_0^k \eta^k r^k & \text{if } [\mathbf{x}_{\text{update}}]_i < -\lambda_0^k \eta^k r^k, \\ 0 & \text{otherwise.} \end{cases} \tag{3.14}$$

The asymmetric soft-thresholding operator is the proximal operator of the asymmetric ℓ_1 penalty function. It enforces sparsity by shrinking values with small magnitude to exactly zero, while the asymmetry ratio r^k causes negative values to be

Algorithm 2 ISTA Layer Forward Pass

Require: Current peak estimate $\mathbf{x}^{k-1}$, observed signal $\mathbf{y}$, low-pass matrix L
Require: Learnable parameters: $\lambda_0^k, \lambda_1^k, \lambda_2^k, r^k, \eta^k$ (all recovered via $\exp(\cdot)$ from log-space)

Data fidelity gradient:
$\mathbf{g}_{\text{data}} \leftarrow (\mathbf{y} - \mathbf{x}^{k-1}) - L(\mathbf{y} - \mathbf{x}^{k-1})$ *(high-pass filter of residual)*
Smoothness penalty gradients:
$\mathbf{w}_1 \leftarrow D_1\mathbf{x}^{k-1} \,/\, (|D_1\mathbf{x}^{k-1}| + \epsilon)$
$\mathbf{w}_2 \leftarrow D_2\mathbf{x}^{k-1} \,/\, (|D_2\mathbf{x}^{k-1}| + \epsilon)$
$\nabla\mathcal{R}_1 \leftarrow \lambda_1^k\, D_1^\top \mathbf{w}_1, \quad \nabla\mathcal{R}_2 \leftarrow \lambda_2^k\, D_2^\top \mathbf{w}_2$
Gradient step:
$\mathbf{x}_{\text{update}} \leftarrow \mathbf{x}^{k-1} + \eta^k\, \mathbf{g}_{\text{data}} - \eta^k\, (\nabla\mathcal{R}_1 + \nabla\mathcal{R}_2)$
Proximal step (asymmetric soft thresholding):
$\mathbf{x}^k \leftarrow \mathcal{S}_{\text{asym}}(\mathbf{x}_{\text{update}};\ \lambda_0^k\eta^k,\ r^k)$ *(Equation 3.14)*

Ensure: Updated peak estimate $\mathbf{x}^k$

thresholded r^k times more aggressively than positive values. Since chromatographic peaks are positive, this encourages positive sparse peaks and strongly suppresses negative artifacts. The threshold $\lambda_0^k\eta^k$, the product of the sparsity weight and the step size, is an effective threshold that the network can modulate independently through either parameter.

The complete ISTA layer forward pass is summarized in Algorithm 2.

3.3 Why ISTA Over Conjugate Gradient

The ISTA-style layer described in Section 3.2 is not the only possible unrolling of the BEADS algorithm. An earlier variant, implemented as the `LBEADS_NET` class using `BEADSLayer` modules, faithfully reproduced the BEADS update by solving the full linear system at each layer via conjugate gradient (CG). This section explains why the CG variant was abandoned in favor of the proximal gradient formulation.

3.3.1 The CG Variant

In the classical BEADS algorithm, each iteration requires solving the linear system

$$\left(B^\top B + A^\top M A\right)\mathbf{z} = \mathbf{d}, \tag{3.15}$$

where

$$M = 2\lambda_0\,\Gamma + D^\top \Lambda\, D, \qquad D = \begin{bmatrix} D_1 \\ D_2 \end{bmatrix}, \qquad \Lambda = \begin{bmatrix} \Lambda_1 & 0 \\ 0 & \Lambda_2 \end{bmatrix}, \tag{3.16}$$

and $\Gamma = \mathrm{diag}(\gamma_1, \ldots, \gamma_N)$ contains the asymmetric penalty weights. The right-hand side $\mathbf{d}$ depends on the observed signal $\mathbf{y}$ and the current layer's learnable parameters λ_0^k and r^k. The peak estimate is then recovered as $\mathbf{x}_{\text{raw}}^k = A\mathbf{z}$, followed by a learnable relaxation step $\mathbf{x}^k = \mathbf{x}^{k-1} + \alpha^k(\mathbf{x}_{\text{raw}}^k - \mathbf{x}^{k-1})$.

The CG variant (`BEADSLayer` in v6) solves Equation 3.15 using a fixed-iteration conjugate gradient method. The left-hand-side matrix-vector product $\mathbf{v} \mapsto (B^\top B + A^\top M A)\mathbf{v}$ is computed in operator form using banded coefficient vectors and vectorized difference operations, avoiding the assembly of any dense matrix. Each CG layer maintains five learnable parameters $(\lambda_0, \lambda_1, \lambda_2, r, \alpha)$ with clamped exponential parameterization, and the CG iterations are warm-started from the previous iterate transformed to the z-domain via an A-system solve.

3.3.2 The Gradient Flow Problem

The CG-based architecture creates a critical obstacle for gradient-based training. Each of the K outer layers contains an inner CG loop that runs for K_{inner} fixed iterations. Backpropagation must differentiate through every CG iteration within every layer, producing a computational graph of effective depth $K \times K_{\text{inner}}$. With a

training configuration of $K = 8$ outer layers and $K_{\text{inner}} = 12$ CG steps, this yields an effective depth of 96 sequential differentiable operations.

Each CG iteration involves multiple banded matrix-vector products with the operators A, B, D_1, D_2 and their transposes, as well as inner products and scalar divisions. The Jacobian of the CG iterate with respect to the input is a product of K_{inner} matrices, each dependent on the intermediate CG state. As is well established in the deep learning literature, such depth causes *gradient vanishing*: the magnitude of gradients with respect to early-layer parameters decays exponentially with depth, effectively preventing the early layers from learning meaningful parameter updates.

Furthermore, the CG solver's convergence behavior introduces additional gradient instability. In practice, K_{inner} CG iterations during training provide only an approximate solution to the linear system, and the quality of this approximation varies across training samples depending on the conditioning of $B^\top B + A^\top M A$. The resulting approximation error couples with the gradient vanishing problem, making the training dynamics unpredictable. Even the warm-starting strategy, which provides a good initial guess $\mathbf{z}_0 = A^{-1}\mathbf{x}^{k-1}$, does not resolve this issue, because the CG convergence trajectory still contributes K_{inner} Jacobian factors to the gradient chain.

3.3.3 Gradient Flow in the ISTA Variant

The ISTA variant provides a dramatic improvement in gradient flow. The key distinction lies in the effective computational depth.

In the CG variant, the effective depth is $K \times K_{\text{inner}}$. Gradients must backpropagate through K_{inner} sequential CG iterations within each of K layers, producing a chain of Jacobians whose product contracts in norm.

In the ISTA variant, the effective depth is simply K. Each layer performs exactly one gradient step (Equation 3.13) followed by one element-wise proximal operation (Equation 3.14). The high-pass filtering step $H(\mathbf{y}-\mathbf{x}^{k-1}) = (\mathbf{y}-\mathbf{x}^{k-1}) - L(\mathbf{y}-\mathbf{x}^{k-1})$ involves a single matrix-vector product with the precomputed dense low-pass matrix L, which is a fixed linear operator whose Jacobian is the operator itself, no iterative unrolling is required.

The consequence is that the ISTA variant can effectively train deeper networks. The design principle that emerges is: *simpler per-layer computations with more layers outperform complex per-layer computations with fewer layers*, provided that the per-layer computation captures the essential structure of the original algorithm. Each ISTA layer performs a less accurate approximation of the BEADS update than a CG layer, it takes a single gradient step rather than solving the full linear system, but the composition of $K = 20$ such layers, each with independently learned parameters, produces a more expressive and trainable network overall. In practice, the CG variant struggled to learn meaningful parameters beyond the first few layers even at $K = 8$, whereas the ISTA variant trains stably at $K = 20$ with all layers contributing to the decomposition.

3.4 Full K-Layer Pipeline

The full LBEADS-NET architecture chains K ISTA layers into a sequential pipeline that maps the observed signal $\mathbf{y}$ to the estimated decomposition $(\hat{\mathbf{x}}, \hat{\mathbf{f}})$. This section describes the complete forward pass of the `LBEADS_NET_Fast` class, which is the architecture used in the final trained model.

3.4.1 Architecture Overview

The network is parameterized by the signal length N, the filter order d (default 1), the normalized cutoff frequency f_c (default 0.006), and the number of unrolled layers K. In the final trained model, $N = 4096$ and $K = 20$.

Each of the K layers has five independent learnable parameters: λ_0^k, λ_1^k, λ_2^k, r^k, and η^k (step size). The total learnable parameter count is $5K = 100$ scalars. The initial values are $\lambda_0 = 0.01$, $\lambda_1 = 0.5$, $\lambda_2 = 0.5$, $r = 6.0$, and $\eta = 0.05$, chosen to provide stable refinement from the high-pass initialization described below. All parameters are stored in log-space and recovered via exponentiation with no clamping.

The network stores three dense $N \times N$ matrices as registered buffers: `A_dense` (the banded denominator matrix), `B_dense` (the banded numerator matrix), and `lowpass_dense` (the precomputed low-pass operator $L = I - BA^{-1}$). Additionally, the product $B^\top B$ is precomputed and stored as `BTB_dense`. These buffers are shared across all layers and are not learnable.

3.4.2 Initialization

Rather than initializing the peak estimate to zero, the network computes a high-pass-filtered initial estimate:

$$\mathbf{x}^0 = \mathbf{y} - L(\mathbf{y}), \tag{3.17}$$

where L denotes the low-pass filter operator. In the implementation, the low-pass filter is applied via the precomputed dense matrix `lowpass_dense`: $L(\mathbf{y}) = \mathbf{y} \cdot L^\top$ (exploiting the batch-row convention), and $\mathbf{x}^0 = \mathbf{y} - L(\mathbf{y})$ is the complementary

high-pass output.

This initialization provides a substantially better starting point than $\mathbf{x}^0 = \mathbf{0}$. The high-pass filtering removes the bulk of the baseline, providing a rough peak estimate that the subsequent ISTA layers refine. Without this initialization, small peaks whose amplitude falls below the effective soft-thresholding level $\lambda_0^1 \eta^1$ would be permanently zeroed out by the proximal step in the first layer and could never be recovered by subsequent layers. The initialization ensures that all peaks present in the high-frequency band of $\mathbf{y}$ are available for refinement from the first layer onward.

3.4.3 Layer Chaining

The K layers are applied sequentially. At each layer k, the update rule from Section 3.2.4 is applied:

$$\mathbf{x}^k_{\text{update}} = \mathbf{x}^{k-1} + \eta^k \, H(\mathbf{y} - \mathbf{x}^{k-1}) - \eta^k \left(\nabla_{\mathbf{x}} \mathcal{R}_1^k + \nabla_{\mathbf{x}} \mathcal{R}_2^k \right), \tag{3.18}$$

$$\mathbf{x}^k = \mathcal{S}_{\text{asym}}\big(\mathbf{x}^k_{\text{update}};\ \lambda_0^k \eta^k,\ r^k\big). \tag{3.19}$$

The observed signal $\mathbf{y}$ is shared across all layers, it defines the data fidelity term, but the penalty weights $\lambda_0^k, \lambda_1^k, \lambda_2^k$, the asymmetry ratio r^k, and the step size η^k are all layer-specific. This independence enables *parameter specialization*: during training, early layers tend to learn coarse peak-baseline separation with larger step sizes and stronger regularization, while later layers learn finer refinement with smaller adjustments. This specialization emerges naturally from the training process without explicit architectural constraints.

As an example of learned specialization, after training on the final model, layer 0

has parameters $\lambda_0 = 0.00755$, $\lambda_1 = 0.576$, $\lambda_2 = 0.740$, $r = 4.153$, and $\eta = 0.0705$, moderately larger step size and derivative penalties for initial shaping. Deeper layers adjust these values to fine-tune the decomposition.

When intermediate supervision is enabled during training, each layer's output $\mathbf{x}^k$ is stored and a corresponding baseline estimate $\mathbf{f}^k = L(\mathbf{y} - \mathbf{x}^k)$ is computed. The loss function can then incorporate supervision at every layer, providing direct gradient signal to each layer and further alleviating gradient attenuation across depth.

3.4.4 Final Output Computation

After the final layer, the peak and baseline estimates are computed as follows.

Peak estimate. The final peak estimate is the output of the K-th layer:

$$\hat{\mathbf{x}} = \mathbf{x}^K. \tag{3.20}$$

No additional output gain, scaling, or post-processing is applied to the peak estimate.

Baseline estimate. The baseline is not predicted directly by the network layers. Instead, it is computed by low-pass filtering the residual:

$$\hat{\mathbf{f}} = L(\mathbf{y} - \hat{\mathbf{x}}), \tag{3.21}$$

where L is the same precomputed dense low-pass matrix used in the initialization step. This ensures that $\hat{\mathbf{f}}$ is inherently smooth, a structural guarantee inherited from the BEADS formulation that a purely data-driven network could not provide

without explicit architectural enforcement. The residual $\mathbf{y} - \hat{\mathbf{x}}$ contains both baseline and noise; the low-pass filter extracts only the smooth baseline component, implicitly discarding noise.

The noise estimate is implicitly defined as $\hat{\mathbf{w}} = \mathbf{y} - \hat{\mathbf{x}} - \hat{\mathbf{f}}$, capturing any signal content that is neither sparse (peaks) nor smooth (baseline). Since the low-pass filter removes high-frequency content from the residual, the noise estimate contains primarily the high-frequency portion of $\mathbf{y} - \hat{\mathbf{x}}$ that is not captured by the peak estimate.

3.5 Non-Learnable Cutoff Frequency and the Autograd Boundary

The Butterworth filter matrices A and B described in Section 3.2.2 are parameterized by the cutoff frequency f_c and the filter order d. Although these quantities fundamentally determine the frequency boundary between baseline and peaks, and thus the quality of the decomposition, they are *fixed* hyperparameters in LBEADS-NET, not learnable parameters. This section explains why.

The SciPy–PyTorch boundary. The filter coefficients $\mathbf{a}$ and $\mathbf{b}$ are computed by the `BAfilt` function, which uses SciPy's sparse matrix construction routines to build the banded Toeplitz matrices A and B from the coefficient vectors. These sparse matrices are then converted to dense NumPy arrays and loaded into PyTorch as registered buffers via `register_buffer`. This conversion crosses an *autograd boundary*: SciPy operations are opaque to PyTorch's automatic differentiation engine. The gradient $\partial L/\partial f_c$ cannot be computed because the chain $f_c \to \mathbf{a}, \mathbf{b} \to A, B \to L \to \hat{\mathbf{f}}$ passes through non-differentiable SciPy operations. Consequently,

f_c has no gradient and cannot be updated by backpropagation.

Implications of a fixed f_c. In the final trained model, $f_c = 0.006$ (normalized frequency, corresponding to a period of approximately 167 samples). This value is fixed for *all* input signals regardless of their spectral content. For signals whose baseline varies on a timescale shorter than $1/f_c$, the low-pass filter cannot track the baseline, and the untracked baseline energy appears as a residual in the peak estimate. Conversely, for signals with broad peaks whose spectral content extends below f_c, the low-pass filter absorbs peak energy into the baseline, a direct mechanism for baseline leakage.

Connection to baseline leakage. The fixed cutoff frequency interacts with the leakage problem in two ways. First, dense peak regions produce broad spectral energy that extends into the low-frequency band below f_c, where it is captured by the low-pass filter and attributed to the baseline. Second, spatially uniform regularization (f_c is the same everywhere) cannot adapt to local signal characteristics: a region with closely spaced peaks may require a lower f_c to avoid absorbing peak flanks, while a region with rapid baseline variation may require a higher f_c. A learnable, spatially varying f_c could in principle adapt the frequency boundary to local signal conditions.

Potential solutions. Several approaches could make the filter boundary learnable. First, one could implement a differentiable IIR filter design entirely within PyTorch, replacing the SciPy coefficient computation with differentiable arithmetic on the frequency-warping parameter t and the coefficient polynomials. Second, one could replace the Butterworth filter entirely with learnable convolution kernels that are trained end-to-end. Third, a meta-learning approach could predict f_c from signal features (e.g., spectral energy distribution) using a lightweight auxiliary network,

using the predicted f_c to construct the filter matrices at inference time. These directions are deferred to future work (Chapter 6).

3.6 Multi-Stage Curriculum Training

Training LBEADS-NET with the full composite loss function described in Section 3.7 from the first epoch leads to optimization instability. The 12-term loss landscape is non-convex and multi-modal: competing objectives, such as sparsity penalties that encourage peak shrinkage versus anti-leakage penalties that penalize baseline over-estimation, create conflicting gradients that prevent convergence. To address this, we adopt a *multi-stage curriculum training* strategy [34] that progressively introduces loss terms in order of increasing complexity.

The curriculum decomposes training into three stages, each defined by a distinct subset of active loss terms and a corresponding epoch budget. The progression follows the principle that simpler objectives should be mastered before harder ones: the network first learns basic peak reconstruction, then learns structured separation with anti-leakage constraints, and finally refines the decomposition with the full loss suite. This approach is analogous to the scoring-and-pacing framework of Hacohen and Weinshall [34], where easier examples are presented before harder ones to accelerate convergence and improve generalization. In our setting, the "difficulty" axis is the complexity of the loss landscape rather than the training data.

3.6.1 Stage A: Peak Reconstruction

The first stage trains with only the reconstruction loss ($\alpha_{\text{mse}} = 1.0$), setting all other loss weights to zero. This establishes the basic peak-baseline separation: the

network learns to produce a peak estimate $\hat{\mathbf{x}}$ that matches the ground truth peak signal $\mathbf{x}$ in the mean squared error sense. By deferring all regularization and anti-leakage penalties, Stage A allows the learnable parameters $\{\lambda_0^k, \lambda_1^k, \lambda_2^k, r^k, \eta^k\}_{k=1}^K$ to move freely toward configurations that minimize reconstruction error without interference from competing objectives.

Stage A serves as a warmup that prevents early divergence. Without it, the anti-leakage losses (which penalize baseline behavior at peak locations) would fire on a poorly initialized decomposition, producing large, noisy gradients that destabilize the parameter updates before the network has learned any useful structure.

3.6.2 Stage B: Baseline Leakage Mitigation

The second stage activates the core set of regularization and anti-leakage losses while retaining the reconstruction anchor. The active terms and their weights are: reconstruction ($\alpha_{\text{mse}} = 1.0$), ℓ_1 sparsity ($\alpha_{\ell_1} = 0.01$), total variation ($\alpha_{\text{tv}} = 0.01$), baseline smoothness ($\alpha_{\text{smooth}} = 0.2$), non-negativity ($\alpha_{\text{neg}} = 2.0$), masked baseline reconstruction ($\alpha_{\text{baseline}} = 0.5$), high-frequency leakage ($\alpha_{\text{leakage}} = 0.3$), peak-baseline orthogonality ($\alpha_{\text{ortho}} = 0.1$), baseline third-derivative ($\alpha_{\text{baseline\_tv}} = 0.05$), and asymmetric baseline ($\alpha_{\text{asym}} = 1.0$ with asymmetry ratio 0.9).

This is the critical stage where the anti-leakage machinery takes effect. The masked baseline reconstruction loss provides direct supervision of the baseline in non-peak regions. The asymmetric baseline loss penalizes baseline over-estimation nine times more heavily than under-estimation, directly targeting the leakage failure mode. The orthogonality and high-frequency leakage penalties enforce mutual exclusivity between the peak and baseline components.

The envelope constraint (α_{envelope}) and frequency separation (α_{freq}) losses are

Table 3.2: Multi-stage curriculum training configuration. Each stage defines a subset of active loss terms and a corresponding epoch budget. Stages are executed sequentially; parameters are carried forward between stages.

Stage	**Epochs**	**Active Loss Terms**
A: Peak Reconstruction	1	Reconstruction (MSE) only
B: Leakage Mitigation	1	MSE + ℓ_1 + TV + smoothness + non-negativity + masked baseline + leakage + orthogonality + baseline TV + asymmetric baseline
C: Full Refinement	1	All terms from global loss configuration (envelope and frequency separation disabled for efficiency)

not activated in Stage B. These terms are computationally expensive, the envelope constraint requires a sliding-window soft-minimum computation, and the frequency separation loss requires a full FFT, and their early activation was found to slow convergence without improving final performance.

3.6.3 Stage C: Full Refinement

The third stage trains with the full set of loss weights from the global loss configuration (Table 3.3), with the exception that the envelope and frequency separation terms remain disabled for computational efficiency. This stage fine-tunes the decomposition learned in Stage B, adjusting all parameters simultaneously under the complete multi-objective landscape.

The stage configurations are summarized in Table 3.2.

3.6.4 Intermediate Supervision

An orthogonal training enhancement explored in later development iterations is *intermediate supervision*: computing the loss not only at the final layer output

$\mathbf{x}^K$ but at every intermediate layer output $\mathbf{x}^k$ for $k = 1, \ldots, K$. The baseline at each layer is computed as $\mathbf{f}^k = L(\mathbf{y} - \mathbf{x}^k)$, and the loss function is applied to each $(\mathbf{x}^k, \mathbf{f}^k)$ pair with layer-dependent weights. This provides direct gradient signal to each layer, further alleviating gradient attenuation across depth.

The concept is motivated by the finding of Chen et al. [26] that training unrolled networks with an intermediate loss function that penalizes reconstruction error at every layer outperforms training with a last-layer-only loss, due to a smoother optimization landscape. In the final trained model (v5), intermediate supervision was not used, all loss terms are applied only to the final output, but the infrastructure for per-layer loss computation is present in the architecture (Section 3.4.3) and was exercised in later development iterations.

3.7 Composite Loss Function

The composite loss function is the central mechanism through which LBEADS-NET learns to produce accurate decompositions while actively suppressing baseline leakage. The loss serves two distinct objectives: (1) *reconstruction fidelity*, the estimated peaks $\hat{\mathbf{x}}$ and baseline $\hat{\mathbf{f}}$ should match the ground truth, and (2) *anti-leakage engineering*, the loss must actively penalize configurations in which peak energy is erroneously absorbed into the baseline estimate.

The second objective is necessary because the network's architecture inherits the sparsity bias of the classical BEADS algorithm (Section 3.1). The ℓ_1-based penalties and the asymmetric soft-thresholding operator encourage sparse peaks; when peaks are densely packed, these operators over-shrink the peak estimate, and the surplus energy is captured by the low-pass filter and attributed to the baseline.

Without explicit anti-leakage terms in the loss, the network has no gradient signal to counteract this failure mode.

The total loss is a weighted sum of twelve terms:

$$\mathcal{L}_{\text{total}} = \sum_{i=1}^{12} \alpha_i \, \mathcal{L}_i, \tag{3.22}$$

where each α_i is a non-negative scalar weight. The terms are organized into four groups by purpose: reconstruction fidelity, sparsity priors on peaks, baseline supervision, and leakage-specific penalties. Each term, its mathematical formulation, and the failure mode it targets are described below.

3.7.1 Reconstruction Fidelity

Term 1: Reconstruction loss ($\alpha_{\text{mse}} = 1.0$). The anchor term measures the discrepancy between the predicted peak signal $\hat{\mathbf{x}}$ and the ground truth peak signal $\mathbf{x}$. The implementation uses the Huber loss [35] rather than the mean squared error:

$$\mathcal{L}_{\text{recon}} = \frac{1}{N} \sum_{i=1}^{N} \ell_\delta(\hat{x}_i - x_i), \qquad \ell_\delta(u) = \begin{cases} \frac{1}{2}u^2 & |u| \le \delta, \\ \delta(|u| - \frac{1}{2}\delta) & |u| > \delta, \end{cases} \tag{3.23}$$

with $\delta = 1.0$. The Huber loss is quadratic for small errors and linear for large errors, providing robustness to outlier samples in which the peak estimate deviates substantially from the ground truth. This term is active in all three curriculum stages and carries the highest weight ($\alpha_{\text{mse}} = 1.0$), ensuring that reconstruction accuracy remains the primary objective throughout training.

3.7.2 Sparsity Priors on Peaks

Term 2: ℓ_1 sparsity ($\alpha_{\ell_1} = 0.005$–0.01). The ℓ_1 penalty encourages the peak signal to be sparse, most sample values should be zero, with non-zero values only at peak locations:

$$\mathcal{L}_{\ell_1} = \frac{1}{N} \sum_{i=1}^{N} |\hat{x}_i|. \tag{3.24}$$

This term encodes the prior that chromatographic peaks occupy a small fraction of the total signal length. The weight is kept deliberately low ($\alpha_{\ell_1} \leq 0.01$) because aggressive ℓ_1 penalization over-shrinks dense peaks, directly causing the baseline leakage that the anti-leakage terms must then counteract. The tension between the ℓ_1 penalty and the anti-leakage losses is a defining feature of the loss design: both are necessary (the ℓ_1 penalty prevents false peaks in flat regions, while the anti-leakage losses prevent peak absorption in dense regions), but their weights must be carefully balanced.

Term 3: Total variation ($\alpha_{\mathbf{tv}} = 0.005$–$0.01$). The total variation penalty promotes piecewise-constant peak signals with sharp transitions:

$$\mathcal{L}_{\text{tv}} = \frac{1}{N-1} \sum_{i=1}^{N-1} |\hat{x}_{i+1} - \hat{x}_i|. \tag{3.25}$$

This encodes the prior that chromatographic peaks have steep rising and falling edges with flat regions between them. Like the ℓ_1 penalty, the TV weight is kept low to avoid over-regularization in dense-peak regions. The ℓ_1 and TV penalties together are both the *cause* of leakage (they over-shrink dense peaks) and necessary for correct behavior in sparse regions, a fundamental tension in the loss design.

Term 4: Non-negativity ($\alpha_{\mathbf{neg}} = 0.1$–$2.0$). Chromatographic peaks are physi-

cally non-negative. The non-negativity penalty discourages negative values in the peak estimate:

$$\mathcal{L}_{\text{neg}} = \frac{1}{N} \sum_{i=1}^{N} \left[\max(0, -\hat{x}_i) \right]^2. \tag{3.26}$$

The squared penalty provides a smooth gradient that grows with the magnitude of the violation. This term complements the asymmetric soft-thresholding in the ISTA layers (Section 3.2.4), which already penalizes negative values more aggressively through the asymmetry ratio r^k. The loss-level penalty provides an additional training signal that is independent of the per-layer thresholding behavior.

In later development iterations (v7), the non-negativity constraint was also enforced architecturally by applying $\texttt{softplus}(\hat{\mathbf{x}}, \beta = 20)$ as a final activation, ensuring that the output is strictly positive regardless of the loss function. This architectural enforcement was not used in the v5 model that constitutes the thesis system.

3.7.3 Baseline Supervision

Term 5: Masked baseline reconstruction ($\alpha_{\text{baseline}} = 0.5$–$2.0$). This term, introduced in the fourth development iteration (v4), was the single most impactful addition to the loss function. It provides direct supervision of the baseline estimate in non-peak regions by constructing a binary peak mask from the ground truth

peak signal:

$$\tau = \max\big(\gamma \cdot \max_i |x_i|,\ \tau_{\min}\big), \tag{3.27}$$

$$m_i = \mathbf{1}\big[|x_i| \geq \tau\big], \tag{3.28}$$

$$\mathcal{L}_{\text{baseline}} = \frac{\sum_{i=1}^{N}(1 - m_i) \cdot (\hat{f}_i - f_i)^2}{\sum_{i=1}^{N}(1 - m_i) + \epsilon}, \tag{3.29}$$

where $\gamma = 0.02$ is the relative threshold, $\tau_{\min} = 10^{-4}$ is the absolute floor, and $\epsilon = 10^{-8}$ prevents division by zero. The mask m_i is one at peak locations and zero in baseline-only regions. The loss supervises the baseline only where peaks are absent, that is, where the ground truth baseline $\mathbf{f}$ is directly observable from the data.

The masking is critical: supervising the baseline at peak locations would require the loss to know the correct baseline *under* the peaks, which is precisely the quantity being estimated. By restricting supervision to background regions, the loss provides a reliable training signal without circular reasoning. The denominator normalizes by the number of background samples, preventing the loss magnitude from varying with the peak density of each training example.

Term 6: Baseline smoothness ($\alpha_{\text{smooth}} = 0.01$–$0.2$). The smoothness penalty penalizes the curvature of the baseline estimate via its second derivative:

$$\mathcal{L}_{\text{smooth}} = \frac{1}{N-2} \sum_{i=1}^{N-2} \big[\hat{f}_{i+2} - 2\hat{f}_{i+1} + \hat{f}_i\big]^2. \tag{3.30}$$

This encodes the physical prior that baseline drift is a slowly varying, low-frequency phenomenon. A perfectly smooth baseline has zero second derivative everywhere; the squared penalty allows moderate curvature (as in gentle drift) while strongly

penalizing sharp localized features (as in leakage-induced bumps).

Term 7: Baseline third derivative ($\alpha_{\text{baseline\_tv}} = 0.0$–$0.1$). The third-derivative penalty catches localized bumps in the baseline that the second-derivative penalty misses:

$$\mathcal{L}_{\text{base\_tv}} = \frac{1}{N-3} \sum_{i=1}^{N-3} \left[\hat{f}_{i+3} - 3\hat{f}_{i+2} + 3\hat{f}_{i+1} - \hat{f}_i\right]^2. \tag{3.31}$$

A localized bump in the baseline produces a large second derivative at its peak but also produces a large *third* derivative at its edges, where the curvature changes sign. The third-derivative penalty targets these transitions, providing additional smoothness enforcement that is complementary to the second-derivative penalty.

3.7.4 Leakage-Specific Penalties

The following four terms were developed specifically to detect and penalize baseline leakage. Each targets a distinct observable signature of the leakage failure mode.

Term 8: High-frequency baseline leakage ($\alpha_{\text{leakage}} = 0.3$–$1.0$). When peak energy leaks into the baseline, it manifests as high-frequency content in the baseline at peak locations. This term directly penalizes this signature:

$$\mathcal{L}_{\text{leakage}} = \begin{cases} \dfrac{\sum_{i=1}^{N} m_i \cdot [\hat{f}_i^{\text{hf}}]^2}{\sum_{i=1}^{N} m_i + \epsilon} & \text{if } \hat{\mathbf{f}}^{\text{hf}} \text{ is provided,} \\ \dfrac{\sum_{i=1}^{N-2} m'_i \cdot [D_2\hat{\mathbf{f}}]_i^2}{\sum_{i=1}^{N-2} m'_i + \epsilon} & \text{otherwise,} \end{cases} \tag{3.32}$$

where $\hat{\mathbf{f}}^{\text{hf}}$ is the high-pass filtered baseline (if the network provides it as an auxiliary output), m_i is the peak mask from Equation 3.28, m'_i is the peak mask restricted to the interior samples (indices $2, \ldots, N-1$), and D_2 is the second-difference operator.

The fallback formulation uses the second derivative of the baseline as a proxy for high-frequency content, since the Butterworth high-pass filter used in BEADS is approximately equivalent to a second-derivative operator at low frequencies.

The normalization by $\sum m_i$ ensures that the loss is computed per-peak-sample rather than per-total-sample, preventing the penalty from being diluted in signals with few peaks.

Term 9: Peak-baseline orthogonality ($\alpha_{\text{ortho}} = 0.1$–$0.5$). This term enforces the principle that the baseline should be locally flat wherever peaks are present. If the baseline has a nonzero gradient at a peak location, it is "tracking" the peak shape, a hallmark of leakage. The v5 implementation penalizes the first derivative of the baseline, weighted by the ground truth peak magnitude:

$$\mathcal{L}_{\text{ortho}} = \frac{1}{N-1} \sum_{i=1}^{N-1} w_i \cdot \left[\hat{f}_{i+1} - \hat{f}_i\right]^2, \tag{3.33}$$

where the weights are

$$w_i = \frac{|x_i|}{\max_j |x_j| + \epsilon}, \qquad \text{(detached from the computational graph).} \tag{3.34}$$

The use of the ground truth peak signal $\mathbf{x}$ (detached) rather than the predicted peak signal $\hat{\mathbf{x}}$ as the weighting function is a deliberate design choice. If $\hat{\mathbf{x}}$ were used, the orthogonality penalty would vanish whenever the peak estimate collapses to zero, precisely the leakage scenario the penalty is intended to prevent. Using the detached ground truth ensures that the penalty remains active at peak locations regardless of the network's current prediction quality.

The term draws conceptual motivation from orthogonal nonnegative matrix factorization, where orthogonality constraints between factor matrices enforce non-

overlapping decompositions [36, 37]. In the signal decomposition context, the "orthogonality" is not between factor matrices but between the peak and baseline components at each spatial location: where one component is active, the other should be locally constant.

Term 10: Asymmetric baseline loss ($\alpha_{\text{asym}} = 1.0$, asymmetry ratio $\alpha = 0.9$). This term applies an asymmetric penalty to the baseline reconstruction error, penalizing over-estimation of the baseline far more heavily than under-estimation:

$$\mathcal{L}_{\text{asym}} = \frac{1}{N} \sum_{i=1}^{N} \begin{cases} \alpha \cdot (\hat{f}_i - f_i)^2 & \text{if } \hat{f}_i > f_i, \\ (1-\alpha) \cdot (\hat{f}_i - f_i)^2 & \text{if } \hat{f}_i \leq f_i, \end{cases} \tag{3.35}$$

where $\alpha = 0.9$ yields a 9:1 asymmetry ratio, over-estimation of the baseline is penalized nine times more heavily than under-estimation of the same magnitude.

The asymmetry directly targets baseline leakage. When peak energy leaks into the baseline, the baseline estimate rises above the true baseline ($\hat{f}_i > f_i$) at peak locations. The 9:1 penalty ratio ensures that this over-estimation is strongly discouraged, even at the cost of slight under-estimation elsewhere. The design is motivated by the asymmetric reweighting schemes of the AsLS family: Eilers and Boelens [3] introduced asymmetric weights into penalized least squares smoothing for baseline estimation, and Baek et al. [4] refined this approach with the arPLS method, which uses a generalized logistic function to assign data-dependent asymmetric weights. Our formulation applies the asymmetry in the *loss function* rather than in the iterative reweighting scheme, but the underlying principle, that over-estimation of the baseline is more harmful than under-estimation, is the same.

Term 11: Envelope constraint ($\alpha_{\text{envelope}} = 0.5$). The envelope constraint

encodes the physical prior that the baseline should not exceed the local minimum of the observed signal:

$$\mathcal{L}_{\text{envelope}} = \frac{1}{N}\sum_{i=1}^{N}\big[\max(0,\ \hat{f}_i - \tilde{y}_i^{\min})\big]^2, \tag{3.36}$$

where $\tilde{y}_i^{\min}$ is a differentiable approximation to the sliding-window local minimum of $\mathbf{y}$, computed via the log-sum-exp trick:

$$\tilde{y}_i^{\min} = -\tau \log \sum_{j\in\mathcal{W}_i} \exp\big(-y_j/\tau\big), \tag{3.37}$$

with window size $|\mathcal{W}_i| = 51$ and temperature $\tau = 0.1$. The local minimum is detached from the computational graph to prevent the loss from encouraging degenerate solutions in which the network manipulates the local minimum computation rather than improving the baseline estimate.

The ReLU activation ensures that only violations, cases where the baseline exceeds the local signal minimum, are penalized. In regions where peaks are absent, the observed signal $\mathbf{y}$ closely approximates the true baseline $\mathbf{f}$, so the local minimum of $\mathbf{y}$ provides an approximate upper bound on $\mathbf{f}$. Penalizing violations of this bound prevents the baseline from rising into peak regions.

Term 12: Frequency separation ($\alpha_{\mathbf{freq}} = 0.05$). The frequency separation loss enforces the spectral role assignment between peaks and baseline in the Fourier domain:

$$\mathcal{L}_{\text{freq}} = \frac{1}{M}\sum_{m=1}^{M}\Big[|\hat{F}_m|^2 \cdot \mathbf{1}[\nu_m > f_c] + |\hat{X}_m|^2 \cdot \mathbf{1}[\nu_m \le f_c]\Big], \tag{3.38}$$

where $\hat{X}_m$ and $\hat{F}_m$ are the discrete Fourier transform coefficients of $\hat{\mathbf{x}}$ and $\hat{\mathbf{f}}$ respectively, ν_m denotes the m-th frequency bin, f_c is the Butterworth cutoff frequency,

and M is the number of frequency bins. The first term penalizes high-frequency content in the baseline (above f_c), and the second term penalizes low-frequency content in the peaks (below f_c). Together, they enforce the same spectral separation that the Butterworth filter imposes structurally, but through a soft penalty rather than a hard filter.

This term is computationally expensive due to the FFT computation and is typically disabled during training for efficiency. When active, its low weight ($\alpha_{\text{freq}} = 0.05$) reflects its role as a regularizer rather than a primary objective.

3.7.5 Summary of Loss Terms

Table 3.3 summarizes all twelve loss terms, their weight ranges, the curriculum stage in which they are first activated, and the failure mode each term targets.

Design tension. The ℓ_1 and TV penalties (Terms 2–3) are both the *cause* of baseline leakage, they over-shrink dense peaks, pushing surplus energy into the baseline, and *necessary* for correct behavior in sparse regions, where they suppress false peaks. The anti-leakage terms (Terms 8–12) exist to counteract the side effects of the sparsity priors. This adversarial dynamic is a fundamental characteristic of the loss design: the sparsity terms and the anti-leakage terms pull the optimization in opposite directions, and the weight configuration determines the equilibrium point. As discussed in Chapter 6, no weight configuration fully resolves this tension when the sparsity assumption is severely violated.

Table 3.3: Summary of the twelve-term composite loss function. Weight ranges reflect values used across development iterations. The "Stage" column indicates the earliest curriculum stage in which the term is active (A = peak reconstruction, B = leakage mitigation, C = full refinement). Terms marked ", " under "Failure Mode" serve as general priors rather than targeting a specific failure.

#	Term	Weight Range	Stage	Failure Mode Targeted
1	Reconstruction (Huber)	1.0	A	,
2	ℓ_1 sparsity	0.005–0.01	B	False peaks in flat regions
3	Total variation	0.005–0.01	B	Non-sharp peak transitions
4	Non-negativity	0.1–2.0	B	Negative artifacts
5	Masked baseline recon.	0.5–2.0	B	Baseline error in non-peak regions
6	Baseline smoothness	0.01–0.2	B	High-curvature baseline
7	Baseline 3rd deriv.	0.0–0.1	B	Localized baseline bumps
8	High-freq. leakage	0.3–1.0	B	Peak energy in baseline
9	Orthogonality	0.1–0.5	B	Baseline tracking peaks
10	Asymmetric baseline	1.0	B	Baseline over-estimation
11	Envelope constraint	0.5	C	Baseline exceeding signal
12	Frequency separation	0.05	C	Spectral role violation

3.8 Hybrid Inference Pipeline

The trained LBEADS-NET produces a peak estimate $\hat{\mathbf{x}}$ and a baseline estimate $\hat{\mathbf{f}}$ in a single forward pass through $K = 20$ ISTA layers. While this output is often of sufficient quality for downstream analysis, the network's output can be further refined through a lightweight post-processing step that leverages the classical BEADS filter structure.

3.8.1 Post-Processing with Low-Pass Refinement

The post-processing pipeline, implemented in the inference code, consists of two steps.

Adaptive noise thresholding. The residual $\mathbf{y} - \hat{\mathbf{x}}$ is high-pass filtered using the same low-pass matrix L employed by the network. The median absolute deviation (MAD) of the high-pass residual provides a robust estimate of the noise level $\hat{\sigma}$:

$$\hat{\sigma} = \frac{\text{median}(|(\mathbf{y} - \hat{\mathbf{x}}) - L(\mathbf{y} - \hat{\mathbf{x}})|)}{0.6745}. \tag{3.39}$$

The peak estimate is then denoised by subtracting $k \cdot \hat{\sigma}$ (with $k = 2.5$) and clamping to non-negative values:

$$\hat{\mathbf{x}}_{\text{clean}} = \max\big(\hat{\mathbf{x}} - k\hat{\sigma},\ 0\big). \tag{3.40}$$

This removes residual noise-level false peaks that the network's soft-thresholding did not fully suppress.

Iterated low-pass baseline recomputation. The baseline is recomputed from the cleaned peak estimate by iterating the low-pass filter:

$$\hat{\mathbf{f}}_{\text{refined}} = L^p(\mathbf{y} - \hat{\mathbf{x}}_{\text{clean}}), \tag{3.41}$$

where L^p denotes p applications of the low-pass filter ($p = 3$ in the default configuration). The iterated application produces a smoother baseline than a single pass, suppressing any residual high-frequency content that may have leaked through.

3.8.2 Classical BEADS Refinement

A more ambitious hybrid strategy, explored but not used in the final system, uses the network output as an initialization for the classical iterative BEADS algorithm. The classical BEADS solver (Section 3.3.1) requires an initial guess $\mathbf{x}^0$; by providing the network's output $\hat{\mathbf{x}}$ as this initial guess rather than the default $\mathbf{x}^0 = \mathbf{y}$, the iterative solver starts from a high-quality decomposition and needs fewer iterations to converge. This approach combines the network's learned adaptivity (no manual parameter tuning) with the classical algorithm's convergence guarantees (the iterative solver is guaranteed to converge to a stationary point of the BEADS objective for any initial condition).

The hybrid strategy is particularly relevant for deployment scenarios where the network has been trained on synthetic data but must be applied to real chromatograms whose characteristics differ from the training distribution. The classical BEADS refinement can correct distribution-shift artifacts in the network's output, provided that the classical BEADS parameters are appropriately chosen. The selection of these parameters, potentially informed by the learned parameters of the network's final layers, is an avenue for future development.

Chapter 4

Experimental Setup

This chapter describes the experimental framework used to evaluate LBEADS-NET. Section 4.1 details the synthetic data generation process that produces ground-truth-labeled training and test signals. Section 4.2 specifies the training protocol, including architecture hyperparameters and optimizer configuration. Section 4.3 defines the evaluation metrics, including two novel leakage-specific measures. Section 4.4 presents the progressive experiment design that systematically isolates the contribution of each loss term. Section 4.5 describes the baseline comparison methods.

4.1 Synthetic Data Generation

Evaluating baseline correction methods requires ground-truth knowledge of the peak signal $\mathbf{x}$, the baseline $\mathbf{f}$, and the noise $\mathbf{w}$ individually. Since real chromatograms provide only the observed mixture $\mathbf{y} = \mathbf{x} + \mathbf{f} + \mathbf{w}$, we train and evaluate on synthetic data where all three components are generated independently and combined according to the additive signal model (Equation 1.1).

The data generation procedure is designed to produce signals whose statistical characteristics match those of real liquid chromatography (LC) and gas chromatography (GC) data. The design parameters were calibrated against a reference set of eight real chromatographic signals of length $N = 4000$, from which we measured peak heights (10–2600+), peak widths (FWHM 3–370+ samples), peak counts (29–58 per signal), baseline amplitudes (0–35), and noise levels ($\sigma \approx 5$). The generator, implemented as the `SyntheticDataGenerator` class, produces signals of length $N = 4096$ using a dedicated NumPy random number generator seeded for reproducibility.

4.1.1 Peak Generation

Chromatographic peaks are generated using a *bi-Gaussian* (asymmetric Gaussian) model that captures the peak tailing commonly observed in real chromatography. For a peak centered at position c with left-side standard deviation σ and tailing parameter τ, the peak shape is

$$p(t) = \begin{cases} A \exp\big(-(t-c)^2/(2\sigma^2) \big) & t \le c, \\ A \exp\big(-(t-c)^2/(2(\sigma+\tau)^2) \big) & t > c, \end{cases} \tag{4.1}$$

where A is the peak amplitude and the effective right-side width $\sigma + \tau$ exceeds the left-side width σ, producing the characteristic tailing shape. The tailing parameter is set to $\tau = \tau_{\text{raw}} \cdot \sigma/5$, where τ_{raw} is drawn uniformly from $[1.0, 15.0]$, with tailing applied to 60% of peaks.

The number of peaks per signal is drawn uniformly from $[20, 55]$. Peak positions are generated through a combination of uniformly distributed isolated peaks and

Gaussian-distributed *clusters* (2–4 peaks with spread 15–80 samples), with a cluster probability of 0.4. This clustering mechanism produces the dense, overlapping peak groups that challenge sparsity-based methods and are central to the baseline leakage problem.

Peak widths are log-uniformly distributed with $\sigma \in [1.5, 25.0]$, producing FWHM values (2.355σ) from approximately 3.5 to 59 samples. Peak amplitudes are also log-uniformly distributed over $[15.0, 2500.0]$, with an 8% probability of generating a high-amplitude outlier peak (amplitude in $[1000.0, 2500.0]$). The log-uniform distributions ensure that the generator produces both narrow/weak and broad/strong peaks, matching the large dynamic range observed in real chromatographic data.

The total peak signal is the sum of all individual peaks: $\mathbf{x} = \sum_{j=1}^{J} p_j$, where J is the number of peaks. In regions where multiple peaks overlap, their contributions sum constructively, producing the dense-peak configurations that stress-test the sparsity assumption.

4.1.2 Baseline Generation

Baselines are generated using one of three drift types, selected uniformly at random:

Cubic spline. A smooth baseline is constructed by cubic spline interpolation through randomly placed knot points. The number of internal knots is drawn from $[4, 8]$, with knot positions uniformly distributed along the signal. Knot amplitudes are drawn from $[0, A_{\text{base}}]$, where $A_{\text{base}} \sim \mathcal{U}(2, 35)$. Endpoint knots are constrained to low amplitudes ($\leq 0.3 \cdot A_{\text{base}}$) to mimic the common pattern of baselines that are near zero at the beginning and end of a chromatographic run. Natural boundary conditions are applied.

Exponential decay with hump. An exponential decay $A_{\text{base}} \exp(-\beta t)$ with decay rate $\beta \sim \mathcal{U}(0.5, 3.0)$ is combined with a broad Gaussian hump centered at a random position. This models the solvent front and column conditioning effects common in liquid chromatography.

Mixed (polynomial + sinusoidal). A low-degree polynomial (degree 2–3) is combined with 1–2 sinusoidal components of frequency 0.3–2.0 cycles per signal. This produces the slowly oscillating baseline drift observed in long-duration chromatographic runs.

After generation, the baseline amplitude is rescaled so that its peak value stands in a target ratio to the peak signal amplitude. The target ratio is log-uniformly distributed over $[0.01, 0.5]$, ensuring that the dataset includes both low-amplitude baselines (where correction is straightforward) and high-amplitude baselines (where the baseline dominates the observed signal). Baselines with minimum values below -2.0 are shifted upward to maintain approximate non-negativity.

4.1.3 Noise and Signal Assembly

Additive Gaussian noise $\mathbf{w} \sim \mathcal{N}(0, \sigma_w^2 I)$ is added to each signal, with the noise standard deviation σ_w drawn uniformly from $[2.0, 6.0]$. The three components are combined to form the observed signal:

$$\mathbf{y} = \mathbf{x} + \mathbf{f} + \mathbf{w}. \tag{4.2}$$

4.1.4 Dataset and Normalization

The complete dataset consists of 200 synthetic signals generated with seed 42 for reproducibility. The dataset is split 80/20 into 160 training signals and 40 test

signals using a shuffled permutation (also seeded with 42). All experiments share this identical split.

Each signal is normalized independently by dividing all three components ($\mathbf{y}$, $\mathbf{x}$, $\mathbf{f}$) by $\max(|\mathbf{y}|)$:

$$\tilde{\mathbf{y}} = \frac{\mathbf{y}}{\max_i |y_i|}, \qquad \tilde{\mathbf{x}} = \frac{\mathbf{x}}{\max_i |y_i|}, \qquad \tilde{\mathbf{f}} = \frac{\mathbf{f}}{\max_i |y_i|}. \tag{4.3}$$

This per-sample normalization maps each observed signal to the range $[-1, 1]$, allowing the network to operate on signals of comparable scale regardless of their original amplitude. The normalization preserves the additive relationship $\tilde{\mathbf{y}} = \tilde{\mathbf{x}} + \tilde{\mathbf{f}} + \tilde{\mathbf{w}}$ (where $\tilde{\mathbf{w}} = \mathbf{w}/\max_i |y_i|$), so the ground-truth labels remain valid after normalization.

Table 4.1 summarizes all data generation parameters.

4.2 Training Protocol

All LBEADS-NET models in the progressive experiments share the same architecture and training procedure, differing only in their loss function configurations. This section describes the common setup.

4.2.1 Architecture Configuration

Each experiment trains a fresh instance of the `LBEADS_NET_Fast` architecture described in Section 3.4. The architecture is configured with $K = 20$ unrolled ISTA layers, Butterworth filter order $d = 1$, and normalized cutoff frequency $f_c = 0.006$. The low-pass filter uses a single iteration ($p = 1$), matching the classical BEADS

Table 4.1: Synthetic data generation parameters. All ranges denote uniform or log-uniform distributions as described in the text.

Parameter	**Description**	**Value / Range**
N	Signal length	4096
n_{signals}	Total signals generated	200
Train / test split	Ratio	80 / 20
Seed	Random seed	42
Peaks		
Number of peaks	Per signal	$[20, 55]$
Amplitude A	Log-uniform	$[15, 2500]$
Width σ	Log-uniform	$[1.5, 25.0]$
Tailing τ_{raw}	Uniform (60% of peaks)	$[1.0, 15.0]$
Cluster probability		0.4
Cluster size	Peaks per cluster	$[2, 4]$
Cluster spread	Standard deviation (samples)	$[15, 80]$
Baseline		
Drift types	Equally probable	Spline / Exp. decay / Mixed
Amplitude A_{base}	Uniform	$[2, 35]$
Baseline-to-peak ratio	Log-uniform	$[0.01, 0.5]$
Spline knots	Uniform (internal)	$[4, 8]$
Noise		
σ_w	Noise std. dev.	$[2.0, 6.0]$
Normalization		
Method	Per-sample	$\mathbf{y} / \max \lvert\mathbf{y}\rvert$

formulation.

Each layer has five independently learnable scalar parameters, yielding a total of $5 \times 20 = 100$ learnable parameters. The initial values are chosen to be conservative, smaller than the classical BEADS defaults, to ensure stable refinement from the high-pass initialization:

- $\lambda_0 = 0.01$ (sparsity weight; classical default: 0.4),
- $\lambda_1 = 0.5$ (first-derivative penalty; classical default: 4.0),
- $\lambda_2 = 0.5$ (second-derivative penalty; classical default: 3.2),

- $r = 6.0$ (asymmetry ratio; same as classical default),
- $\eta = 0.05$ (step size; no classical analogue).

All parameters are stored in log-space and recovered via exponentiation, guaranteeing positivity without clamping (Section 3.2.1).

4.2.2 Optimizer and Schedule

Training uses the Adam optimizer [38] with learning rate 10^{-3} and default momentum parameters ($\beta_1 = 0.9$, $\beta_2 = 0.999$). A step-wise learning rate schedule reduces the learning rate by a factor of 0.5 every 50 epochs, though this has minimal effect in the 30-epoch training budget used in the progressive experiments. Gradient norms are clipped to a maximum of 1.0 to prevent training instability from occasional large-gradient batches.

Training runs for 30 epochs with a batch size of 16. Given the training set of 160 signals, each epoch consists of 10 gradient update steps. The reconstruction loss uses the Huber function with $\delta = 0.1$ (tighter than the $\delta = 1.0$ used in the curriculum training of Section 3.6, reflecting the normalized signal scale). All computations use float64 precision for numerical stability in the BEADS matrix operations.

Device selection follows a priority order: CUDA GPU if available, otherwise CPU. The MPS (Apple Silicon) backend is explicitly bypassed because it does not support float64 operations required by the BEADS filter matrices.

Each experiment is initialized with a fresh random seed (42), ensuring that differences between experiments arise solely from the loss function configuration, not from initialization or data-ordering stochasticity.

Table 4.2: Training hyperparameters. All progressive experiments share this configuration; only the loss function weights differ across experiments.

Hyperparameter	**Description**	**Value**
Architecture		
K	Number of unrolled ISTA layers	20
d	Butterworth filter order	1
f_c	Normalized cutoff frequency	0.006
p	Low-pass filter iterations	1
Parameters	Total learnable scalars	100
Precision	Floating-point format	float64
Optimizer		
Algorithm		Adam
Learning rate		10^{-3}
β_1, β_2	Momentum parameters	$0.9, 0.999$
Gradient clipping	Max norm	1.0
Training		
Epochs	Per experiment	30
Batch size		16
Huber δ	Reconstruction loss	0.1
Seed	Initialization and data order	42

Single-stage vs. curriculum training. The progressive experiments described in Section 4.4 use *single-stage* training: the loss configuration is fixed for all 30 epochs. This design choice is deliberate, the goal is to isolate the effect of each loss term under controlled conditions, without the confound of stage transitions. The multi-stage curriculum described in Section 3.6 is the training strategy for the final deployed model, not for the ablation-style progressive experiments.

Table 4.2 summarizes the training hyperparameters.

4.3 Evaluation Metrics

All experiments are evaluated on the 40-signal held-out test set using nine metrics organized into four groups: peak reconstruction quality, baseline accuracy, quantification fidelity, and leakage detection. All metrics are computed per signal and reported as test-set averages (mean ± standard deviation).

4.3.1 Peak Reconstruction

Peak MSE. The mean squared error between the predicted and ground-truth peak signals measures point-wise reconstruction accuracy:

$$\text{Peak MSE} = \frac{1}{N}\sum_{i=1}^{N}(\hat{x}_i - x_i)^2. \tag{4.4}$$

Peak MAE. The mean absolute error provides a robust alternative less sensitive to outlier errors:

$$\text{Peak MAE} = \frac{1}{N}\sum_{i=1}^{N}|\hat{x}_i - x_i|. \tag{4.5}$$

Peak correlation. The Pearson correlation coefficient measures the linear relationship between predicted and true peak shapes, independent of scale:

$$\rho = \frac{\sum_{i=1}^{N}(\hat{x}_i - \bar{\hat{x}})(x_i - \bar{x})}{\sqrt{\sum_{i=1}^{N}(\hat{x}_i - \bar{\hat{x}})^2 \sum_{i=1}^{N}(x_i - \bar{x})^2}}, \tag{4.6}$$

where $\bar{\hat{x}}$ and $\bar{x}$ denote the sample means. A correlation of 1.0 indicates perfect shape recovery; values significantly below 1.0 indicate systematic shape distortion. If either signal has zero variance (a degenerate case), the correlation is set to 0.0.

4.3.2 Baseline Accuracy

Baseline MSE. The mean squared error between the predicted and ground-truth baselines:

$$\text{Baseline MSE} = \frac{1}{N}\sum_{i=1}^{N}(\hat{f}_i - f_i)^2. \tag{4.7}$$

Baseline MAE. The mean absolute error for the baseline:

$$\text{Baseline MAE} = \frac{1}{N}\sum_{i=1}^{N}|\hat{f}_i - f_i|. \tag{4.8}$$

4.3.3 Quantification Fidelity

Area error. Peak area integration is the standard method for determining analyte concentration in chromatography. The relative area error measures how well the predicted peaks preserve total integrated area:

$$\text{Area Error} = \frac{\left|\sum_{i=1}^{N}\hat{x}_i - \sum_{i=1}^{N}x_i\right|}{\sum_{i=1}^{N}x_i + \epsilon} \times 100\%, \tag{4.9}$$

where $\epsilon = 10^{-12}$ prevents division by zero. An area error of 0% indicates perfect preservation of total peak area. This metric is critical for analytical chemistry applications, where area errors directly translate to concentration measurement errors.

4.3.4 Leakage Detection

Two novel metrics are introduced to detect and quantify baseline leakage, the phenomenon in which peak energy is erroneously absorbed into the baseline estimate. Both metrics rely on a binary *peak mask* constructed from the ground-truth peak

signal:

$$m_i = \mathbf{1}\Big[|x_i| \geq \max\big(\gamma \cdot \max_j |x_j|,\ \tau_{\min}\big)\Big], \tag{4.10}$$

with relative threshold $\gamma = 0.02$ and absolute floor $\tau_{\min} = 10^{-4}$. The complementary background mask is $m_i^{\text{bg}} = 1 - m_i$.

Baseline Leakage Index (BLI). The BLI measures the extent to which the predicted baseline *over-estimates* the true baseline in peak regions, which is the signature of peak energy being absorbed into the baseline:

$$\text{BLI} = \frac{\sum_{i=1}^{N} \max(0,\ \hat{f}_i - f_i) \cdot m_i}{\sum_{i=1}^{N} x_i \cdot m_i + \epsilon}. \tag{4.11}$$

The numerator sums only the positive baseline residual (over-estimation) at peak locations, and the denominator normalizes by the total peak energy in those locations. A BLI of zero indicates no baseline over-estimation at peak locations. Positive BLI values indicate leakage: the predicted baseline rises above the true baseline where peaks are present, absorbing peak energy. The $\max(0, \cdot)$ function ensures that baseline *under*-estimation does not reduce the BLI, since under-estimation is a different failure mode (conservative baseline estimate) that does not constitute leakage.

Peak-to-Baseline Error Ratio (PBER). The PBER compares the root-mean-squared baseline error in peak regions to the baseline error in background regions:

$$\text{PBER} = \frac{\text{RMSE}_{\text{peak}}}{\text{RMSE}_{\text{bg}} + \epsilon}, \tag{4.12}$$

where

$$\text{RMSE}_{\text{peak}} = \sqrt{\frac{\sum_i m_i(\hat{f}_i - f_i)^2}{\sum_i m_i}}, \qquad \text{RMSE}_{\text{bg}} = \sqrt{\frac{\sum_i m_i^{\text{bg}}(\hat{f}_i - f_i)^2}{\sum_i m_i^{\text{bg}}}}. \tag{4.13}$$

A PBER of approximately 1.0 indicates that baseline errors are uniformly distributed, the baseline is equally accurate in peak and non-peak regions. A PBER significantly greater than 1.0 indicates that the baseline is *worse* where peaks are present, which is the hallmark of leakage. The PBER complements the BLI: while the BLI measures the magnitude of directional (upward) leakage, the PBER detects any systematic degradation of baseline accuracy at peak locations, regardless of sign.

4.3.5 Peak Quality

Negative fraction. The fraction of predicted peak samples with negative values:

$$\text{Neg. Fraction} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[\hat{x}_i < 0]. \tag{4.14}$$

Since chromatographic peaks are physically non-negative, the negative fraction should be near zero for a well-behaved model. Non-zero values indicate either over-correction artifacts (the model subtracts too much baseline in some regions) or insufficient asymmetric thresholding. This metric is directly related to the non-negativity loss term (Section 3.7.2).

4.4 Progressive Experiment Design

The progressive experiment series is the primary evaluation strategy for this thesis. The central premise is that classical BEADS requires careful per-signal parameter optimization, and our goal is to demonstrate how progressive loss engineering builds toward a model that generalizes without tuning. Each experiment in the series adds one key idea to the loss function and measures its impact on all nine evaluation metrics, isolating the contribution of each loss term under otherwise identical conditions.

The six experiments are structured as a development narrative: starting from a classical (non-learned) baseline and advancing through increasingly sophisticated loss configurations. The first experiment uses classical BEADS with no learning; the remaining five train LBEADS-NET with progressively richer loss functions. All five trained models share the architecture, optimizer, training data, test data, and hyperparameters described in Section 4.2, only the loss weights differ.

4.4.1 Experiment 1.0: Classical BEADS Baseline

The first experiment applies the classical (non-learned) BEADS algorithm directly to the normalized test signals, without any training. The algorithm is run with fixed parameters: $f_c = 0.006$, $d = 1$, $r = 6$, $\lambda_0 = 0.4$, $\lambda_1 = 4.0$, $\lambda_2 = 3.2$, and 30 iterations.

These parameter values are the published defaults from Ning et al. [2], not optimized for the synthetic test data. The test signals have been normalized to unit scale (Section 4.1.4), but the BEADS parameters were designed for unnormalized chromatograms with amplitudes in the hundreds. This deliberate mismatch demon-

strates the core limitation of classical BEADS: *a single parameter configuration cannot generalize across signal conditions*. Tuning λ_0, λ_1, λ_2, and r for each test signal would likely improve classical BEADS performance, but such per-signal tuning is precisely the burden that LBEADS-NET aims to eliminate.

This experiment serves as the reference against which all learned models are compared. It establishes what the classical algorithm achieves when applied "out of the box" without per-signal adjustment.

4.4.2 Experiment 1.1: MSE Only (Naive Unrolling)

The first learned model is trained with only the reconstruction loss: $\alpha_{\text{mse}} = 1.0$, all other loss weights set to zero. This is the simplest possible training objective, the network learns to minimize the mean squared error (Huber) between its peak estimate $\hat{\mathbf{x}}$ and the ground-truth peaks $\mathbf{x}$.

This experiment tests a fundamental question: *does algorithm unrolling alone, without careful loss design, suffice for accurate baseline correction?* By providing no structural guidance beyond reconstruction accuracy, the experiment reveals the baseline behavior of the unrolled architecture when the loss function offers no signal about baseline quality, peak sparsity, or non-negativity.

4.4.3 Experiment 1.2: MSE + Sparsity Priors

The second learned model adds three sparsity-related loss terms to the reconstruction objective:

- ℓ_1 sparsity penalty on peaks ($\alpha_{\ell_1} = 0.01$),
- Total variation penalty on peaks ($\alpha_{\text{tv}} = 0.01$),

- Non-negativity penalty ($\alpha_{\text{neg}} = 0.1$).

These terms encode the prior knowledge that chromatographic peaks are sparse, piecewise-smooth, and non-negative. The experiment tests whether incorporating these physical priors into the loss function improves performance beyond pure reconstruction. A key observation is that sparsity priors can *worsen* leakage in dense-peak regions by over-shrinking overlapping peaks, pushing the surplus energy into the baseline. The low weights ($\alpha_{\ell_1} = \alpha_{\text{tv}} = 0.01$) are chosen to provide mild regularization without aggressive peak shrinkage.

4.4.4 Experiment 1.3: + Baseline Supervision

The third learned model adds direct baseline supervision to the Experiment 1.2 configuration:

- Masked baseline reconstruction ($\alpha_{\text{baseline}} = 1.0$),
- Baseline smoothness ($\alpha_{\text{smooth}} = 0.01$).

The masked baseline reconstruction loss (Section 3.7.3) provides direct gradient signal about the baseline quality in non-peak regions. This is the critical hypothesis of the progressive series: *direct baseline supervision prevents leakage by giving the optimizer information about the baseline that pure peak reconstruction cannot provide*. Without baseline supervision, the optimizer has no gradient signal to discourage the baseline from absorbing peak energy, the reconstruction loss penalizes the peak estimate but not the baseline estimate. The masked baseline loss closes this gap.

4.4.5 Experiment 1.4: + Orthogonality (v5 Configuration)

The fourth learned model adds structural separation constraints to the Experiment 1.3 configuration:

- Peak-baseline orthogonality ($\alpha_{\text{ortho}} = 0.5$),
- Baseline third-derivative penalty ($\alpha_{\text{baseline\_tv}} = 0.1$).

The orthogonality loss (Section 3.7.4) penalizes the element-wise product $|\hat{x}_i| \cdot |\hat{f}_i|$, enforcing that the peak and baseline components are mutually exclusive at each sample point. The baseline third-derivative penalty (Section 3.7.3) catches localized bumps in the baseline that the second-derivative smoothness penalty misses. This configuration corresponds to the v5 development iteration, the model architecture that constitutes the thesis system.

4.4.6 Experiment 1.5: Full Anti-Leakage Battery (v7 Configuration)

The fifth learned model activates the full suite of anti-leakage loss terms, starting from a modified base configuration and adding four additional penalties:

- High-frequency baseline leakage penalty ($\alpha_{\text{leakage}} = 1.0$),
- Asymmetric baseline loss ($\alpha_{\text{asym}} = 1.0$, asymmetry ratio $\alpha = 0.9$),
- Envelope constraint ($\alpha_{\text{envelope}} = 0.5$),
- Frequency separation ($\alpha_{\text{freq}} = 0.05$).

This configuration also adjusts several base weights: the ℓ_1 and TV penalties are reduced to 0.005, the non-negativity weight is increased to 2.0, the baseline

Table 4.3: Loss configurations for the progressive experiment series. Each row shows the active loss terms and their weights. Dashes indicate inactive terms ($\alpha = 0$). Experiment 1.0 uses classical BEADS with no learning.

Loss Term	**1.0** Classical	**1.1** MSE	**1.2** +Sparse	**1.3** +Baseline	**1.4** +Ortho	**1.5** Full v7
Reconstruction	,	1.0	1.0	1.0	1.0	1.0
ℓ_1 sparsity	,	,	0.01	0.01	0.01	0.005
Total variation	,	,	0.01	0.01	0.01	0.005
Non-negativity	,	,	0.1	0.1	0.1	2.0
Baseline recon.	,	,	,	1.0	1.0	2.0
Baseline smooth	,	,	,	0.01	0.01	0.01
Orthogonality	,	,	,	,	0.5	0.5
Baseline 3rd deriv.	,	,	,	,	0.1	,
Leakage penalty	,	,	,	,	,	1.0
Asymmetric baseline	,	,	,	,	,	1.0
Envelope	,	,	,	,	,	0.5
Freq. separation	,	,	,	,	,	0.05

reconstruction weight is increased to 2.0, and the baseline third-derivative penalty is disabled ($\alpha_{\text{baseline\_tv}} = 0.0$). These adjustments reflect the v7 development iteration's finding that strong anti-leakage terms require reduced sparsity penalties to avoid conflicting gradient directions, and stronger non-negativity enforcement to compensate for the reduced sparsity shrinkage.

This experiment tests whether the additional anti-leakage terms, asymmetric baseline penalization, envelope constraints, and frequency-domain separation, improve performance beyond the baseline supervision and orthogonality of Experiment 1.4.

Table 4.3 summarizes the loss configurations for all six experiments.

4.5 Baseline Comparison Methods

The primary comparison in this thesis is between LBEADS-NET variants (Experiments 1.1–1.5) and the classical BEADS algorithm (Experiment 1.0). This comparison is structured to highlight the central contribution: *generalization without per-signal parameter tuning*.

4.5.1 Classical BEADS

The classical BEADS algorithm [2] is the natural baseline for LBEADS-NET, since the network's architecture is derived from its iterative optimization structure. The classical algorithm requires specification of six parameters (f_c, d, r, λ_0, λ_1, λ_2) and runs for a fixed number of iterations (30 in our experiments).

In Experiment 1.0, BEADS is applied with published default parameters. The comparison against LBEADS-NET is not primarily about numerical superiority: a carefully tuned classical BEADS configuration for each individual test signal would likely outperform LBEADS-NET on that specific signal. The comparison instead demonstrates the *generalization advantage*: LBEADS-NET applies the same learned parameters to all test signals without modification, whereas classical BEADS requires per-signal tuning to achieve optimal results. The performance of classical BEADS with fixed, non-optimized parameters illustrates the degradation that occurs when tuning is absent, precisely the setting in which LBEADS-NET is designed to operate.

4.5.2 Additional Classical Methods

Several additional classical baseline correction methods are available for comparison through the `pybaselines` library: asymmetrically reweighted Penalized Least Squares (arPLS) [4], adaptive iteratively reweighted Penalized Least Squares (airPLS) [5], and the Statistics-sensitive Non-linear Iterative Peak-clipping algorithm (SNIP) [6]. These methods share the same fundamental limitation as BEADS: each requires one or more hyperparameters that must be tuned per signal or per instrument. A comprehensive comparison with these methods is deferred to future work; the progressive experiments focus on the LBEADS-NET development trajectory, where classical BEADS serves as the most meaningful reference point because it is the algorithm from which the network architecture is derived.

4.5.3 Framing the Comparison

The experimental design reflects the thesis argument: the value of LBEADS-NET lies not in beating a perfectly tuned classical method on a per-signal basis, but in providing a *tuning-free* method that achieves competitive performance across a distribution of signals. The progressive experiment series demonstrates how systematic loss engineering, from naive reconstruction through structured priors to targeted anti-leakage penalties, builds toward this goal. Results for all six experiments are presented in Chapter 5.

Chapter 5

Results and Analysis

This chapter presents the experimental results from the progressive experiment series described in Section 4.4. Section 5.1 reports quantitative results across all six experiments, analyzing the contribution of each progressive loss term addition. Section 5.2 presents ablation studies that isolate the contribution of individual loss components and characterize the effect of network depth. Section 5.5 examines the learned per-layer parameters to characterize the emergent specialization that arises from end-to-end training. Section 5.6 discusses preliminary results on real chromatographic data. Section 5.7 synthesizes the key findings.

All results are computed on the 40-signal held-out test set described in Section 4.1.4. Metrics are reported as mean ± standard deviation across the test set unless otherwise noted. Metric definitions are given in Section 4.3; experiment configurations are summarized in Table 4.3.

5.1 Progressive Development Results

The progressive experiment series evaluates six configurations: classical BEADS with fixed parameters (Experiment 1.0) and five LBEADS-NET variants with increasingly complex loss functions (Experiments 1.1–1.5). Each experiment is described in Section 4.4. This section presents the quantitative comparison, analyzes baseline leakage behavior, and discusses the over-regularization effect observed in the most complex configuration.

5.1.1 Quantitative Comparison

Table 5.1 presents the full results for all six experiments across seven evaluation metrics. The table reveals several clear trends, which we discuss in order of their significance.

All LBEADS-NET variants outperform Classical BEADS on peak reconstruction. The most striking result is the consistent improvement of all five LBEADS-NET variants over the classical BEADS baseline. Peak MSE improves by approximately 2×: the best LBEADS-NET configuration (Experiment 1.3) achieves a Peak MSE of 9.30×10^{-3}, compared to 21.27×10^{-3} for Classical BEADS. Baseline MSE shows a similar pattern, improving from 12.72×10^{-3} to approximately $7.4–7.7 \times 10^{-3}$ across all learned variants.

This result validates the central premise of the thesis: a learned model that applies fixed (trained) parameters across all test signals outperforms a classical algorithm applied with fixed (hand-specified) parameters. The classical BEADS parameters ($\lambda_0 = 0.4$, $\lambda_1 = 4.0$, $\lambda_2 = 3.2$) were calibrated for unnormalized chromatograms with amplitudes in the hundreds, but the test signals are normalized

Table 5.1: Progressive experiment results on the 40-signal test set. All values are mean ± standard deviation. Bold entries indicate the best value in each column (excluding Classical BEADS, which uses a fundamentally different evaluation setting). Peak MSE and Baseline MSE are $\times 10^{-3}$ for readability. Neg. Fraction is reported as a proportion of total signal length.

Experiment	**Peak MSE**	**Peak Corr.**	**Baseline MSE**	**BLI**	**PBER**	**Area Err. %**	**Neg. Frac.**
1.0 Classical BEADS	21.27 ±9.2	0.746 ±0.210	12.72	0.837	8.35	99.99	0.000
1.1 MSE Only	9.79 ±4.7	0.748 ±0.05	7.45	0.593 ±0.075	5.48	49.47	0.295
1.2 +Sparse	9.38 ±4.7	0.785 ±0.05	7.54	0.597 ±0.075	5.33	49.51	0.216
1.3 +Baseline	**9.30** ±4.7	**0.793** ±0.05	7.48	0.596 ±0.075	5.52	51.55	0.211
1.4 +Ortho (v5)	9.36 ±4.7	0.785 ±0.05	**7.40**	**0.592** ±0.075	5.64	50.95	**0.211**
1.5 Full v7	10.01 ±4.7	0.744 ±0.05	7.69	0.611 ±0.075	5.57	57.63	0.305

to unit scale (Section 4.1.4). The resulting parameter mismatch produces severe over-regularization: the 99.99% area error indicates that the classical BEADS output preserves almost none of the original peak area. In contrast, the worst LBEADS-NET variant (Experiment 1.5) achieves an area error of 57.63%, still substantial, but a qualitatively different failure mode. Where classical BEADS erases nearly all peak content, LBEADS-NET preserves roughly half of the peak area across all configurations.

It is important to note that this comparison demonstrates the *tuning problem*, not a fundamental inferiority of the BEADS algorithm. Per-signal optimization of the classical BEADS parameters would likely achieve competitive or superior performance on each individual signal. The advantage of LBEADS-NET is precisely that no per-signal tuning is required: the same learned parameters generalize across

all 40 test signals with diverse peak densities, baseline shapes, and noise levels.

Sparsity priors improve peak shape quality. Comparing Experiment 1.1 (MSE only) to Experiment 1.2 (MSE + sparsity priors), the addition of ℓ_1, total variation, and non-negativity penalties improves peak correlation from 0.748 to 0.785, a meaningful improvement in peak shape fidelity. The negative fraction decreases from 0.295 to 0.216, indicating that the non-negativity penalty ($\alpha_{\text{neg}} = 0.1$) reduces over-correction artifacts that produce physically impossible negative peak values. Peak MSE also improves modestly from 9.79×10^{-3} to 9.38×10^{-3}.

However, the sparsity priors do not improve baseline leakage: BLI increases slightly from 0.593 to 0.597, and PBER decreases only marginally from 5.48 to 5.33. The area error remains essentially unchanged (49.47% vs. 49.51%). This is consistent with the analysis in Section 1.2.3: sparsity priors can improve peak shape quality, smoother, sparser, non-negative peaks, without addressing the fundamental leakage mechanism by which peak energy is absorbed into the baseline estimate.

Baseline supervision is the single most impactful addition. Experiment 1.3, which adds masked baseline reconstruction and baseline smoothness losses to the Experiment 1.2 configuration, achieves the best peak correlation (0.793) and the best peak MSE (9.30×10^{-3}) among all configurations. The improvement in correlation from 0.785 (Experiment 1.2) to 0.793 (Experiment 1.3) is modest in absolute terms but consistent with the hypothesis of Section 4.4.4: direct gradient signal about baseline quality in non-peak regions provides information that pure peak reconstruction cannot.

The negative fraction continues to decrease, reaching 0.211, a 28.5% reduction from the MSE-only baseline of 0.295. Baseline MSE improves slightly from 7.54×10^{-3} to 7.48×10^{-3}. The BLI remains at 0.596, essentially unchanged from

Experiment 1.2, suggesting that baseline supervision improves overall accuracy without specifically targeting leakage in peak regions.

Orthogonality achieves the best baseline quality. Experiment 1.4, which adds peak-baseline orthogonality and the baseline third-derivative penalty, achieves the best baseline MSE (7.40×10^{-3}) and the lowest BLI (0.592) among all configurations. The orthogonality loss directly penalizes the element-wise product of peak and baseline magnitudes, pushing the decomposition toward mutual exclusivity at each sample point. The resulting BLI improvement, from 0.596 (Experiment 1.3) to 0.592 (Experiment 1.4), is small, however, reinforcing the observation that leakage is resistant to loss-level interventions.

Peak correlation decreases slightly from 0.793 to 0.785 with the addition of orthogonality, suggesting a mild trade-off: enforcing mutual exclusivity between peak and baseline components produces cleaner baselines but slightly less accurate peak shapes. This trade-off is a characteristic signature of multi-objective optimization with competing loss terms.

5.1.2 Baseline Leakage Analysis

Baseline leakage, the absorption of peak energy into the baseline estimate, is the central failure mode that motivates much of the loss engineering in LBEADS-NET. The BLI and PBER metrics, defined in Section 4.3.4, are specifically designed to detect and quantify this phenomenon. This section presents a dedicated analysis of these metrics across the progressive experiment series.

Classical BEADS exhibits severe leakage with fixed parameters. Classical BEADS (Experiment 1.0) produces the highest BLI (0.837) and PBER (8.35) of any configuration. A BLI of 0.837 indicates that the predicted baseline over-estimates

the true baseline by an amount equal to 83.7% of the total peak energy in peak regions, nearly all peak energy is absorbed into the baseline. A PBER of 8.35 indicates that the root-mean-squared baseline error in peak regions is 8.35× larger than in background regions, a severe and systematic degradation.

This extreme leakage is the direct consequence of the parameter mismatch described above. The classical BEADS sparsity weights ($\lambda_0 = 0.4$) are far too aggressive for normalized signals with unit-scale amplitudes: the ℓ_1 penalty over-shrinks the peak component, and the surplus energy is redistributed into the baseline estimate. The 99.99% area error confirms that the classical algorithm essentially treats the entire signal as baseline under these parameters. This result demonstrates the tuning problem in its most acute form: the same parameters that perform well on unnormalized data produce catastrophic failure on normalized data.

LBEADS-NET reduces leakage but cannot eliminate it. All five LBEADS-NET variants achieve BLI values in the range 0.592–0.611, representing a 27–29% reduction from the classical BEADS BLI of 0.837. PBER values range from 5.33 to 5.64, representing a 32–36% reduction from the classical PBER of 8.35. These are meaningful improvements, but the residual leakage remains substantial: a BLI of approximately 0.59 indicates that the predicted baseline still absorbs peak energy equivalent to 59% of the total peak content in peak regions.

Table 5.2 isolates the leakage metrics for easier comparison across experiments.

Leakage is insensitive to loss configuration. The most significant finding in Table 5.2 is the near-invariance of BLI across Experiments 1.1–1.4. Despite progressively adding sparsity priors, baseline supervision, orthogonality penalties, and baseline third-derivative regularization, the BLI varies by only 0.005 across

Table 5.2: Baseline leakage metrics across the progressive experiment series. BLI measures the directional (upward) leakage of peak energy into the baseline; PBER measures the ratio of baseline error in peak regions to baseline error in background regions. Values closer to zero (BLI) and 1.0 (PBER) indicate less leakage.

Experiment	**BLI**	**PBER**	**Key Addition**
1.0 Classical BEADS	0.837	8.35	Fixed parameters
1.1 MSE Only	0.593	5.48	Learned parameters
1.2 +Sparse	0.597	5.33	ℓ_1 + TV + non-neg
1.3 +Baseline	0.596	5.52	Baseline supervision
1.4 +Ortho (v5)	0.592	5.64	Orthogonality
1.5 Full v7	0.611	5.57	Full anti-leakage battery

these four configurations (from 0.592 to 0.597). This insensitivity suggests that once the basic ISTA architecture with learned parameters is in place, additional loss terms have diminishing returns on leakage reduction.

The PBER tells a similar story: values range from 5.33 to 5.64 across Experiments 1.1–1.5, with no monotonic improvement as loss complexity increases. The small variations appear to reflect noise in the optimization process rather than systematic effects of the loss terms. In particular, the full v7 configuration (Experiment 1.5) achieves BLI = 0.611, *worse* than the MSE-only baseline of 0.593, indicating that the additional anti-leakage terms not only fail to reduce leakage but slightly increase it, likely due to competing gradient directions in the 12-term loss function.

This finding confirms the thesis's fundamental insight, stated in Section 1.3: all loss-based mitigations are regularizers on top of a sparsity-biased formulation, and no amount of loss engineering fully overcomes the leakage when the sparsity assumption is violated. The BLI plateau at approximately 0.59 across all LBEADS-NET configurations represents a fundamental limit of the ISTA-based architecture: the ℓ_1-induced sparsity bias is embedded in the network's forward computation,

and loss terms can adjust the learned parameters but cannot alter the architectural inductive bias.

This conclusion generalizes beyond LBEADS-NET to any unrolled network that inherits ℓ_1-based sparsity priors, as independently corroborated by concurrent work on unrolled sparse-recovery networks [11] and physics-informed deep learning approaches [12]. Addressing baseline leakage at a fundamental level would require architectural modifications, such as replacing the ℓ_1 penalty with a group sparsity formulation that operates at the peak level rather than the sample level, rather than additional loss terms.

5.1.3 The Over-Regularization Effect

Experiment 1.5, the full v7 configuration with all 12 loss terms active, exhibits a striking counter-result: it performs *worse* than several simpler configurations on multiple metrics. This section analyzes this degradation and its implications.

Performance degradation in Experiment 1.5. Compared to the best intermediate configurations, Experiment 1.5 shows degradation on four of seven metrics:

- Peak correlation drops from 0.793 (Experiment 1.3, the best) to 0.744, essentially returning to the MSE-only level (0.748 in Experiment 1.1).
- Peak MSE increases from 9.30×10^{-3} (Experiment 1.3) to 10.01×10^{-3}, the worst among all LBEADS-NET variants.
- Negative fraction rises from 0.211 (Experiments 1.3 and 1.4) to 0.305, despite the non-negativity weight being increased 20× from 0.1 to 2.0.
- Area error increases from 49.47% (Experiment 1.1) to 57.63%.

- BLI increases from 0.592 (Experiment 1.4) to 0.611, the worst among all LBEADS-NET variants.

Diagnosis: competing objectives with insufficient training budget. The full v7 configuration activates 12 loss terms simultaneously, including four anti-leakage terms (high-frequency leakage penalty, asymmetric baseline loss, envelope constraint, frequency separation) that were not present in Experiments 1.1–1.4. These terms introduce competing gradient directions: the asymmetric baseline loss ($\alpha = 0.9$) penalizes baseline over-estimation 9× more than under-estimation, pushing the baseline *downward*; the envelope constraint prevents the baseline from exceeding local signal minima, also pushing downward; yet the orthogonality loss pushes the peak and baseline toward mutual exclusivity, which can push the baseline *upward* in regions where the peak is near zero.

With only 30 training epochs (10 gradient steps per epoch, totaling 300 gradient updates), the optimizer cannot converge on all 12 objectives simultaneously. The result is a compromise solution that satisfies none of the objectives well. The increase in negative fraction from 0.211 to 0.305 is particularly telling: despite a 20× increase in the non-negativity weight, the competing gradient directions from other loss terms overwhelm the non-negativity enforcement, producing more negative peak values than the simpler configurations.

Implications for loss engineering. This result carries an important practical lesson: more loss terms do not guarantee better performance. The relationship between loss complexity and model quality is non-monotonic, beyond a certain point, additional objectives degrade performance by fragmenting the optimization landscape. The optimal configuration within the 30-epoch training budget is Experiment 1.3 (MSE + sparsity priors + baseline supervision), which uses 6 active

loss terms. The 12-term configuration requires a longer training horizon to realize its potential benefits.

This observation motivated the three-stage curriculum training strategy described in Section 3.6, which introduces loss terms gradually across training stages rather than activating all terms simultaneously. By allowing the model to first learn accurate reconstruction (Stage A), then structured separation (Stage B), and finally full anti-leakage refinement (Stage C), the curriculum provides the optimizer with a tractable learning trajectory through the multi-objective landscape. The progressive experiment results confirm that this staged approach is not merely a convenience but a necessity: the full loss battery requires careful temporal orchestration to produce benefits rather than degradation.

5.2 Ablation Studies

To complement the progressive development experiments (which *add* loss terms sequentially), this section presents ablation studies that *remove* individual components from the full configuration and vary the network depth. All ablation experiments use the same data, training protocol, and seed as the progressive series.

5.2.1 Loss Term Ablations

Four loss-term ablation experiments isolate the contribution of individual loss groups by removing them from the full configuration while keeping all other terms active.

[Ablation analysis to be completed after experiments are run.]

Table 5.3: Loss term ablation results. Each row removes one group of loss terms from the full configuration (Ablation F). Metrics are reported as mean over the test set. Arrows indicate the direction of improvement; bold indicates best in column.

Ablation	**Removed**	**Peak MSE↓**	**Corr↑**	**BLI↓**	**PBER↓**	**Neg Frac↓**
A: No baseline sup.	$\alpha_{\text{base}}, \alpha_{\text{leak}}$	,	,	,	,	,
B: No non-negativity	α_{neg}	,	,	,	,	,
C: Sparsity only	baseline terms	,	,	,	,	,
D: Minimal (MSE+base)	sparsity + ortho	,	,	,	,	,
F: Full config (ref.)	,	,	,	,	,	,

Table 5.4: Network depth ablation. All experiments use the full loss configuration with 60 epochs (10 warmup + 50 full). Total learnable parameters scale linearly as $5K$.

K	**Params**	**Peak MSE↓**	**Corr↑**	**BLI↓**	**PBER↓**	**Epoch Time**
4	20	,	,	,	,	,
10	50	,	,	,	,	,
20	100	,	,	,	,	,
30	150	,	,	,	,	,

5.2.2 Network Depth Ablation

The depth ablation varies the number of unrolled layers $K \in \{4, 10, 20, 30\}$ while keeping the loss configuration and all other hyperparameters fixed. This directly tests the algorithm unrolling hypothesis: more layers should improve performance up to a point, after which diminishing returns or training instability may set in.

[Depth ablation analysis to be completed after experiments are run.]

5.2.3 Ablation Summary

[Ablation summary to be completed after all experiments are run. Expected to confirm: (1) baseline supervision is critical, (2) non-negativity improves peak quality but not leakage, (3) K=20 provides good balance of capacity and trainability, (4)

simpler loss configs can match complex ones with limited training budget.]

5.3 Qualitative Examples

[Qualitative example figures to be generated from demo.py. Will include 2–3 panels showing easy, hard, and comparison cases.]

5.4 Training Dynamics

[Training dynamics figures to be generated from the Orchestration run epoch data. Will show loss curves across 3 curriculum stages and per-component loss decomposition.]

5.5 Learned Parameter Analysis

One of the key advantages of algorithm unrolling over black-box neural networks is that the learned parameters retain their physical interpretability. Each LBEADS-NET layer possesses five learnable scalar parameters, λ_0 (sparsity weight), λ_1 (first-derivative penalty), λ_2 (second-derivative penalty), r (asymmetry ratio), and η (step size), whose values can be inspected and analyzed. This section examines the per-layer parameter values that emerge from training, using data from a 6-layer model trained for 100 epochs with the three-stage curriculum (Section 3.6).

5.5.1 Layer Specialization

Table 5.5 shows the final learned parameter values for selected layers of the 6-layer curriculum-trained model. Three patterns of layer specialization emerge.

Table 5.5: Final learned parameters for selected layers of the 6-layer curriculum-trained model (100 epochs, 3-stage curriculum, signal length $N = 1024$). Parameters that reached clamp boundaries are marked with $*$.

Parameter	**Layer 0** (Early)	**Layer 3** (Middle)	**Layer 5** (Final)
λ_0 (sparsity)	0.481	1.198	2.000^*
r (asymmetry)	3.36	2.00^*	9.67
η (step size)	0.248	0.191	0.243

Sparsity weight increases across layers. The sparsity parameter λ_0 increases monotonically from 0.481 at Layer 0 to 1.198 at Layer 3 to 2.000 at Layer 5 (where it hits the upper parameter clamp). This progression indicates that early layers perform gentle, conservative peak-baseline separation with low sparsity penalty, while later layers apply increasingly aggressive sparsity enforcement to refine the decomposition. The pattern is consistent with the behavior of iterative thresholding algorithms, where early iterations establish a coarse estimate and later iterations sharpen the solution through stronger thresholding.

The fact that λ_0 reaches the upper clamp at Layer 5 suggests that the final layer would prefer an even stronger sparsity penalty if permitted. This is a direct consequence of the ISTA architecture: the final layer receives a partially converged estimate and must apply strong shrinkage to produce the final output. In the classical BEADS algorithm, this corresponds to the final iterations where the estimate is close to convergence and strong regularization refines the details.

Asymmetry ratio shows a non-monotonic pattern. The asymmetry ratio r exhibits a distinctive non-monotonic pattern: 3.36 at Layer 0, dropping to 2.00 (the lower clamp) at Layer 3, and rising sharply to 9.67 at Layer 5. The early and middle layers apply moderate asymmetry, distinguishing positive peaks from

negative artifacts with moderate penalties. The middle layer's $r = 2.00$ (at the lower clamp) suggests minimal asymmetric penalization during the intermediate refinement stage. The final layer's high asymmetry ratio ($r = 9.67$) produces strong penalization of negative values relative to positive values, consistent with a cleanup role in which the final layer aggressively suppresses residual negative artifacts.

Step size is approximately uniform. The step size η varies little across layers: 0.248, 0.191, and 0.243 for Layers 0, 3, and 5, respectively. This relative stability suggests that the optimal step size for the ISTA update is primarily determined by the spectral properties of the BEADS operator matrices (which are shared across layers via the fixed filter parameters), rather than by the layer's position in the network. The modest reduction at Layer 3 may reflect a more conservative update strategy during intermediate refinement.

5.5.2 Emergent Specialization

The parameter patterns described above, increasing sparsity, non-monotonic asymmetry, stable step size, were not designed into the architecture. All layers are initialized with identical parameter values, and the specialization emerges entirely from end-to-end training via backpropagation. This emergent behavior is a key advantage of the algorithm unrolling framework: the architecture inherits the optimization structure of BEADS, while the training process discovers a parameter schedule that is tailored to the data distribution.

The resulting layer specialization can be interpreted as a learned iteration schedule. Classical BEADS applies the same parameters at every iteration; LBEADS-NET effectively learns a non-uniform iteration schedule in which early iterations are conservative (low λ_0, moderate r) and later iterations are aggressive (high λ_0,

high r). This interpretation provides a principled justification for the per-layer parameterization: rather than constraining all layers to share the same parameters (as in the classical algorithm), the unrolled network exploits the freedom to specialize, producing a more effective decomposition with fewer total layers.

The clamping events at Layers 3 and 5, r hitting the lower clamp and λ_0 hitting the upper clamp, respectively, indicate that the parameter clamp ranges may be overly restrictive for some layers. Widening the clamp ranges is a straightforward modification that could improve performance by allowing the optimizer to explore a larger parameter space; however, wider ranges also increase the risk of numerical instability in the BEADS matrix operations, necessitating careful validation.

5.6 Real Chromatogram Results

All quantitative results presented in Section 5.1 and Section 5.5 are computed on synthetic data where ground-truth peak and baseline components are known. This section briefly discusses the application of LBEADS-NET to real chromatographic signals, for which no ground truth is available.

A demonstration pipeline (`demo_chromatogram.py`) applies the trained LBEADS-NET model to real liquid chromatography signals of length $N = 4000$. Since the ground-truth decomposition is unknown for real data, evaluation is necessarily qualitative. Visual inspection of the results shows that the model produces smooth, physically plausible baselines and non-negative peak estimates that are broadly consistent with expert expectations. The baseline tracks the low-frequency drift without obvious leakage artifacts in sparse-peak regions; in dense-peak regions, the baseline behavior is more difficult to assess visually, consistent with the residual

leakage quantified in the synthetic experiments.

However, the absence of ground truth for real data precludes quantitative validation. Several unsupervised quality indicators could be used for approximate assessment in future work:

- *Negative fraction*: the fraction of predicted peak samples with negative values, which should be near zero for physically valid outputs. This metric can be computed without ground truth.

- *Baseline smoothness*: the energy of the second or third derivative of the predicted baseline, which should be low for a well-behaved baseline estimate.

- *Residual noise structure*: the autocorrelation of the residual $\mathbf{y} - \hat{\mathbf{x}} - \hat{\mathbf{f}}$, which should be approximately white if the peak and baseline components have been correctly separated.

- *Peak area consistency*: comparison of integrated peak areas across repeated injections of the same sample, which should show low variance if the baseline correction is stable.

The qualitative nature of the real-data evaluation is a limitation of this work. Establishing a ground-truth-labeled real chromatographic dataset, through, for example, carefully controlled spike-and-recovery experiments or consensus labeling by multiple expert chromatographers, is an important direction for future validation.

5.7 Summary of Findings

The results of the progressive experiment series, ablation studies, and learned parameter analysis yield the following principal findings:

1. **LBEADS-NET generalizes without per-signal tuning.** All five LBEADS-NET variants outperform classical BEADS with fixed parameters on peak MSE ($\sim$2$\times$ improvement), baseline MSE ($\sim$1.7$\times$ improvement), and area error (reducing from 99.99% to 49–58%). The classical BEADS failure with default parameters on normalized data demonstrates the tuning problem in its most acute form; LBEADS-NET eliminates this failure mode by learning parameters that generalize across the signal distribution.

2. **Baseline supervision is the single most impactful loss term.** Among the progressive additions, masked baseline reconstruction (Experiment 1.3) produces the largest improvements in peak correlation ($0.748 \rightarrow 0.793$ from MSE-only to +Baseline) and peak MSE ($9.79 \times 10^{-3} \rightarrow 9.30 \times 10^{-3}$). Direct gradient signal about baseline quality provides information that pure peak reconstruction cannot supply.

3. **Sparsity priors improve peak shape quality but do not address leakage.** The addition of ℓ_1, total variation, and non-negativity penalties (Experiment 1.2) improves peak correlation from 0.748 to 0.785 and reduces the negative fraction from 0.295 to 0.216, but leaves the BLI essentially unchanged ($0.593 \rightarrow 0.597$). Sparsity priors produce cleaner, more physically plausible peak shapes without reducing the absorption of peak energy into the baseline.

4. **Baseline leakage persists at BLI $\approx$ 0.59 across all configurations.** The BLI varies by only 0.005 across Experiments 1.1–1.4, indicating that once the learned ISTA architecture is in place, additional loss terms have negligible effect on leakage. This near-invariance represents a fundamental

limit of the ℓ_1-based sparsity formulation: the architectural inductive bias determines the leakage floor, and loss engineering can adjust parameters but cannot alter the inductive bias itself.

5. **Over-regularization is a concrete risk.** The full v7 configuration (Experiment 1.5, 12 loss terms) performs worse than simpler configurations: peak correlation drops to 0.744 (from 0.793 in Experiment 1.3), negative fraction rises to 0.305 (from 0.211), and BLI increases to 0.611 (from 0.592 in Experiment 1.4). With only 30 training epochs, the optimizer cannot converge on 12 competing objectives, producing a compromise solution that satisfies none well. This finding validates the need for the three-stage curriculum training strategy (Section 3.6).

6. **Learned parameters exhibit emergent layer specialization.** The sparsity weight λ_0 increases from 0.481 (Layer 0) to 2.000 (Layer 5), the asymmetry ratio r follows a non-monotonic pattern ($3.36 \rightarrow 2.00 \rightarrow 9.67$), and the step size remains approximately uniform ($\sim$0.19–0.25). This specialization emerges entirely from end-to-end training and can be interpreted as a learned non-uniform iteration schedule, early layers perform coarse separation, later layers perform aggressive refinement, that the classical algorithm's fixed-parameter iteration cannot achieve.

Collectively, these findings support the thesis argument that algorithm unrolling provides a principled framework for bridging interpretability and generalization in chromatographic baseline correction, while also revealing the fundamental limits of sparsity-based formulations when applied to dense-peak signals. The persistent baseline leakage at BLI ≈ 0.59 across all configurations is not a failure of the loss

engineering approach but an honest characterization of what loss-level interventions can and cannot achieve within an ℓ_1-regularized architecture.

Chapter 6

Discussion and Limitations

The results presented in Chapter 5 establish that LBEADS-NET achieves meaningful improvements over classical BEADS with fixed parameters, while simultaneously revealing persistent failure modes that loss engineering alone cannot resolve. This chapter provides an analytical treatment of the principal limitations, organized from the most consequential to the most specific. Section 6.1 presents the centerpiece analysis: the diagnosis, mitigation journey, and fundamental limits of baseline leakage. Section 6.2 addresses the training/inference signal length mismatch. Section 6.3 analyzes the conjugate gradient variant's gradient collapse and extracts design principles for algorithm unrolling. Section 6.4 discusses the non-learnable cutoff frequency and its implications. Section 6.5 examines the softplus approximation and its train/inference gap. Section 6.6 revisits the over-regularization problem from a multi-objective optimization perspective.

Each section follows a common analytical structure: observation of the failure mode, identification of its mechanism, summary of mitigation attempts, explanation of why the limitation persists, and extraction of a generalizable lesson for the

algorithm unrolling community.

6.1 Baseline Leakage , Diagnosis and Mitigation

Baseline leakage, the absorption of peak energy into the baseline estimate, is the central failure mode of LBEADS-NET and, more broadly, of any sparsity-based signal decomposition applied to dense-peak chromatographic signals. The progressive experiment series in Section 5.1.2 demonstrates that the Baseline Leakage Index (BLI) plateaus at approximately 0.59 across all LBEADS-NET configurations, regardless of loss complexity. This section provides a root cause analysis, traces the mitigation journey across development iterations, and articulates the fundamental limits that produce this plateau.

6.1.1 Root Cause Analysis

The baseline leakage phenomenon in LBEADS-NET is inherited from the classical BEADS formulation. Three interacting failure modes, identified in Section 2.2.1.5, drive the leakage:

Sparsity assumption violation. The ℓ_1 penalty $\lambda_0 \sum_n \theta(x_n)$ in the BEADS cost function (Equation 2.1) assumes that the peak component $\mathbf{x}$ is sparse: most entries are zero, and non-zero values are concentrated at isolated peak locations. In dense-peak regions, common in metabolomics, complex mixture analysis, and gradient elution chromatography, this assumption is violated. The penalty over-shrinks the aggregate peak energy, and the surplus is redistributed into the baseline estimate $\hat{\mathbf{f}}$. In the unrolled LBEADS-NET, this mechanism manifests through the asymmetric soft-thresholding operator (Equation 3.14), which applies element-wise

ℓ_1-type shrinkage at every layer. The shrinkage is cumulative: $K = 20$ layers of soft thresholding compound the over-shrinkage effect in dense regions.

Low-pass filter bandwidth mismatch. The Butterworth filter cutoff frequency $f_c = 0.006$ defines a fixed spectral boundary between peak and baseline content. Broad peaks, and closely spaced peaks whose aggregate spectral energy extends into the low-frequency band below f_c, have their low-frequency content captured by the low-pass filter L and attributed to the baseline. This mechanism operates independently of the sparsity penalty: even with $\lambda_0 = 0$, the low-pass filter would absorb some peak energy in dense regions. Conversely, if f_c is reduced to preserve broad peaks, rapidly varying baseline features escape the filter and contaminate the peak estimate.

Spatially uniform regularization. The regularization weights λ_0^k, λ_1^k, λ_2^k are scalar values applied uniformly across all signal positions within each layer. A sparse region with well-separated peaks requires strong sparsity enforcement to suppress false peaks between true peaks; a dense region requires weaker enforcement to preserve the aggregate peak signal. Uniform weights represent a compromise that is suboptimal in both regimes. Although the per-layer parameterization of LBEADS-NET allows different regularization strengths at different depths, it does not provide spatial adaptivity within a single layer.

These three failure modes are not independent. In dense-peak regions, the sparsity penalty over-shrinks the peak estimate (Failure Mode 1), and the resulting surplus energy has a spectral profile that overlaps with the low-frequency baseline band (Failure Mode 2). The uniform regularization (Failure Mode 3) prevents the network from selectively weakening the sparsity penalty in dense regions while maintaining it in sparse regions. The compounding of these mechanisms produces

the systematic upward bias in the baseline estimate that the BLI metric captures.

The community-wide nature of this challenge is confirmed by concurrent work. Gharbi et al. [11] reported that ℓ_1-based unrolled architectures (U-PD, U-ISTA) exhibit bias in recovered signal baselines and peak intensities, and that a hybrid penalty mixing Fair and Cauchy potentials partially mitigated this effect. The DIRAS+ framework [12] similarly identified that physics-based baseline correction methods with sparsity assumptions produce systematic errors in dense-peak regions. These independent findings corroborate the thesis that baseline leakage is an inherent property of sparsity-based signal decomposition, not an artifact of a particular implementation.

6.1.2 The Mitigation Journey

The development of LBEADS-NET spanned six major iterations (v3–v8), with baseline leakage mitigation as a central design objective from v3 onward. Each iteration introduced specific loss terms or architectural modifications targeting one or more of the three root causes. This section traces the engineering progression, documenting what each intervention targeted, how it helped, and why it was insufficient alone. The quantitative effects are summarized in the progressive experiment series (Table 5.2).

v3: Sparsity losses (ℓ_1, total variation). The initial loss configuration included ℓ_1 and total variation penalties on the peak estimate, intended to produce sparse, piecewise-smooth peaks. These terms improved peak shape quality, peak correlation increased from 0.748 (MSE only, Experiment 1.1) to 0.785 (Experiment 1.2), and reduced the negative fraction from 0.295 to 0.216. However, the BLI remained essentially unchanged (0.593 to 0.597). The irony is clear in retrospect: the ℓ_1

penalty is itself the primary cause of leakage in dense regions. Adding stronger sparsity enforcement improves peak shapes in sparse regions while exacerbating the over-shrinkage that drives leakage in dense regions.

v4: Baseline supervision (masked MSE). The introduction of masked baseline reconstruction (Equation 3.29) was the single most impactful addition across the entire development trajectory. By providing direct gradient signal about baseline quality in non-peak regions, this term addressed an information gap: the reconstruction loss $\mathcal{L}_{\text{recon}}$ supervises only the peak estimate, providing no direct incentive for baseline accuracy. Masked baseline supervision produced the best peak correlation (0.793 in Experiment 1.3) and the best peak MSE (9.30×10^{-3}). The masking strategy, supervising the baseline only where peaks are absent, was a critical design choice, avoiding the circular reasoning that would arise from supervising the baseline under peaks.

Despite these improvements, the BLI remained at 0.596, nearly identical to the MSE-only configuration. The explanation is that masked baseline supervision improves baseline accuracy in non-peak regions but provides no gradient signal in the dense-peak regions where leakage actually occurs. The loss function has no mechanism to detect that the baseline is over-estimated under peaks, because the mask excludes precisely those regions from supervision.

v6: Gradient-based orthogonality. The peak-baseline orthogonality loss (Equation 3.33) penalized the first derivative of the baseline, weighted by the ground-truth peak magnitude, enforcing the principle that the baseline should be locally flat wherever peaks are present. This term achieved the lowest BLI in the progressive series (0.592 in Experiment 1.4) and the best baseline MSE (7.40×10^{-3}). The improvement was modest, a BLI reduction of 0.004 from the MSE-only baseline,

indicating that the indirect penalty on baseline shape could suppress some leakage signatures but could not eliminate the underlying energy redistribution.

v7: Full anti-leakage battery. The v7 configuration activated the complete set of anti-leakage terms: asymmetric baseline loss (9:1 penalty ratio for baseline over-estimation vs. under-estimation), element-wise orthogonality, envelope constraint (preventing the baseline from exceeding local signal minima), and frequency separation (enforcing spectral role assignment in the Fourier domain). Additionally, a softplus activation was applied to enforce strict non-negativity of the peak estimate.

The v7 results (Experiment 1.5) revealed a counterintuitive outcome: the full anti-leakage battery produced *worse* performance than simpler configurations. The BLI increased to 0.611, peak correlation dropped to 0.744, and the negative fraction rose to 0.305 despite a 20× increase in the non-negativity weight. The over-regularization analysis in Section 5.1.3 explains this degradation: twelve competing loss terms with only 30 training epochs produce conflicting gradient directions that prevent convergence on any individual objective.

v7: Softplus constraint. The application of softplus($\hat{\mathbf{x}}, \beta$) as a final activation enforced strict positivity of the peak estimate, eliminating a degenerate failure mode in which the network produced negative peak values to satisfy the orthogonality constraint. While this architectural modification did not directly reduce the BLI, it closed off a pathological solution pathway that could emerge when the orthogonality and non-negativity losses competed.

Summary: diminishing returns. The mitigation journey reveals a pattern of diminishing returns. The largest improvement came from the simplest intervention (masked baseline supervision), and each subsequent addition produced progressively

smaller gains. Beyond Experiment 1.3, the BLI varied by only 0.019 across all configurations (0.592 to 0.611), with the most complex configuration performing worst. This pattern is not coincidental; it reflects a fundamental limit that the next section articulates.

6.1.3 Fundamental Limits of Sparsity-Based Decomposition

The persistent BLI plateau at approximately 0.59 across all LBEADS-NET configurations represents a fundamental limit of the ℓ_1-based sparsity formulation, not a failure of the loss engineering effort. This section articulates why loss-level interventions cannot overcome this limit and identifies the generalizable insight for the algorithm unrolling community.

All mitigations are regularizers on a sparsity-biased architecture. The anti-leakage loss terms (asymmetric baseline, orthogonality, envelope constraint, frequency separation) are external regularizers applied to the *output* of a network whose *forward computation* is governed by ℓ_1-based soft thresholding. The soft-thresholding operator (Equation 3.14) is embedded in the architecture: it is applied at every layer, and its shrinkage behavior is determined by the learned parameters λ_0^k and η^k. The loss function can adjust these parameters through gradient descent, but it cannot alter the functional form of the operator itself. The shrinkage is element-wise: it operates on individual signal samples without awareness of their spatial context or peak membership.

When the peak signal is dense, element-wise shrinkage systematically underestimates the peak amplitude at every position, because the soft-thresholding function subtracts a positive threshold $\lambda_0^k \eta^k$ from each sample. In sparse regions, this subtraction produces the desired sparsification (small values are driven to zero). In

dense regions, it produces a uniform amplitude reduction that the low-pass filter captures and attributes to the baseline. No loss term can prevent this mechanism without either (i) driving the effective threshold to zero (which would disable the sparsity prior entirely) or (ii) making the threshold spatially adaptive (which the current architecture does not support).

The BLI plateau as evidence. The near-invariance of the BLI across Experiments 1.1–1.4 (0.592–0.597) provides empirical evidence for this fundamental limit. Four progressively more complex loss configurations, spanning from a single MSE term to eleven active terms, produced a BLI range of 0.005. The optimizer adjusted the learned parameters differently under each loss configuration, but converged to approximately the same leakage level. This convergence suggests that the BLI plateau is determined by the architectural inductive bias (element-wise ℓ_1 shrinkage) rather than by the loss landscape.

Contrast with arPLS: qualitatively different failure modes. An instructive comparison with the arPLS family of methods clarifies the nature of the sparsity-induced limit. As discussed in Section 2.2.1.5, arPLS makes no sparsity assumption on the peak signal. Its failure mode in dense-peak regions is *baseline elevation*: the estimated baseline follows the lower envelope of the dense peaks, producing an overestimate that is smooth and monotonic in the peak region. This failure mode is qualitatively different from the *baseline leakage* observed in BEADS and LBEADS-NET, where the baseline estimate acquires high-frequency content that tracks the shapes of individual peaks.

The distinction is significant for downstream analysis. Baseline elevation (arPLS-type failure) produces a systematic but smooth bias in peak areas that is approximately constant across neighboring peaks. Baseline leakage (BEADS-type failure)

produces a peak-shape-dependent bias that varies from peak to peak, making post-hoc correction more difficult. The arPLS failure mode is arguably less damaging for quantitative analysis, suggesting that sparsity-free formulations may offer advantages in dense-peak regimes despite their own limitations.

Generalization to the algorithm unrolling community. This analysis generalizes beyond LBEADS-NET to any unrolled network that inherits ℓ_1-based sparsity priors. The conclusion is: *the inductive bias of the proximal operator defines the performance floor that loss engineering cannot breach.* For unrolled ISTA networks applied to signals that violate the sparsity assumption, the element-wise soft-thresholding operator imposes a structural constraint on the decomposition that no loss function, however complex, can fully overcome. Addressing this limit requires architectural modifications that change the proximal operator itself, for example, replacing element-wise ℓ_1 shrinkage with group sparsity operators that act at the peak level rather than the sample level, or adopting learned proximal maps that can adapt their shrinkage profile to local signal density.

This finding contributes to the growing understanding that the choice of proximal operator in unrolled networks is not merely a convenience but a fundamental design decision that determines the network's representational limits. Gharbi et al. [11] arrived at a related conclusion through their comparison of ℓ_1-based and hybrid-penalty unrolled architectures, finding that the penalty function's convexity properties directly influence the bias-variance trade-off in the recovered signal.

6.2 Training/Inference Length Mismatch

LBEADS-NET is trained on synthetic signals of fixed length $N = 1024$ but must process real chromatographic signals that commonly range from $N = 2048$ to $N = 8192$ or longer. This section analyzes which architectural components are sensitive to signal length, which are invariant, and how the mismatch affects inference quality.

Length-sensitive components: dense filter matrices. The precomputed dense matrices A, B, $L = I - BA^{-1}$, and $B^\top B$ are all $N \times N$ (Section 3.2.2). Changing N requires recomputing all filter matrices from the Butterworth coefficients $\mathbf{a}$ and $\mathbf{b}$. While the filter coefficients themselves depend only on f_c and d (not on N), the Toeplitz matrices A and B are N-dependent in their dimensions and boundary conditions. The boundary treatment, how the banded structure is handled at the edges of the matrix, changes with N, producing slightly different filtering behavior near the signal endpoints. For interior samples far from the boundaries, the filter behavior is length-invariant; for samples within d positions of either endpoint, the behavior differs between training and inference lengths.

Length-sensitive components: spectral resolution. The normalized cutoff frequency $f_c = 0.006$ specifies the peak-baseline boundary relative to the Nyquist frequency. At $N = 1024$, the corresponding physical frequency depends on the sampling rate; at $N = 4096$, the same f_c corresponds to a different number of spectral bins, altering the effective sharpness of the frequency transition in the discrete domain. In particular, a signal of length $N = 4096$ has $4\times$ the spectral resolution of an $N = 1024$ signal, meaning that the low-pass filter L operates on a finer frequency grid. The learned parameters λ_0^k, η^k, etc., were optimized

for the spectral characteristics at $N = 1024$ and may not be optimal at different resolutions.

Length-invariant components. Several components are inherently length-invariant. The element-wise soft-thresholding operator (Equation 3.14) applies independently to each sample and is unaffected by signal length. The first- and second-difference operators D_1, D_2 and their transposes (Equation 3.7–Equation 3.10) are implemented as vectorized slicing operations that naturally extend to any length. The log-space parameterization and the proximal gradient update rule (Equation 3.13) are length-agnostic.

Practical implications. At inference time, the filter matrices are recomputed for the input signal length, so the network can process signals of arbitrary length. However, the learned parameters were optimized for a specific spectral resolution and boundary condition regime. The primary risk is that $f_c = 0.006$ may be suboptimal at $N = 4096$: the finer spectral grid means that the low-pass filter has a narrower transition band (in physical frequency units), potentially producing sharper frequency separation than the training regime assumed. This could manifest as either improved or degraded baseline estimation, depending on the spectral content of the input signal.

Potential mitigations. Several approaches could address the length mismatch. Variable-length training, in which the training data includes signals of multiple lengths, would expose the network to diverse spectral resolutions and boundary conditions. Sliding-window inference, in which long signals are processed in overlapping segments of the training length and the results are blended, would avoid the filter recomputation issue entirely. A convolutional architecture that replaces the dense matrix filter with learned convolutional kernels would be inherently

length-invariant, at the cost of departing from the BEADS filter structure. These directions are left for future work.

6.3 The Conjugate Gradient Variant's Failure

The ISTA-style layer described in Section 3.2 was adopted after an earlier conjugate gradient (CG) variant failed to train effectively. This section provides a detailed analysis of the CG failure, as it yields design principles that are relevant to the algorithm unrolling community beyond the specific context of baseline correction.

6.3.1 Backpropagation Depth and Gradient Vanishing

The CG-based architecture (Section 3.3.1) solves the full BEADS linear system (Equation 3.15) at each of K outer layers using K_{inner} fixed CG iterations. Backpropagation through the trained network must differentiate through every CG iteration within every layer, producing a computational graph of effective depth $K \times K_{\text{inner}}$. With the training configuration of $K = 8$ outer layers and $K_{\text{inner}} = 12$ CG steps, this yields an effective backpropagation depth of 96 sequential differentiable operations.

Each CG iteration involves multiple matrix-vector products with the banded operators A, B, D_1, D_2 and their transposes, inner products for computing step sizes and conjugate directions, and scalar divisions. The Jacobian of the CG iterate at step $j + 1$ with respect to the iterate at step j depends on the intermediate CG state, specifically, on the search direction, residual, and step size at step j. The product of 12 such Jacobians within a single layer produces a composite Jacobian whose spectral radius can either grow (causing gradient explosion) or contract

(causing gradient vanishing), depending on the conditioning of the linear system.

In practice, the conditioning of $B^\top B + A^\top M^{(k)} A$ varies across training samples and across layers, because the reweighting matrix $M^{(k)}$ depends on the current peak estimate $\mathbf{x}^{k-1}$. This variability makes the gradient dynamics unpredictable: some samples produce well-conditioned systems with stable gradients, while others produce ill-conditioned systems with rapidly decaying gradients. The net effect, averaged over training batches, is gradient vanishing: the early layers receive gradient magnitudes that are orders of magnitude smaller than the later layers, effectively preventing them from learning meaningful parameter updates.

The ISTA variant eliminates this depth problem entirely. Each ISTA layer performs exactly one gradient step (Equation 3.13) followed by one element-wise proximal operation (Equation 3.14), yielding an effective backpropagation depth of $K = 20$, roughly $5\times$ shallower than the CG variant at $K = 8$, despite having $2.5\times$ more layers. The per-layer Jacobian of the ISTA update is dominated by the low-pass matrix L (a fixed linear operator) and the element-wise soft-thresholding function (whose Jacobian is a diagonal matrix of zeros and ones). The spectral properties of these operators are stable across samples and layers, producing consistent gradient flow throughout the network.

6.3.2 CG Convergence and Differentiability

A subtler issue with the CG variant is that the CG solver's convergence behavior is not smoothly differentiable. In exact arithmetic, CG converges to the exact solution in at most N iterations for an $N \times N$ system. In practice, $K_{\text{inner}} = 12$ iterations provide only an approximate solution, and the quality of this approximation depends on the condition number of the system matrix $B^\top B + A^\top M^{(k)} A$. The

approximation error is not a smooth function of the system matrix entries, small perturbations in $M^{(k)}$ can produce disproportionate changes in the CG trajectory when the system is near-singular.

This non-smoothness creates two problems for training. First, the loss landscape inherits the CG solver's sensitivity to conditioning, producing sharp ridges and valleys that gradient-based optimizers navigate poorly. Second, the CG residual norm, which is typically used as a convergence criterion in classical CG, is not differentiable at the point where the solver would terminate in an adaptive-stopping implementation. Although LBEADS-NET uses fixed-iteration CG (avoiding the non-differentiable stopping criterion), the underlying sensitivity to conditioning remains.

6.3.3 Design Principle for Algorithm Unrolling

The failure of the CG variant and the success of the ISTA variant yield a concrete design principle: *not all iterative algorithms unroll equally well, and the inner solver's gradient properties determine trainability.* Specifically:

- Algorithms with simple per-iteration operations (single gradient step, element-wise proximal operator) produce stable gradient flow and train effectively at depths of 20 or more layers.
- Algorithms with complex per-iteration operations (inner iterative solvers such as CG) produce deep computational graphs within each layer, causing gradient vanishing that limits effective depth to only a few layers.
- The design preference for unrolled networks should be: *simpler per-layer computations with more layers*, rather than complex per-layer computations

with fewer layers. The ISTA variant's 20 layers of simple updates outperform the CG variant's 8 layers of complex updates, both in training stability and in final decomposition quality.

This principle aligns with findings in the broader algorithm unrolling literature. Chen et al. [26] found that the number of unrolled iterations has a significantly greater impact on performance than the complexity of the neural network used within each iteration. The LBEADS-NET experience provides a complementary finding from the opposite direction: making the per-iteration computation *more* complex (CG solver vs. gradient step) degrades performance by limiting the achievable depth.

The practical implication for practitioners is clear: when unrolling an iterative algorithm that contains an inner solver (such as a linear system solve, an eigendecomposition, or a nested optimization), the inner solver should be replaced with a single-step approximation (such as a gradient step or a proximal operator) that maintains stable gradient flow. The loss in per-layer accuracy is more than compensated by the ability to train deeper networks with more effective parameter specialization.

6.4 Non-Learnable Cutoff Frequency and the Autograd Boundary

The Butterworth filter cutoff frequency f_c is arguably the most consequential hyperparameter in the BEADS formulation. It defines the spectral boundary between content attributed to peaks (above f_c) and content attributed to the baseline

(below f_c). In LBEADS-NET, $f_c = 0.006$ is a fixed, non-learnable hyperparameter. This section explains why, and discusses the implications.

The autograd boundary. The filter matrices A and B are constructed by the `BAfilt` function, which computes the Butterworth coefficient vectors **a** and **b** from f_c and the filter order d (Section 3.2.2). This computation uses SciPy's sparse matrix construction routines and NumPy array operations to build the banded Toeplitz matrices, which are then converted to dense PyTorch tensors and registered as non-learnable buffers. The conversion from SciPy to PyTorch crosses an *autograd boundary*: PyTorch's automatic differentiation engine cannot trace through SciPy operations, so the gradient $\partial\mathcal{L}/\partial f_c$ is undefined. Without a gradient, f_c cannot be updated by backpropagation.

Why this matters for leakage. The fixed cutoff frequency interacts directly with the baseline leakage mechanism. Consider two signals processed by the same trained network: one with narrow, well-separated peaks whose spectral content is concentrated above f_c, and another with broad, densely packed peaks whose spectral content extends below f_c. For the first signal, the frequency separation between peaks and baseline is clean, and the low-pass filter correctly assigns low-frequency content to the baseline. For the second signal, the low-frequency tails of broad peaks are captured by the low-pass filter and erroneously attributed to the baseline, a direct mechanism for leakage that operates independently of the ℓ_1 penalty.

A learnable f_c could in principle adapt the frequency boundary to the spectral characteristics of each input signal. For signals with broad peaks, the network could learn a lower f_c, preserving more peak energy in the high-pass band. For signals with narrow peaks and a rapidly varying baseline, it could learn a higher f_c, capturing more baseline variation. The current fixed f_c represents a compromise

that is suboptimal for both extremes.

Connection to spatially varying f_c. An even more expressive extension would be a spatially varying cutoff frequency $f_c(n)$ that adapts to local signal characteristics at each position n. Dense-peak regions would use a lower f_c to avoid absorbing peak energy, while baseline-only regions would use a higher f_c to capture all baseline variation. This would address Failure Mode 3 (spatially uniform regularization) at the filter level, complementing any spatial adaptivity in the regularization weights. However, a spatially varying cutoff frequency would require replacing the Toeplitz-structured Butterworth filter with a non-stationary filter, fundamentally changing the BEADS formulation.

Potential solutions. Three approaches could make the filter cutoff learnable, as outlined in Section 3.5: (i) implementing a differentiable IIR filter design entirely within PyTorch, replacing the SciPy coefficient computation with differentiable arithmetic on the frequency-warping parameter t (Equation 2.6) and the coefficient polynomials; (ii) replacing the Butterworth filter with learned convolutional kernels trained end-to-end; or (iii) using a lightweight auxiliary network to predict f_c from signal features such as the spectral energy distribution. Each approach involves trade-offs between expressiveness, interpretability, and computational cost. The first approach preserves the Butterworth structure and is the most natural extension of the current architecture; the second sacrifices the Butterworth interpretation but gains full flexibility; the third preserves interpretability but introduces a secondary model that must be jointly trained.

6.5 Softplus Approximation , Train/Inference Gap

The v7 development iteration introduced a softplus activation to enforce non-negativity of the peak estimate. The softplus function is defined as

$$\text{softplus}(x;\beta) = \frac{1}{\beta}\ln(1+e^{\beta x}), \tag{6.1}$$

which approximates $\max(0,x)$ (the ReLU function) for large β. With $\beta = 5$, as used in the v7 architecture, the approximation is close but imperfect: $\text{softplus}(0;5) = \frac{\ln 2}{5} \approx 0.139$, compared to $\text{ReLU}(0) = 0$.

Training advantage. During training, the softplus activation provides a smooth gradient at and near the zero crossing, enabling gradient flow through samples whose true peak value is zero. A hard ReLU would produce zero gradient for all negative inputs, preventing the optimizer from adjusting parameters that affect the peak estimate in baseline-only regions. The softplus activation thus improves trainability at the cost of a small positive bias.

Inference artifact. At inference time, the positive bias $\ln(2)/\beta \approx 0.139$ (for $\beta = 5$) manifests as a small pedestal in the peak estimate at positions where the true peak value is zero. In baseline-only regions, the network's pre-activation output is near zero or slightly negative; the softplus maps these values to approximately 0.139 rather than exactly zero. This pedestal is small relative to peak amplitudes (which are order unity in the normalized signal) but is systematic: it affects every sample in every baseline-only region.

The pedestal has two consequences. First, it inflates the computed peak

area, introducing a length-dependent positive bias (proportional to the number of baseline-only samples). Second, it reduces the apparent sparsity of the peak estimate, because baseline-only samples are mapped to a small positive value rather than exactly zero. Both effects are minor for well-separated peaks with large amplitudes, but they become measurable for low-amplitude peaks near the noise floor.

General lesson. The softplus train/inference gap illustrates a broader challenge in algorithm unrolling: differentiable relaxations of non-smooth operators introduce systematic biases that do not cancel between training and inference. In the proximal gradient framework, the canonical proximal operators (soft thresholding, projection onto non-negative orthant) are non-smooth, and their gradients are discontinuous at the origin. Replacing them with smooth approximations improves trainability but introduces a train/test mismatch whose magnitude depends on the approximation tightness. Practitioners should be aware of this trade-off and validate that the bias introduced by smooth approximations is acceptable for the downstream application. Increasing β reduces the bias but steepens the gradient near zero, potentially reintroducing the vanishing gradient problem that the softplus was intended to resolve.

6.6 The Over-Regularization Problem

Experiment 1.5 (Section 5.1.3) demonstrated that the full v7 configuration with twelve active loss terms performed worse than several simpler configurations on multiple metrics. This section analyzes the over-regularization phenomenon from a multi-objective optimization perspective and extracts practical lessons for loss

design in algorithm unrolling.

Non-monotonic relationship between loss complexity and model quality. The progressive experiment results reveal a clear non-monotonic relationship. Performance improves from Experiment 1.1 (1 loss term) through Experiment 1.3 (6 loss terms), reaching optimal peak correlation at 0.793 with the addition of baseline supervision. Beyond this point, adding orthogonality (Experiment 1.4, 8 terms) produces marginal improvement in baseline quality (BLI 0.592 vs. 0.596) at the cost of slight peak correlation degradation (0.785 vs. 0.793). The full v7 configuration (Experiment 1.5, 12 terms) degrades on four of seven metrics relative to Experiment 1.3.

This non-monotonic pattern is consistent with the multi-objective optimization literature. Each loss term defines a separate objective, and the gradient of the total loss is a weighted sum of per-term gradients. When the per-term gradients point in compatible directions (as with MSE and baseline supervision, which both encourage accurate decomposition), their combination accelerates convergence. When they point in conflicting directions (as with the ℓ_1 penalty, which encourages sparse peaks, and the asymmetric baseline loss, which discourages baseline over-estimation), their combination can produce a gradient field with saddle points and narrow valleys that the optimizer navigates poorly.

The training budget constraint. The over-regularization effect is exacerbated by the limited training budget. With 30 training epochs (10 gradient steps per epoch, totaling 300 gradient updates), the optimizer has insufficient time to navigate the twelve-dimensional trade-off surface. The curriculum training strategy (Section 3.6) was designed to address this by introducing loss terms in stages, allowing the model to settle on a good solution for simpler objectives before confronting harder ones.

However, the progressive experiments used a uniform training protocol (all terms active from epoch 1) to isolate the contribution of each addition, which exposed the over-regularization effect.

Practical lesson: match loss complexity to training budget. The over-regularization finding carries a direct practical lesson for algorithm unrolling: the number of active loss terms should be matched to the available training budget. With a limited budget (tens of epochs), a simpler loss configuration with three to six well-chosen terms outperforms a complex configuration with twelve terms. The marginal benefit of each additional loss term decreases as the total number of terms increases, while the marginal cost (in terms of optimization difficulty) increases. The optimal operating point is problem-specific and depends on the number of training epochs, the learning rate schedule, the batch size, and the degree of conflict among the loss terms.

This observation connects to the broader question of curriculum design in multi-objective optimization. The three-stage curriculum (Section 3.6), pure reconstruction, then structured separation, then full refinement, provides one principled approach to managing loss complexity. Alternative strategies, such as loss-weight annealing (gradually increasing the weights of anti-leakage terms), Pareto-optimal loss balancing (dynamically adjusting weights to maintain progress on all objectives), or population-based training (searching over loss weight configurations), could further improve the trade-off between loss complexity and training efficiency.

Chapter 7

Conclusions and Future Work

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Appendix A

APPENDIX TITLE

The following includes copies of .....

Bibliography

[1] D. A. Skoog, D. M. West, F. J. Holler, and S. R. Crouch, *Fundamentals of Analytical Chemistry*, 9th ed. Cengage Learning, 2014.

[2] X. Ning, I. W. Selesnick, and L. Duval, "Chromatogram baseline estimation and denoising using sparsity (BEADS)," *Chemometrics and Intelligent Laboratory Systems*, vol. 139, pp. 156–167, 2014.

[3] P. H. C. Eilers and H. F. M. Boelens, "Baseline correction with asymmetric least squares smoothing," Leiden University Medical Centre, Tech. Rep., 2005.

[4] S.-J. Baek, A. Park, Y.-J. Ahn, and J. Choo, "Baseline correction using asymmetrically reweighted penalized least squares smoothing," *Analyst*, vol. 140, no. 1, pp. 250–257, 2015.

[5] Z.-M. Zhang, S. Chen, and Y.-Z. Liang, "Baseline correction using adaptive iteratively reweighted penalized least squares," *Analyst*, vol. 135, no. 5, pp. 1138–1146, 2010.

[6] M. Morháč and V. Matoušek, "Peak clipping algorithms for background estimation in spectroscopic data," *Applied Spectroscopy*, vol. 62, no. 1, pp. 91–106, 2008.

[7] A. Kensert, G. Collaerts, K. Efthymiadis, T. S. Bos, C. Bruun Andersen, M. Degerman, and P. Ström, "Deep convolutional autoencoder for the simultaneous removal of baseline noise and baseline drift in chromatographic data," *Journal of Chromatography A*, vol. 1646, p. 462093, 2021.

[8] T. Chen, Z. Luo, J.-N. He, and J.-G. Xu, "Automatic baseline correction of raman spectra using a deep residual network combined with U-Net," *Analyst*, vol. 147, no. 18, pp. 4126–4134, 2022.

[9] J. Zhao, Y. Lu, S. Guo, J. He, Y. Liu, and G. Song, "1dtrans: An efficient baseline correction method for raman spectra based on the Transformer architecture," *Spectrochimica Acta Part A: Molecular and Biomolecular Spectroscopy*, 2025.

[10] Y. Hu, B. Fan, Y. Liu, G. He, R. Yin, J. Li, and J. Du, "RSPSSL: A raman spectral pre-processing method based on self-supervised learning," *Light: Science & Applications*, vol. 13, p. 248, 2024.

[11] H. Gharbi, E. Chouzenoux, and J.-C. Pesquet, "Unrolled deep networks for sparse signal restoration in analytical chemistry," in *IEEE International Workshop on Machine Learning for Signal Processing (MLSP)*, 2024.

[12] X. Ding, J. Kang, G. Li, M. Wu, and Z. Zhou, "Adaptive physics-aware Raman baseline correction with machine learning predicted parameters," *Analytical Chemistry*, 2025.

[13] K. Gregor and Y. LeCun, "Learning fast approximations of sparse coding," in *Proceedings of the 27th International Conference on Machine Learning (ICML)*, 2010, pp. 399–406.

[14] V. Monga, Y. Li, and Y. C. Eldar, "Algorithm unrolling: Interpretable, efficient deep learning for signal and image processing," *IEEE Signal Processing Magazine*, vol. 38, no. 2, pp. 18–44, 2021.

[15] J. Zhang and B. Ghanem, "ISTA-Net: Interpretable optimization-inspired deep network for image compressive sensing," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 1828–1837.

[16] Y. Yang, J. Sun, H. Li, and Z. Xu, "ADMM-CSNet: A deep learning approach for image compressive sensing," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 3, pp. 521–538, 2020.

[17] E. Grushka, "Characterization of exponentially modified Gaussian peaks in chromatography," *Analytical Chemistry*, vol. 44, no. 11, pp. 1733–1738, 1972.

[18] P. J. Naish and S. Hartwell, "Exponentially modified Gaussian functions—a good model for chromatographic peaks in isocratic HPLC?" *Chromatographia*, vol. 26, pp. 285–296, 1988.

[19] P. H. C. Eilers, "A perfect smoother," *Analytical Chemistry*, vol. 75, no. 14, pp. 3631–3636, 2003.

[20] C. G. Ryan, E. Clayton, W. L. Griffin, S. H. Sie, and D. R. Cousens, "SNIP, a statistics-sensitive background treatment for the quantitative analysis of PIXE spectra in geoscience applications," *Nuclear Instruments and Methods in Physics Research Section B*, vol. 34, no. 3, pp. 396–402, 1988.

[21] J. Liu, X. Chen, Z. Wang, and W. Yin, "ALISTA: Analytic weights are as good

as learned weights in LISTA," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019.

[22] D. You, J. Xie, and J. Zhang, "ISTA-Net++: Flexible deep unfolding network for compressive sensing," in *Proceedings of the IEEE International Conference on Multimedia and Expo (ICME)*, 2021, pp. 1–6.

[23] Z. Zheng, W. Dai, D. Xue, C. Li, J. Zou, and H. Xiong, "Hybrid ISTA: Unfolding ISTA with convergence guarantees using free-form deep neural networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 5, pp. 5985–6000, 2023.

[24] Y. Yang, J. Sun, H. Li, and Z. Xu, "Deep ADMM-Net for compressive sensing MRI," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016, pp. 10–18.

[25] S. Diamond, V. Sitzmann, F. Heide, and G. Wetzstein, "Unrolled optimization with deep priors," *arXiv preprint arXiv:1705.08041*, 2018.

[26] Y. Chen, J. Chen, A. Maleki, and S. Jalali, "Comprehensive examination of unrolled networks for solving linear inverse problems," *Entropy*, vol. 27, no. 1, p. 85, 2025.

[27] H. Gharbi, C. Villa, E. Chouzenoux, and J.-C. Pesquet, "Unrolled deep networks for sparse signal restoration," in *HAL Preprint (hal-03988686v2)*, 2023.

[28] Q. Han, Y. Dang, and Q. Han, "Denoising and baseline correction methods for Raman spectroscopy based on convolutional autoencoder: A unified solution," *Sensors*, vol. 24, no. 11, p. 3468, 2024.

[29] T. Chen, H. Son, B. Park, and S.-J. Baek, "Baseline correction using a deep-learning model combining ResNet and UNet," *Analyst*, vol. 147, no. 18, pp. 4285–4292, 2022.

[30] A. Aradhye, S. Mandal, I. Gray, and J. Campbell, "Adaptive physics-aware Raman baseline correction with machine learning predicted parameters," *Analytical Chemistry*, 2025.

[31] H. Cui, Y. Chen, and J. Zhao, "Beyond traditional airPLS: Improved baseline removal in SERS with parameter-focused optimization and prediction," *Analytical Chemistry*, 2025.

[32] I. Daubechies, M. Defrise, and C. De Mol, "An iterative thresholding algorithm for linear inverse problems with a sparsity constraint," *Communications on Pure and Applied Mathematics*, vol. 57, no. 11, pp. 1413–1457, 2004.

[33] A. Beck and M. Teboulle, "A fast iterative shrinkage-thresholding algorithm for linear inverse problems," *SIAM Journal on Imaging Sciences*, vol. 2, no. 1, pp. 183–202, 2009.

[34] G. Hacohen and D. Weinshall, "On the power of curriculum learning in training deep networks," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019, pp. 2535–2544.

[35] P. J. Huber, "Robust estimation of a location parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.

[36] C. Ding, T. Li, W. Peng, and H. Park, "Orthogonal nonnegative matrix tri-factorizations for clustering," in *Proceedings of the 12th ACM SIGKDD*

International Conference on Knowledge Discovery and Data Mining, 2006, pp. 126–135.

[37] F. Pompili, N. Gillis, P.-A. Absil, and F. Glineur, "Two algorithms for orthogonal nonnegative matrix factorization with application to clustering," *Neurocomputing*, vol. 141, pp. 15–25, 2014.

[38] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.